actuarial-science-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `actuarial_science_ai999x.about()` for the same answer.

Every capability actuarial-science-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import actuarial_science_ai999x

result = actuarial_science_ai999x.solve('act_interest_rate_conversion', i=0.05, m=12)
result.show()
# d = 0.047619, v = 0.952381, delta = 0.0487902, i_m = 0.0488895, d_m = 0.0486911
```

By topic

topic	capabilities
Credibility / Bayesian credibility	Poisson-gamma Bayesian credibility
Credibility / Buhlmann	Buhlmann credibility premium (known structural parameters)
Credibility / empirical Bayes	Empirical Buhlmann credibility from a data table, Buhlmann-Straub credibility with exposures
Credibility / limited fluctuation	Limited fluctuation full credibility standard for claim counts, Limited fluctuation full credibility for severity and aggregate losses, Partial credibility (square root rule)
Interest theory / annuities-certain	Annuity-certain present and accumulated values, Present value of a perpetuity
Interest theory / cash flows	Present value of a series of cash flows
Interest theory / compound interest	Accumulated and present value of a single amount
Interest theory / duration and immunization	Macaulay and modified duration and convexity of cash flows
Interest theory / loans	Loan amortization: level payment, interest and principal, outstanding balance
Interest theory / rates of interest	Equivalent rates from an effective annual interest rate, Effective annual rate from a nominal rate, Effective rates from a constant force of interest
Interest theory / varying annuities	Increasing and decreasing annuities-certain
Interest theory / yield rates	Yield rate of a level annuity
Life annuities / deferred annuities	Deferred whole life annuity-due
Life annuities / m-thly payments	M-thly whole life annuity-due under UDD, Woolhouse approximation for an m-thly annuity

topic	capabilities
Life annuities / temporary annuities	Temporary life annuity
Life annuities / whole life	Whole life annuity
Life insurance / continuous benefits	Continuous whole life insurance and annuity under UDD
Life insurance / deferred insurance	Deferred whole life insurance
Life insurance / endowment insurance	Endowment insurance (fully discrete)
Life insurance / term insurance	Term insurance (fully discrete)
Life insurance / whole life	Whole life insurance (fully discrete)
Life tables / central death rate	Death probability from the central death rate
Life tables / commutation functions	Commutation functions D, N, C and M
Life tables / expectation of life	Curtate and complete expectation of life, Temporary (n-year) expectation of life
Life tables / fractional ages	Survival to a fractional age (UDD, constant force, Balducci)
Life tables / life table functions	Life table functions dx, qx and px from survivor numbers
Life tables / pure endowment	Pure endowment (actuarial discount factor)
Life tables / survival probabilities	Probability of surviving t years from a life table, Deferred death probability u|t q x
Loss models / aggregate losses	Mean and variance of aggregate losses (compound distribution), Compound Poisson mean, variance and skewness, Normal approximation to aggregate losses
Loss models / coverage modifications	Deductible, policy limit, coinsurance and inflation with exponential losses, Deductible, policy limit, coinsurance and inflation with Pareto losses, Insurer payment for one loss (deductible, limit, coinsurance)
Loss models / deductibles	Ordinary deductible with exponential losses, Ordinary deductible with Pareto losses, Ordinary deductible with lognormal losses
Loss models / frequency distributions	Poisson claim frequency probabilities, Negative binomial claim frequency probabilities
Loss models / individual risk model	Individual risk model: aggregate mean and variance
Loss models / Panjer recursion	Panjer recursion for a compound Poisson distribution, Panjer recursion for a compound negative binomial distribution
Loss models / premium principles	Premium principles: expected value, variance, standard deviation
Loss models / stop-loss insurance	Stop-loss premium for a compound Poisson distribution
Loss reserving / Bornhuetter-Ferguson	Bornhuetter-Ferguson reserve
Loss reserving / Cape Cod	Cape Cod (Stanard-Buhlmann) reserve
Loss reserving / chain-ladder	Chain-ladder age-to-age and cumulative development factors, Chain-ladder ultimate losses and reserves, Chain-ladder with simple-average development factors

topic	capabilities
Loss reserving / expected claims	Expected claims reserving method
Loss reserving / Mack chain-ladder	Mack's standard error of chain-ladder reserves
Multiple decrements / associated single decrement	Associated single decrement rates from a multiple decrement table (UDD)
Multiple decrements / multiple decrement tables	Multiple decrement probabilities from single decrement rates (UDD)
Multiple lives / joint life and last survivor	Joint life and last survivor annuities and insurances
Multiple lives / reversionary annuities	Reversionary annuity
Premiums / gross premiums	Gross annual premium for whole life insurance with expenses
Premiums / loss random variable	Loss at issue of whole life insurance: mean, standard deviation, probability of loss
Premiums / net premiums	Net annual premium for whole life insurance, Net annual premium for term insurance, Net annual premium for endowment insurance, Net annual premium for limited-payment whole life insurance
Ratemaking / loss ratio method	Indicated rate change by the loss ratio method
Ratemaking / pure premium method	Indicated rate by the pure premium method
Reserves / gross premium reserves	Gross premium reserve for whole life insurance with expenses
Reserves / prospective reserves	Net premium reserve for whole life insurance, Net premium reserve for endowment insurance, Net premium reserve for term insurance
Reserves / recursion	Policy value recursion (one year)
Reserves / retrospective reserves	Retrospective reserve (accumulated fund per survivor)
Reserves / Thiele's differential equation	Thiele's equation: continuous endowment insurance reserve (constant forces), Continuous whole life insurance, premium and reserve under Makeham's law
Risk measures / VaR and TVaR	VaR and TVaR of a normal loss, VaR and TVaR of an exponential loss, VaR and TVaR of a Pareto loss, VaR and TVaR of a lognormal loss
Survival estimation / Kaplan-Meier	Kaplan-Meier product-limit survival estimate
Survival estimation / Nelson-Aalen	Nelson-Aalen cumulative hazard estimate
Survival estimation / parametric fitting	Exponential maximum-likelihood fit with censoring
Survival models / parametric laws	De Moivre's law of mortality, Constant force of mortality (exponential lifetime), Makeham's law of mortality, Gompertz's law of mortality

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
act_interest_rate_conversion	Equivalent rates from an effective annual interest rate	i, m	d, v, delta, i_m, d_m
act_nominal_to_effective_rate	Effective annual rate from a nominal rate	rate, m, kind	i, d, delta, v
act_force_of_interest	Effective rates from a constant force of interest	delta	i, d, v
act_accumulated_value	Accumulated and present value of a single amount	P, i, t	AV, PV, factor
act_annuity_certain	Annuity-certain present and accumulated values	n, i	a_n, a_due, s_n, s_due, a_cont
act_perpetuity	Present value of a perpetuity	i	a_inf, a_due_inf, a_cont_inf
act_increasing_decreasing_annuity	Increasing and decreasing annuities-certain	n, i	Ia, Ia_due, Da, Da_due
act_loan_amortization	Loan amortization: level payment, interest and principal, outstanding balance	L, i, n, t	R, interest_t, principal_t, balance_t, total_interest
act_annuity_yield	Yield rate of a level annuity	PV, R, n	i, a_n
act_present_value_cash_flows	Present value of a series of cash flows	amounts, times, i	PV, AV_at_last
act_duration_convexity	Macaulay and modified duration and convexity of cash flows	amounts, times, i	PV, macaulay, modified, convexity
act_life_table_functions	Life table functions dx, qx and px from survivor numbers	lx	dx, qx, px
act_survival_probability	Probability of surviving t years from a life table	lx, x0, x, t	tpx, tqx, deaths
act_deferred_death_probability	Deferred death probability u|t q x	lx, x0, x, u, t	prob, upx
act_life_expectancy	Curtate and complete expectation of life	lx, x0, x	e_curtate, e_complete
act_temporary_expectation	Temporary (n-year) expectation of life	lx, x0, x, n	e_curtate, e_complete
act_central_death_rate	Death probability from the central death rate	m, assumption	q, p
act_fractional_age	Survival to a fractional age (UDD, constant force, Balducci)	q, s, assumption	spx, sqx, mu
act_commutation_functions	Commutation functions D, N, C and M	lx, x0, x, i	D_x, N_x, C_x, M_x, A_x, a_due_x
act_pure_endowment	Pure endowment (actuarial discount factor)	lx, x0, x, n, i	nEx, accumulation
act_de_moivre_law	De Moivre's law of mortality	omega, x, t	tpx, tqx, mu, e_complete, var_T
act_constant_force_mortality	Constant force of mortality (exponential lifetime)	mu, t	tpx, tqx, e_complete, e_curtate, var_T
act_makeham_law	Makeham's law of mortality	A, B, c, x, t	tpx, tqx, mu_x, mu_xt, e_complete
act_gompertz_law	Gompertz's law of mortality	B, c, x, t	tpx, tqx, mu_x, mu_xt, e_complete

id	operation	inputs	outputs
act_whole_life_insurance	Whole life insurance (fully discrete)	q, i, S	A, A2, EPV, sd
act_term_insurance	Term insurance (fully discrete)	q, i, n, S	A1, A1_2, EPV, sd
act_endowment_insurance	Endowment insurance (fully discrete)	q, i, n, S	A, A1, nEx, A2, EPV, sd
act_deferred_insurance	Deferred whole life insurance	q, i, u, S	uA, EPV, uEx
act_whole_life_annuity	Whole life annuity	q, i, P	a_due, a_imm, EPV, sd
act_temporary_annuity	Temporary life annuity	q, i, n, P	a_due, a_imm, EPV
act_deferred_annuity	Deferred whole life annuity-due	q, i, u, P	ua_due, EPV
act_continuous_insurance_udd	Continuous whole life insurance and annuity under UDD	q, i	A_bar, A, a_bar
act_mthly_annuity_udd	m-thly whole life annuity-due under UDD	q, i, m	a_m, alpha, beta, a_due
act_woolhouse_approximation	Woolhouse approximation for an m-thly annuity	a_due, m, i, mu	a_m3, a_m2
act_net_premium_whole_life	Net annual premium for whole life insurance	q, i, S	P, A, a_due
act_net_premium_term	Net annual premium for term insurance	q, i, n, S	P, A1, a_due
act_net_premium_endowment	Net annual premium for endowment insurance	q, i, n, S	P, A, a_due
act_net_premium_limited_pay	Net annual premium for limited-payment whole life insurance	q, i, h, S	P, A, a_due_h
act_gross_premium	Gross annual premium for whole life insurance with expenses	q, i, S, e0, r, c, E	G, P, loading
act_loss_at_issue	Loss at issue of whole life insurance: mean, standard deviation, probability of loss	q, i, S, P	mean, sd, prob_loss
act_reserve_whole_life	Net premium reserve for whole life insurance	q, i, S, t	V, P, A_t, a_due_t
act_reserve_endowment	Net premium reserve for endowment insurance	q, i, n, S, t	V, P
act_reserve_term	Net premium reserve for term insurance	q, i, n, S, t	V, P
act_retrospective_reserve	Retrospective reserve (accumulated fund per survivor)	q, i, S, P, t	V, tEx
act_policy_value_recursion	Policy value recursion (one year)	V, P, e, i, q, S, E	V_next
act_gross_premium_reserve	Gross premium reserve for whole life insurance with expenses	q, i, S, e0, r, c, E, t	V, G
act_thiele_endowment	Thiele's equation: continuous endowment insurance reserve (constant forces)	mu, delta, n, S, t	P, A_bar, a_bar, V
act_makeham_continuous_whole_life	Continuous whole life insurance, premium and reserve under Makeham's law	A, B, c, x, i, S, t	a_bar, A_bar, P, V

id	operation	inputs	outputs
act_joint_life_last_survivor	Joint life and last survivor annuities and insurances	qx, qy, i	a_joint, a_last, A_joint, A_last
act_reversionary_annuity	Reversionary annuity	qx, qy, i, P	a_rev, EPV
act_multiple_decrement_to_single	Associated single decrement rates from a multiple decrement table (UDD)	q1, q2	q1_single, q2_single, q_total
act_single_to_multiple_decrement	Multiple decrement probabilities from single decrement rates (UDD)	q1s, q2s	q1, q2, q_total
act_poisson_frequency	Poisson claim frequency probabilities	lam, k	pmf, cdf, mean, var
act_negative_binomial_frequency	Negative binomial claim frequency probabilities	r, beta, k	pmf, cdf, mean, var
act_compound_distribution_moments	Mean and variance of aggregate losses (compound distribution)	EN, VarN, EX, VarX	ES, VarS
act_compound_poisson_moments	Compound Poisson mean, variance and skewness	lam, m1, m2, m3	mean, var, skew
act_individual_risk_model	Individual risk model: aggregate mean and variance	q, b	mean, var, prob_no_claim
act_normal_approximation_aggregate	Normal approximation to aggregate losses	mean, var, s	prob_exceed, z
act_panjer_poisson	Panjer recursion for a compound Poisson distribution	lam, f, smax	g, F_smax
act_panjer_negative_binomial	Panjer recursion for a compound negative binomial distribution	r, beta, f, smax	g, F_smax
act_stop_loss_premium	Stop-loss premium for a compound Poisson distribution	lam, f, d	premium, ES, limited
act_exponential_deductible	Ordinary deductible with exponential losses	theta, d	mean, lev_d, per_loss, per_payment, LER
act_pareto_deductible	Ordinary deductible with Pareto losses	alpha, theta, d	mean, lev_d, per_loss, per_payment, LER
act_lognormal_deductible	Ordinary deductible with lognormal losses	mu, sigma, d	mean, lev_d, per_loss, per_payment, LER
act_exponential_coverage_modifications	Deductible, policy limit, coinsurance and inflation with exponential losses	theta, d, u, c, r	per_loss, per_payment, max_payment
act_pareto_coverage_modifications	Deductible, policy limit, coinsurance and inflation with Pareto losses	alpha, theta, d, u, c, r	per_loss, per_payment, max_payment
act_single_loss_payment	Insurer payment for one loss (deductible, limit, coinsurance)	x, d, u, c	payment, retained
act_var_tvar_normal	VaR and TVaR of a normal loss	mu, sigma, p	VaR, TVaR
act_var_tvar_exponential	VaR and TVaR of an exponential loss	theta, p	VaR, TVaR
act_var_tvar_pareto	VaR and TVaR of a Pareto loss	alpha, theta, p	VaR, TVaR

id	operation	inputs	outputs
act_var_tvar_lognormal	VaR and TVaR of a lognormal loss	mu, sigma, p	VaR, TVaR
act_premium_principles	Premium principles: expected value, variance, standard deviation	mean, var, theta, principle	premium, loading
act_limited_fluctuation_full_credibility	Limited fluctuation full credibility standard for claim counts	p, k	n_full, z
act_limited_fluctuation_severity_aggregate	Limited fluctuation full credibility for severity and aggregate losses	p, k, mean_x, sd_x	claims_aggregate, claims_severity, claims_frequency
act_partial_credibility	Partial credibility (square root rule)	n, n_full, xbar, M	Z, premium
act_buhlmann_credibility	Buhlmann credibility premium (known structural parameters)	mu, EPV, VHM, n, xbar	k, Z, premium
act_buhlmann_empirical	Empirical Buhlmann credibility from a data table	X	EPV, VHM, k, Z, premiums, mean
act_buhlmann_straub	Buhlmann-Straub credibility with exposures	X, m	EPV, VHM, k, Z, premiums, mean
act_poisson_gamma_credibility	Poisson-gamma Bayesian credibility	alpha, theta, counts	post_mean, Z, prior_mean, post_alpha, post_theta
act_chain_ladder_factors	Chain-ladder age-to-age and cumulative development factors	T	ldf, cdf
act_chain_ladder_reserve	Chain-ladder ultimate losses and reserves	T	ultimate, reserve, total_reserve
act_chain_ladder_simple_average	Chain-ladder with simple-average development factors	T	ldf, ultimate, total_reserve
act_bornhuetter_ferguson	Bornhuetter-Ferguson reserve	T, P, elr	ultimate, reserve, total_reserve
act_cape_cod	Cape Cod (Stanard-Buhlmann) reserve	T, P	elr, ultimate, reserve, total_reserve
act_mack_standard_error	Mack's standard error of chain-ladder reserves	T	se, total_se, sigma2
act_expected_claims_method	Expected claims reserving method	P, elr, paid	ultimate, reserve, total_reserve
act_loss_ratio_rate_change	Indicated rate change by the loss ratio method	L, EP, F, V, Q	change, loss_ratio, permissible
act_pure_premium_rate	Indicated rate by the pure premium method	L, X, Fe, V, Q	pure_premium, rate
act_kaplan_meier	Kaplan-Meier product-limit survival estimate	times, events, t	S, F, deaths
act_nelson_aalen	Nelson-Aalen cumulative hazard estimate	times, events, t	H, S
act_exponential_mle	Exponential maximum-likelihood fit with censoring	times, events, t	hazard, mean, S

Equivalent rates from an effective annual interest rate, `act_interest_rate_conversion`

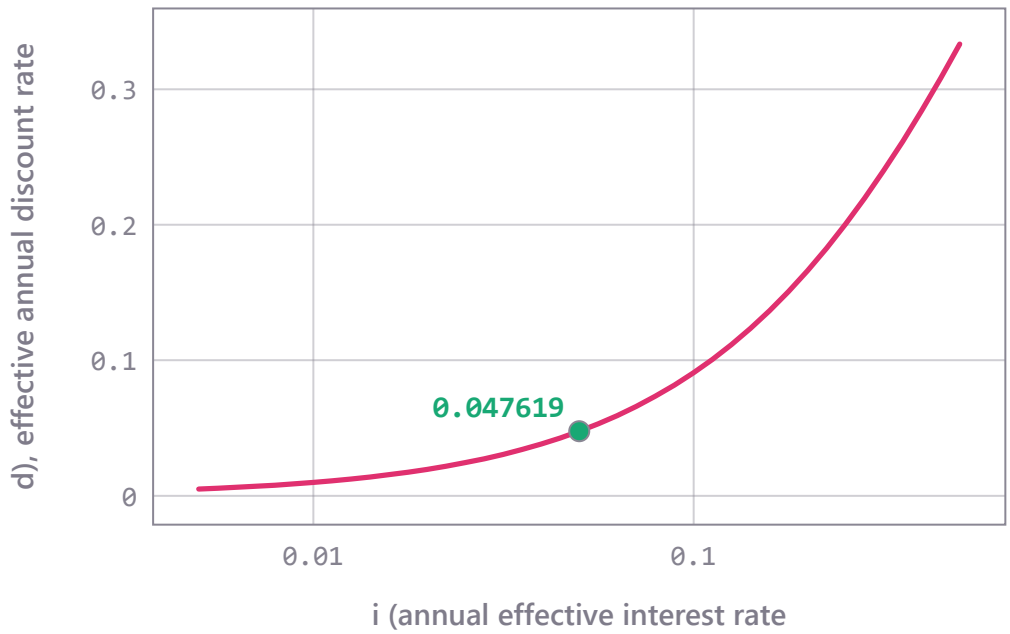
The effective discount rate, discount factor, force of interest and the nominal interest and discount rates convertible m times a year equivalent to an annual effective interest rate.

$$d = \frac{i}{1+i},\ \delta = \ln(1+i),\ i^{(m)} = m\left[(1+i)^{1/m} - 1\right],\ d^{(m)} = m\left[1 - v^{1/m}\right]$$

input	meaning	condition
`i`	annual effective interest rate (5% or 0.05)	
`m`	conversion periods per year for the nominal rates (12 = monthly)	

output	meaning
`d`	effective annual discount rate
`v`	annual discount factor $1/(1+i)$
`delta`	force of interest ln(1+i)
`i_m`	nominal interest rate convertible m-thly
`d_m`	nominal discount rate convertible m-thly

- **Method:** $d = i/(1+i)$, $v = 1/(1+i)$, $\delta = ln$(1+i), i(m) = m[$(1+i)^{1/m} - 1$], d(m) = m[$1 - v^{1/m}$]
- **Assumes:** $0 < i < 1$; $1 \le m \le 365$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



d (effective annual discount rate) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: m = 12. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, d = 0.047619.

```
actuarial_science_ai999x.solve('act_interest_rate_conversion', i=0.05, m=12)
# -> d = 0.047619, v = 0.952381, delta = 0.0487902, i_m = 0.0488895, d_m = 0.0486911   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_interest_rate_conversion', i=0.08, m=4)
# -> d = 0.0740741, v = 0.925926, delta = 0.076961, i_m = 0.0777062, d_m = 0.0762254   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Interest rate conversion: i = 0.05, m = 12	SOLVED	d = 0.047619, v = 0.952381, delta = 0.0487902, i_m = 0.0488895, d_m = 0.0486911
Interest rate conversion: i = 0.08, m = 4	SOLVED	d = 0.0740741, v = 0.925926, delta = 0.076961, i_m = 0.0777062, d_m = 0.0762254
Interest rate conversion: m = 12	NEEDS_INPUT	act_interest_rate_conversion needs i (annual effective interest rate (5% or 0.05)). Example: solve('act_interest_rate_co
Interest rate conversion: i = 0.05, m = 12, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Equivalent rates: i = 5%, m = 12	SOLVED	d = 0.047619, v = 0.952381, delta = 0.0487902, i_m = 0.0488895, d_m = 0.0486911
Interest rate conversion: i = 5, m = 12	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Effective annual rate from a nominal rate, `act_nominal_to_effective_rate`

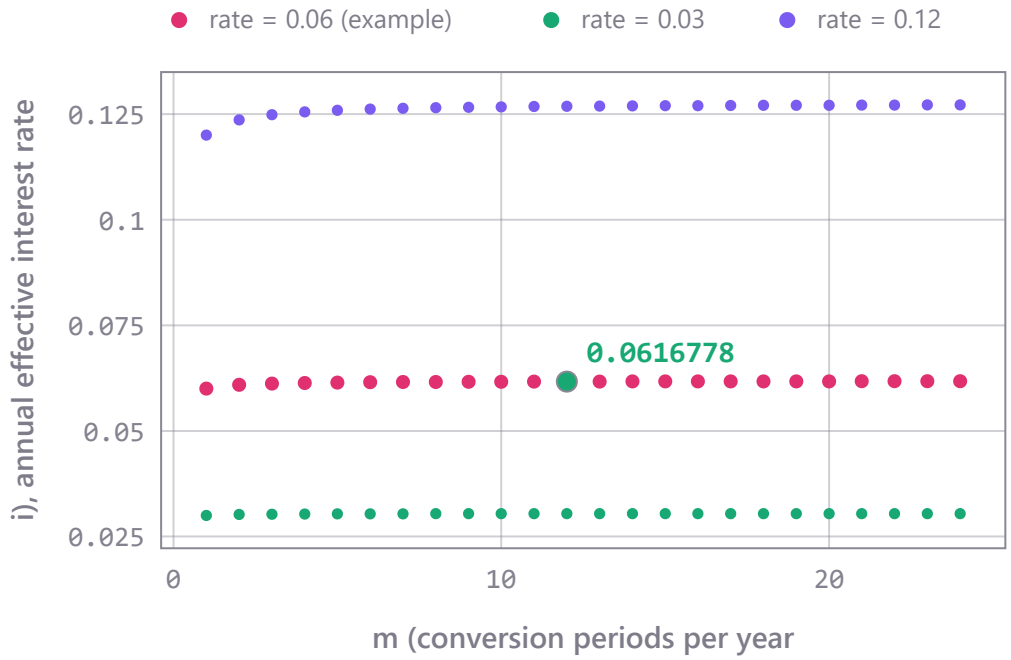
The annual effective interest and discount rates, force of interest and discount factor equivalent to a nominal interest or discount rate convertible m times a year.

$$1+i=\left(1+\frac{i^{(m)}}{m}\right)^{m}=\left(1-\frac{d^{(m)}}{m}\right)^{-m}$$

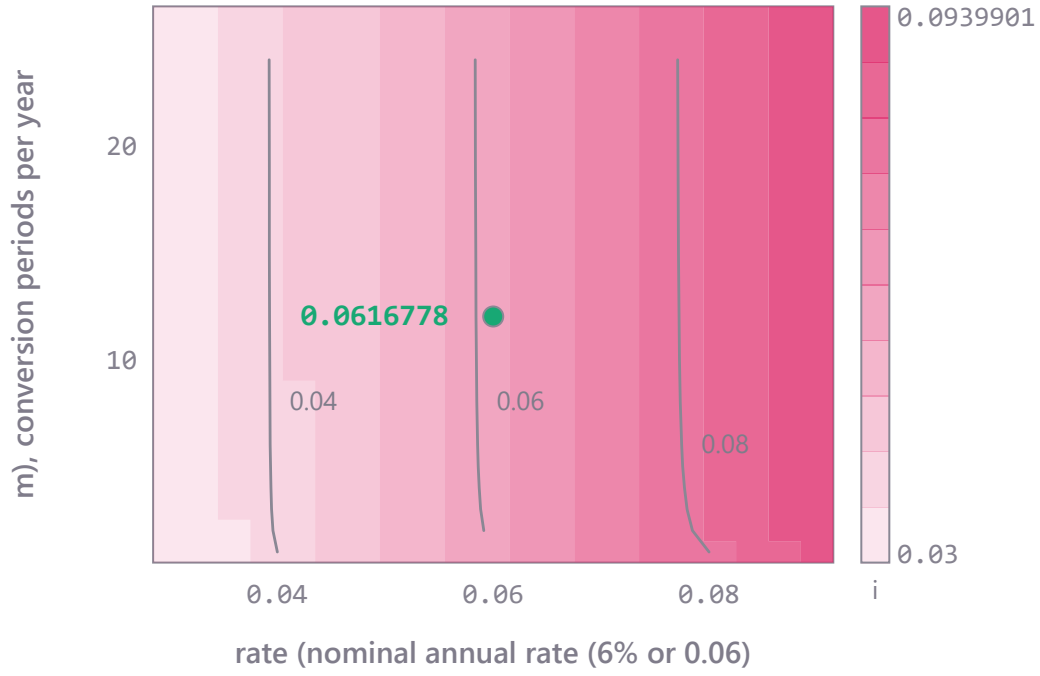
input	meaning	condition
`rate`	nominal annual rate (6% or 0.06)	
`m`	conversion periods per year (12 = monthly)	
`kind`	the nominal rate is an interest rate i(m) or a discount rate d(m)	

output	meaning
`i`	annual effective interest rate
`d`	annual effective discount rate
`delta`	force of interest
`v`	annual discount factor

- **Method:** 1 + i = (1 + i(m)/m)^m or (1 - d(m)/m)^(-m); $d=i/(1+i)$; $\delta=ln$(1+i)
- **Assumes:** 0 < rate < 1; $1\le m\le 365$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



i (annual effective interest rate) against m (conversion periods per year (12 = monthly)) from 1 to 24. One curve for each value of rate (nominal annual rate (6% or 0.06)): 0.06, 0.03, 0.12. Other inputs as in the worked example: kind = interest. 24 points per curve computed by actuarial-science-ai999x; marked: the worked example, m = 12, i = 0.0616778.



i (annual effective interest rate) over rate (nominal annual rate (6% or 0.06)) from 0.03 to 0.09 and m (conversion periods per year (12 = monthly)) from 1 to 24: each of the 21 × 12 cells is one run of actuarial-science-ai999x, shaded by its value (light 0.03 to dark 0.0939901); iso-lines at i = 0.04, 0.06, 0.08. Other inputs as in the worked example: kind = interest. Marked: the worked example, rate = 0.06, m = 12, i = 0.0616778.

```
actuarial_science_ai999x.solve('act_nominal_to_effective_rate', rate=0.06, m=12, kind='interest')
# -> i = 0.0616778, d = 0.0580947, delta = 0.0598505, v = 0.941905   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_nominal_to_effective_rate', rate=0.08, m=4, kind='discount')
# -> i = 0.0841658, d = 0.0776318, delta = 0.0808108, v = 0.922368   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Effective rate from a nominal rate: rate = 0.06, m = 12, kind = interest	SOLVED	i = 0.0616778, d = 0.0580947, delta = 0.0598505, v = 0.941905
Effective rate from a nominal rate: rate = 0.08, m = 4, kind = discount	SOLVED	i = 0.0841658, d = 0.0776318, delta = 0.0808108, v = 0.922368
Effective rate from a nominal rate: m = 12, kind = interest	NEEDS_INPUT	act_nominal_to_effective_rate needs rate (nominal annual rate (6% or 0.06)). Example: solve('act_nominal_to_effective_ra
Effective rate from a nominal rate: rate = 0.06, m = 12, kind = interest, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Nominal to effective rate: rate = 6%, m = 12, kind = interest	SOLVED	i = 0.0616778, d = 0.0580947, delta = 0.0598505, v = 0.941905

Effective rates from a constant force of interest, `act_force_of_interest`

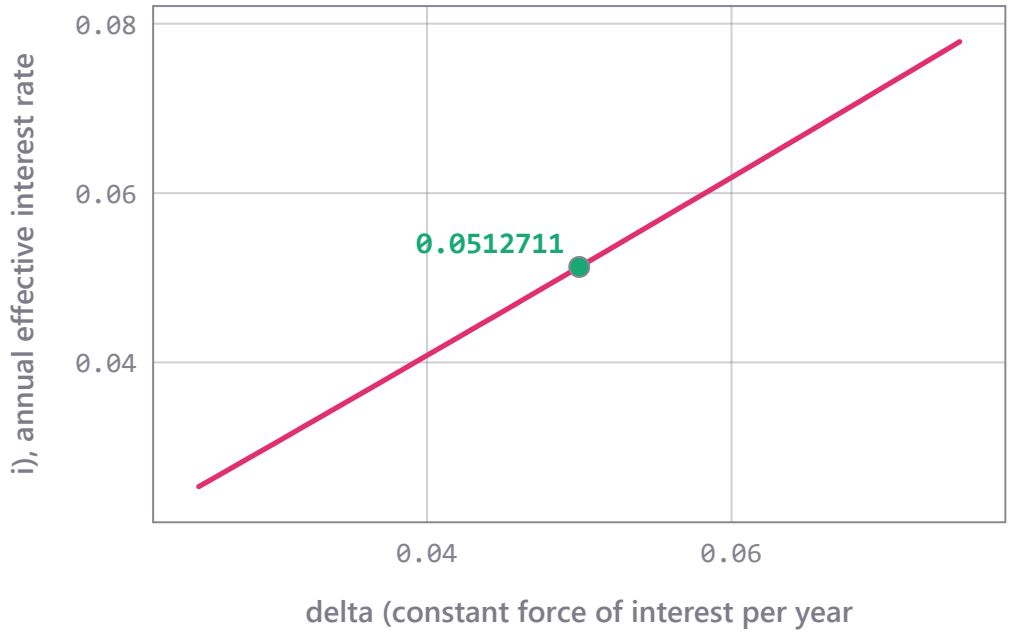
The annual effective interest rate, discount rate and discount factor for a constant force of interest delta.

$$i = e^{\delta} - 1, \quad d = 1 - e^{-\delta}, \quad v = e^{-\delta}$$

input	meaning	condition
`delta`	constant force of interest per year (0.05 or 5%)	

output	meaning
`i`	annual effective interest rate
`d`	annual effective discount rate
`v`	annual discount factor

- **Method:** $i = e^{\delta} - 1$, $d = 1 - e^{-\delta}$, $v = e^{-\delta}$
- **Assumes:** $0 < \delta < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



i (annual effective interest rate) against delta (constant force of interest per year (0.05 or 5%)) from 0.025 to 0.075. 31 points computed by actuarial-science-ai999x; marked: the worked example, delta = 0.05, i = 0.0512711.

```
actuarial_science_ai999x.solve('act_force_of_interest', delta=0.05)
# -> i = 0.0512711, d = 0.0487706, v = 0.951229   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_force_of_interest', delta=0.1)
# -> i = 0.105171, d = 0.0951626, v = 0.904837   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Force of interest: delta = 0.05	SOLVED	i = 0.0512711, d = 0.0487706, v = 0.951229
Force of interest: delta = 0.1	SOLVED	i = 0.105171, d = 0.0951626, v = 0.904837
Force of interest:	NEEDS_INPUT	act_force_of_interest needs delta (constant force of interest per year (0.05 or 5%)). Example: solve('act_force_of_inter
Force of interest: delta = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Accumulated and present value of a single amount, `act_accumulated_value`

The value of an amount accumulated for t years at compound interest, the present value of the same amount due in t years, and the accumulation factor $(1+i)^t$.

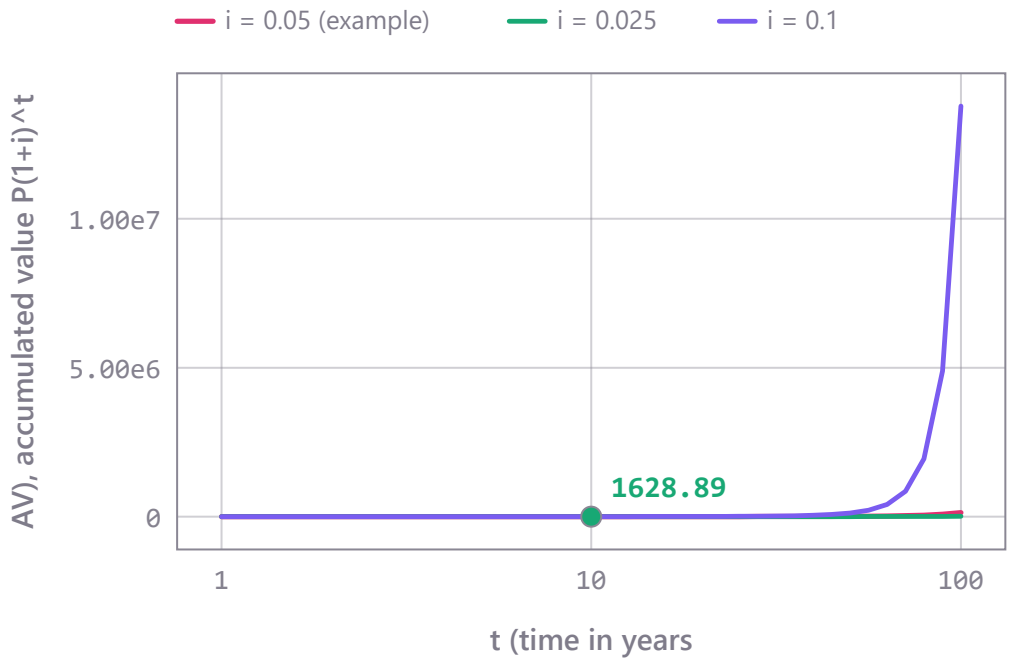
$$AV = P(1+i)^t, \quad PV = P\,v^t$$

input	meaning	condition
`P`	amount	
`i`	annual effective interest rate (5% or 0.05)	

input	meaning	condition
`t`	time in years	

output	meaning
`AV`	accumulated value P(1+i)^t
`PV`	present value P v^t
`factor`	accumulation factor $(1+i)^t$

- **Method:** $AV = P$(1+i)^t; $PV = P$(1+i)^(-t)
- **Assumes:** $0 < i < 1$; $0 \leq t \leq 200$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



AV (accumulated value P(1+i)^t) against t (time in years) from 1 to 100 on a logarithmic axis. One curve for each value of i (annual effective interest rate (5% or 0.05)): 0.05, 0.025, 0.1. Other inputs as in the worked example: P = 1000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, t = 10, AV = 1628.89.

```
actuarial_science_ai999x.solve('act_accumulated_value', P=1000.0, i=0.05, t=10.0)
# -> AV = 1628.89, PV = 613.913, factor = 1.62889   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_accumulated_value', P=2500.0, i=0.07, t=3.5)
# -> AV = 3167.99, PV = 1972.86, factor = 1.26719   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Accumulated value: P = 1000, i = 0.05, t = 10	SOLVED	AV = 1628.89, PV = 613.913, factor = 1.62889
Accumulated value: P = 2500, i = 0.07, t = 3.5	SOLVED	AV = 3167.99, PV = 1972.86, factor = 1.26719
Accumulated value: i = 0.05, t = 10	NEEDS_INPUT	act_accumulated_value needs P (amount). Example: solve('act_accumulated_value', P=1000, i=0.05, t=10).

question	outcome	result
Accumulated value: P = 1000, i = 0.05, t = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Annuity-certain present and accumulated values, `act_annuity_certain`

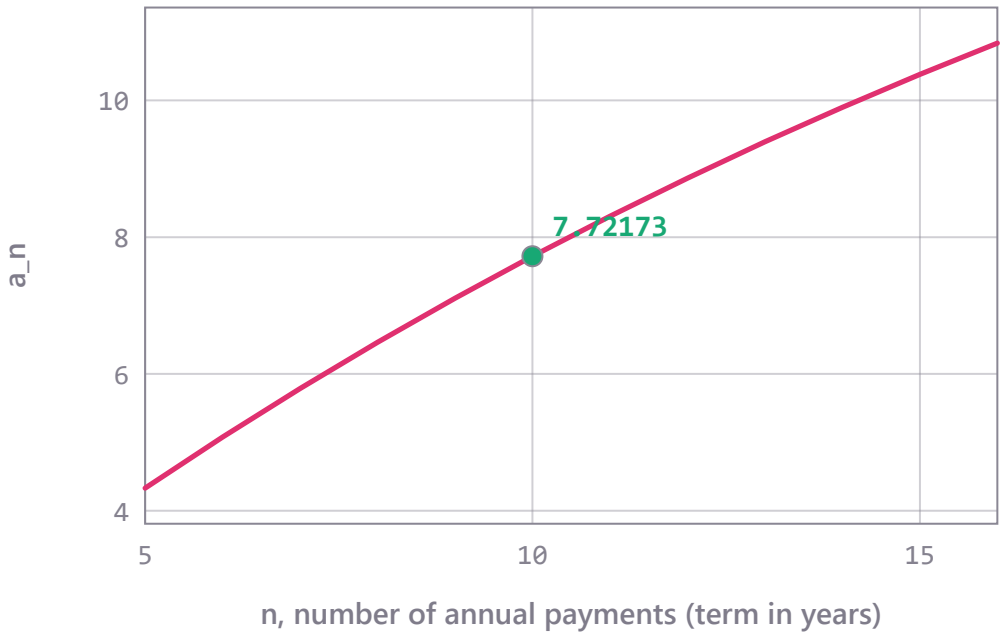
The present value of n level payments of 1 at the end (immediate) or start (due) of each year, their accumulated values at time n, and the continuously payable annuity over n years.

$$a_{\overline{n}|} = \frac{1 - v^n}{i},\ \ddot{a}_{\overline{n}|} = \frac{1 - v^n}{d},\ s_{\overline{n}|} = \frac{(1+i)^n - 1}{i},\ \bar{a}_{\overline{n}|} = \frac{1 - v^n}{\delta}$$

input	meaning	condition
`n`	number of annual payments (term in years)	
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`a_n`	present value of an annuity-immediate a_n
`a_due`	present value of an annuity-due
`s_n`	accumulated value of an annuity-immediate s_n
`s_due`	accumulated value of an annuity-due
`a_cont`	present value of a continuous annuity over n years

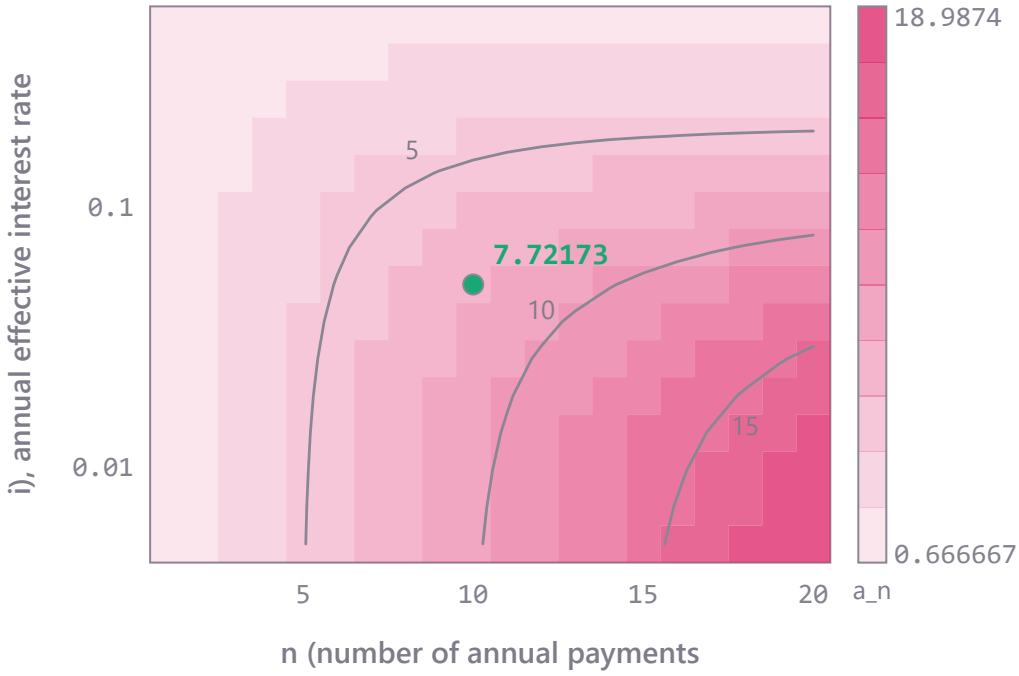
- **Method:** $a_n = (1 - v^n)/i$, a-due = (1 - v^n)/d, s_n = ((1+i)^n - 1)/i, s-due = ((1+i)^n - 1)/d, continuous = (1 - v^n)/delta
- **Assumes:** $n \geq 1; 0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a_n (present value of an annuity-immediate a_n) against n (number of annual payments (term in years)), with i = 0.05. 12 points computed by actuarial-science-ai999x; marked: n = 10, a_n = 7.72173.



a_n (present value of an annuity-immediate a_n) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of n (number of annual payments (term in years)): 10, 5, 20. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, a_n = 7.72173.



a_n (present value of an annuity-immediate a_n) over n (number of annual payments (term in years)) from 1 to 20 and i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 (logarithmic vertical axis): each of the 20 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 0.666667 to dark 18.9874); iso-lines at a_n = 5, 10, 15. Marked: the worked example, n = 10, i = 0.05, a_n = 7.72173.

```
actuarial_science_ai999x.solve('act_annuity_certain', n=10, i=0.05)
# -> a_n = 7.72173, a_due = 8.10782, s_n = 12.5779, s_due = 13.2068, a_cont = 7.91321   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_annuity_certain', n=25, i=0.08)
# -> a_n = 10.6748, a_due = 11.5288, s_n = 73.1059, s_due = 78.9544, a_cont = 11.0963   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Annuity-certain: n = 10, i = 0.05	SOLVED	a_n = 7.72173, a_due = 8.10782, s_n = 12.5779, s_due = 13.2068, a_cont = 7.91321
Annuity-certain: n = 25, i = 0.08	SOLVED	a_n = 10.6748, a_due = 11.5288, s_n = 73.1059, s_due = 78.9544, a_cont = 11.0963
Annuity-certain: i = 0.05	NEEDS_INPUT	act_annuity_certain needs n (number of annual payments (term in years)). Example: solve('act_annuity_certain', n=10, i=0
Annuity-certain: n = 10, i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Annuity certain: n = 10, i = 5%	SOLVED	a_n = 7.72173, a_due = 8.10782, s_n = 12.5779, s_due = 13.2068, a_cont = 7.91321
Annuity-certain: n = 0, i = 5%	INVALID_INPUT	n: must be at least 1, got 0.

Present value of a perpetuity, `act_perpetuity`

The present value of payments of 1 a year forever, at the end (immediate) or start (due) of each year, and paid continuously.

$$a_{\overline{\infty}|} = \frac{1}{i}, \quad \ddot{a}_{\overline{\infty}|} = \frac{1}{d}, \quad \bar{a}_{\overline{\infty}|} = \frac{1}{\delta}$$

input	meaning	condition
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`a_inf`	perpetuity-immediate $1/i$
`a_due_inf`	perpetuity-due $1/d$
`a_cont_inf`	continuous perpetuity $1/\delta$

- **Method:** a_inf = 1/i, a-due_inf = 1/d = (1+i)/i, continuous $1/\delta$
- **Assumes:** $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a_inf (perpetuity-immediate 1/i) against i (annual effective interest rate (5% or 0.05)) from 0.025 to 0.075. 31 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, a_inf = 20.

```
actuarial_science_ai999x.solve('act_perpetuity', i=0.05)
# -> a_inf = 20, a_due_inf = 21, a_cont_inf = 20.4959   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_perpetuity', i=0.04)
# -> a_inf = 25, a_due_inf = 26, a_cont_inf = 25.4967   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Perpetuity: i = 0.05	SOLVED	a_inf = 20, a_due_inf = 21, a_cont_inf = 20.4959
Perpetuity: i = 0.04	SOLVED	a_inf = 25, a_due_inf = 26, a_cont_inf = 25.4967
Perpetuity:	NEEDS_INPUT	act_perpetuity needs i (annual effective interest rate (5% or 0.05)). Example: solve('act_perpetuity', i=0.05).
Perpetuity: i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Increasing and decreasing annuities-certain, `act_increasing_decreasing_annuity`

The present values of an n-year annuity paying 1, 2, ..., n (increasing) and n, $n-1$, ..., 1 (decreasing), immediate and due.

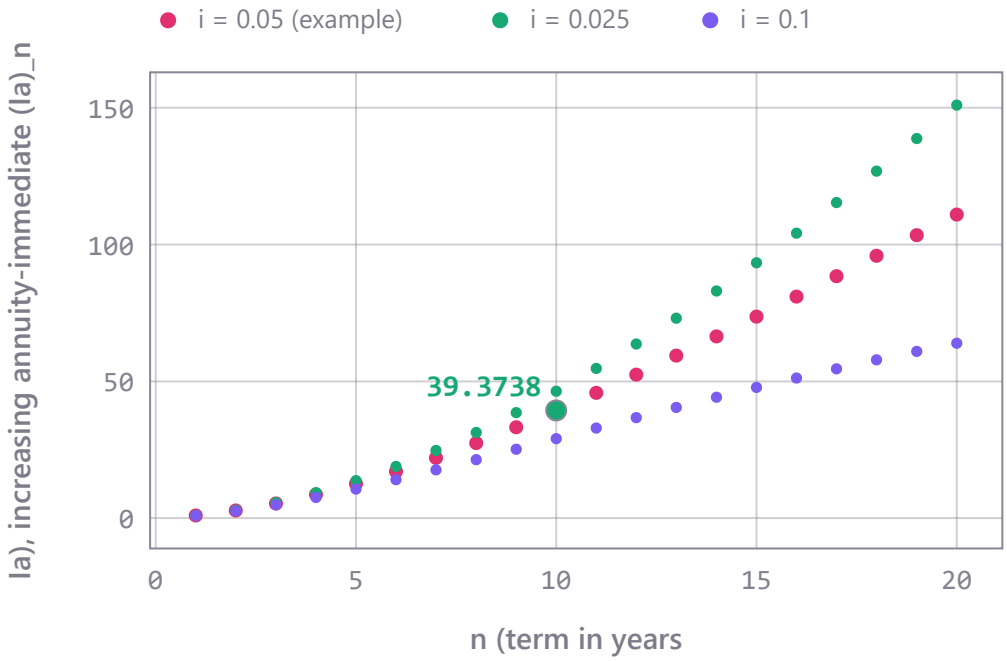
$$(Ia)_{\overline{n}|} = \frac{\ddot{a}_{\overline{n}|} - nv^n}{i}, \quad (Da)_{\overline{n}|} = \frac{n - a_{\overline{n}|}}{i}$$

input	meaning	condition
n	term in years	

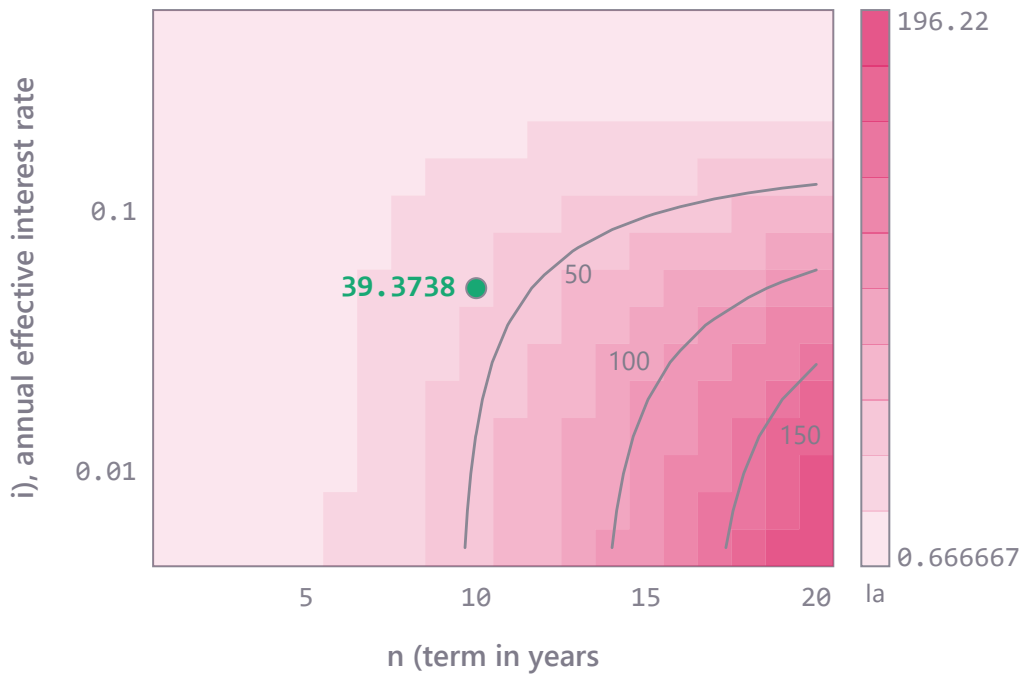
input	meaning	condition
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`Ia`	increasing annuity-immediate (Ia)_n
`Ia_due`	increasing annuity-due
`Da`	decreasing annuity-immediate (Da)_n
`Da_due`	decreasing annuity-due

- **Method:** (Ia)_n = (a-due_n - n v^n)/i; $(Da)_{\boldsymbol{n}} = (\boldsymbol{n} - \boldsymbol{a_n})/\boldsymbol{i}$; due values times $(\boldsymbol{1} + \boldsymbol{i})$
- **Assumes:** $\boldsymbol{n} \geq \boldsymbol{1}$; $\boldsymbol{0} < \boldsymbol{i} < \boldsymbol{1}$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ia (increasing annuity-immediate (Ia)_n) against n (term in years) from 1 to 20. One curve for each value of i (annual effective interest rate (5% or 0.05)): 0.05, 0.025, 0.1. 20 points per curve computed by actuarial-science-ai999x; marked: the worked example, n = 10, Ia = 39.3738.



Ia (increasing annuity-immediate (Ia)_n) over n (term in years) from 1 to 20 and i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 (logarithmic vertical axis): each of the 20 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 0.666667 to dark 196.22); iso-lines at Ia = 50, 100, 150. Marked: the worked example, n = 10, i = 0.05, Ia = 39.3738.

```
actuarial_science_ai999x.solve('act_increasing_decreasing_annuity', n=10, i=0.05)
# -> Ia = 39.3738, Ia_due = 41.3425, Da = 45.5653, Da_due = 47.8436   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_increasing_decreasing_annuity', n=5, i=0.1)
# -> Ia = 10.6526, Ia_due = 11.7178, Da = 12.0921, Da_due = 13.3013   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Increasing and decreasing annuity: n = 10, i = 0.05	SOLVED	Ia = 39.3738, Ia_due = 41.3425, Da = 45.5653, Da_due = 47.8436
Increasing and decreasing annuity: n = 5, i = 0.1	SOLVED	Ia = 10.6526, Ia_due = 11.7178, Da = 12.0921, Da_due = 13.3013
Increasing and decreasing annuity: i = 0.05	NEEDS_INPUT	act_increasing_decreasing_annuity needs n (term in years). Example: solve('act_increasing_decreasing_annuity', n=10, i=0
Increasing and decreasing annuity: n = 10, i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Loan amortization: level payment, interest and principal, outstanding balance, `act_loan_amortization`

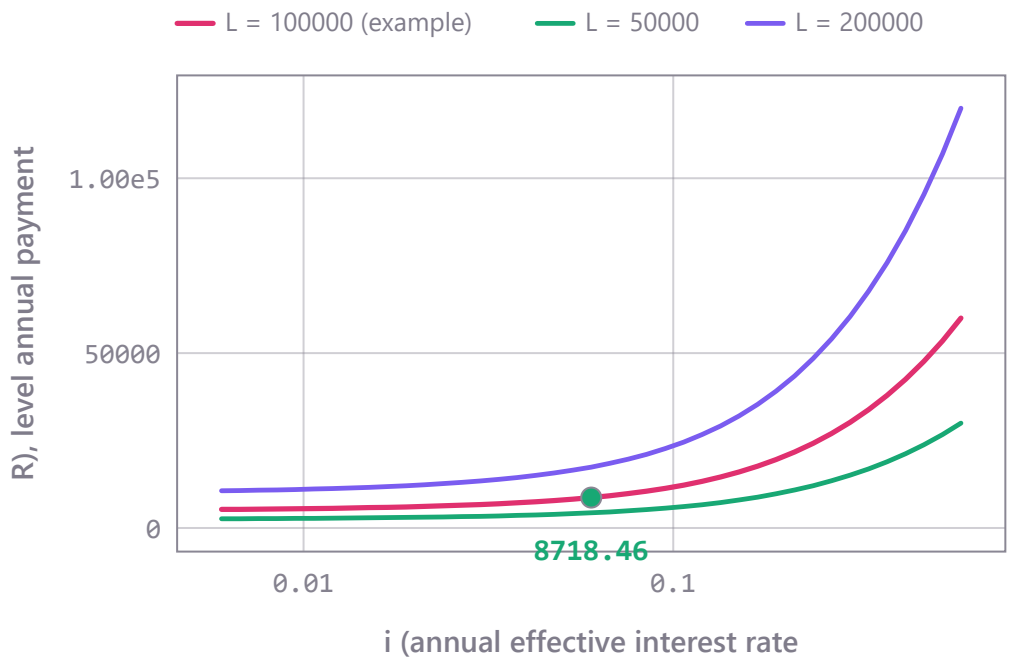
The level annual payment that repays a loan over n years, the interest and principal parts of payment t and the outstanding balance just after it.

$$R = \frac{L}{a_{\overline{n}|}}, \quad I_t = R\left(1 - v^{n-t+1}\right), \quad P_t = R\,v^{n-t+1}, \quad B_t = R\,a_{\overline{n-t}|}$$

input	meaning	condition
`L`	loan amount	
`i`	annual effective interest rate (5% or 0.05)	
`n`	number of annual payments	
`t`	payment number of interest (1 to n)	

output	meaning
`R`	level annual payment
`interest_t`	interest part of payment t
`principal_t`	principal repaid in payment t
`balance_t`	outstanding balance just after payment t
`total_interest`	total interest paid over the loan

- **Method:** $R = L/a_{\overline{n}|}$; balance before payment $t = R$ a_(n-t+1); interest = i x balance; principal = R v^{n-t+1}; balance after = R a_(n-t)
- **Assumes:** $L > 0$; $0 < i < 1$; $1 \le t \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



R (level annual payment) against i (annual effective interest rate (5% or 0.05)) from 0.006 to 0.6 on a logarithmic axis. One curve for each value of L (loan amount): 100000, 50000, 200000. Other inputs as in the worked example: n = 20, t = 5. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.06, R = 8718.46.

```
actuarial_science_ai999x.solve('act_loan_amortization', L=100000.0, i=0.06, n=20, t=5)
# -> R = 8718.46, interest_t = 5286.47, principal_t = 3431.99, balance_t = 84675.8, total_interest
= 74369.1   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_loan_amortization', L=5000.0, i=0.1, n=4, t=2)
# -> R = 1577.35, interest_t = 392.265, principal_t = 1185.09, balance_t = 2737.56, total_interest
= 1309.42   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Loan amortization: L = 100000, i = 0.06, n = 20, t = 5	SOLVED	R = 8718.46, interest_t = 5286.47, principal_t = 3431.99, balance_t = 84675.8, total_interest = 74369.1
Loan amortization: L = 5000, i = 0.1, n = 4, t = 2	SOLVED	R = 1577.35, interest_t = 392.265, principal_t = 1185.09, balance_t = 2737.56, total_interest = 1309.42
Loan amortization: i = 0.06, n = 20, t = 5	NEEDS_INPUT	act_loan_amortization needs L (loan amount). Example: solve('act_loan_amortization', L=100000, i=0.06, n=20, t=5).
Loan amortization: L = 100000, i = 0.06, n = 20, t = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Loan amortization: L = 5000, i = 10%, n = 4, t = 5	INVALID_INPUT	Loan amortization: level payment, interest and principal, outstanding balance: the payment number t must lie between 1 a

Yield rate of a level annuity, `act_annuity_yield`

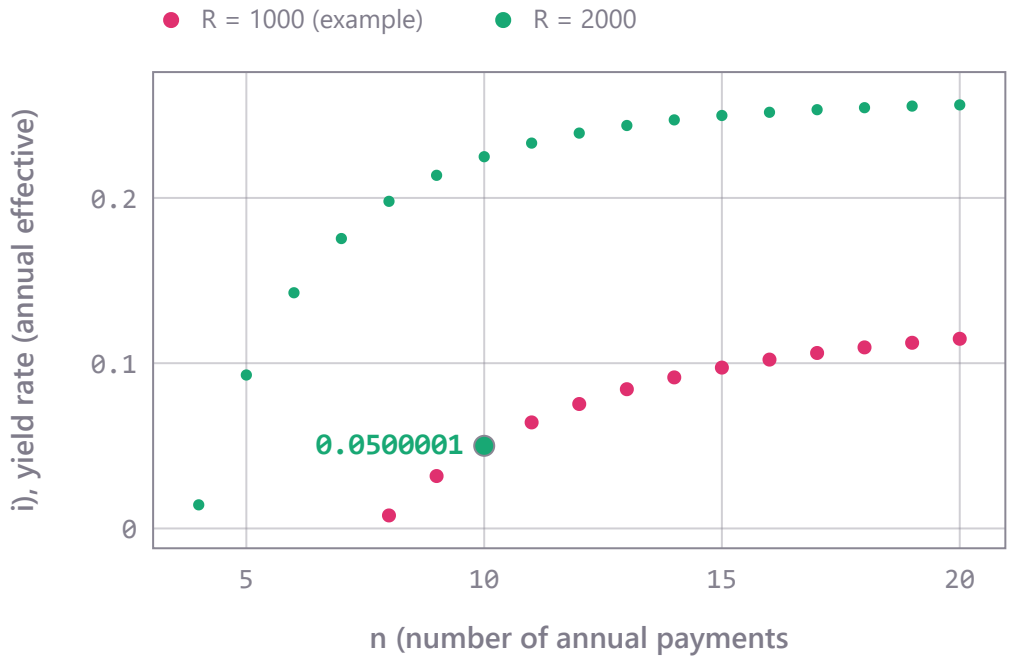
The annual effective interest rate at which n level payments R at the ends of the years are worth a given present value.

$$R\,\frac{1-(1+i)^{-n}}{i} = PV$$

input	meaning	condition
`PV`	present value (price) of the annuity	
`R`	level annual payment	
`n`	number of annual payments	

output	meaning
`i`	yield rate (annual effective)
`a_n`	annuity factor PV/R

- **Method:** solve R a_n(i) = PV for i by bisection (a_n decreases in i, so the root is unique)
- **Assumes:** $0 < PV < n$ R; yield below 99%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



i (yield rate (annual effective)) against n (number of annual payments) from 1 to 20. One curve for each value of R (level annual payment): 1000, 2000. Other inputs as in the worked example: PV = 7721.73. 13 points per curve computed by actuarial-science-ai999x; marked: the worked example, n = 10, i = 0.0500001.

```
actuarial_science_ai999x.solve('act_annuity_yield', PV=7721.73, R=1000.0, n=10)
# -> i = 0.0500001, a_n = 7.72173   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_annuity_yield', PV=4000.0, R=1000.0, n=5)
# -> i = 0.0793083, a_n = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Yield of an annuity: PV = 7721.73, R = 1000, n = 10	SOLVED	i = 0.0500001, a_n = 7.72173
Yield of an annuity: PV = 4000, R = 1000, n = 5	SOLVED	i = 0.0793083, a_n = 4
Yield of an annuity: R = 1000, n = 10	NEEDS_INPUT	act_annuity_yield needs PV (present value (price) of the annuity). Example: solve('act_annuity_yield', PV=7721.73, R=100
Yield of an annuity: PV = 7721.73, R = 1000, n = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Yield of an annuity: PV = 12000, R = 1000, n = 10	INVALID_INPUT	Yield rate of a level annuity: the present value must be below the total of the payments (a positive yield).

Present value of a series of cash flows, `act_present_value_cash_flows`

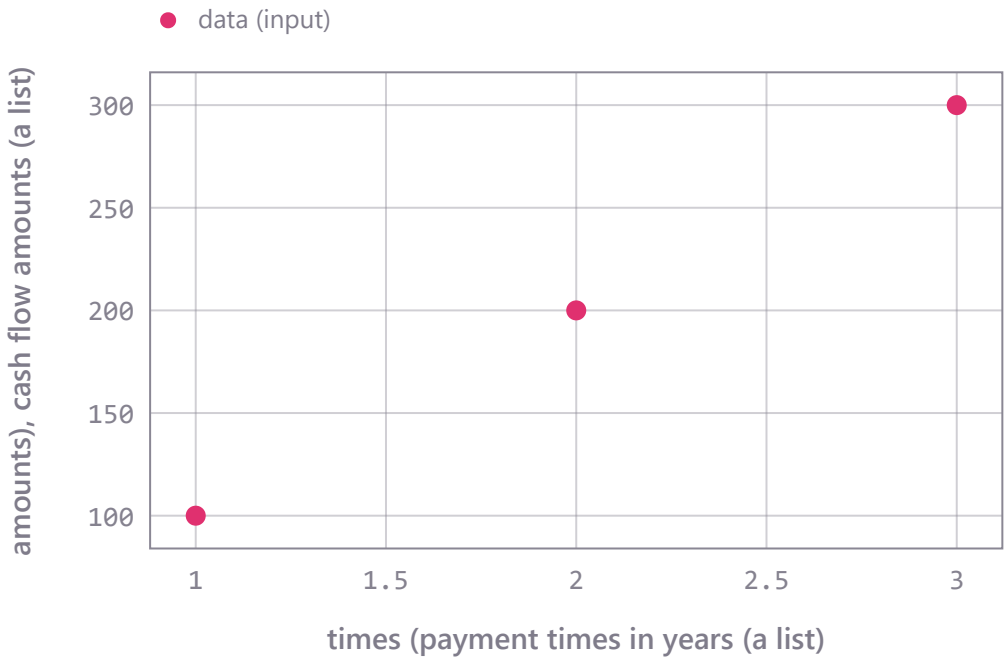
The present value at time 0 of amounts paid at stated times (years), and their accumulated value at the last payment time.

$$PV = \sum_k C_k \, v^{t_k}$$

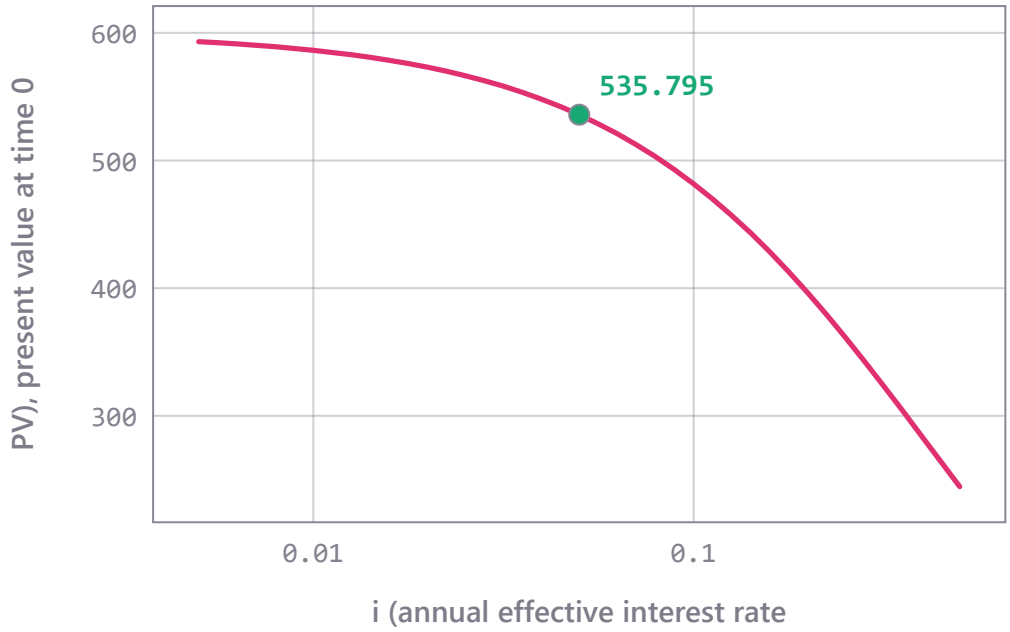
input	meaning	condition
`amounts`	cash flow amounts (a list)	
`times`	payment times in years (a list)	
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`PV`	present value at time 0
`AV_at_last`	accumulated value at the last payment time

- **Method:** PV = sum C_k v^{t_k}; $AV = PV$ (1+i)^(t_max)
- **Assumes:** one time per amount; $0 \leq t \leq 200$; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data amounts (cash flow amounts (a list)) against times (payment times in years (a list)) as given in the worked example (3 points). Also returned: PV = 535.795, AV_at_last = 620.25. Other inputs: i = 0.05.



PV (present value at time 0) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: amounts = [100.0, 200.0, 300.0], times = [1.0, 2.0, 3.0]. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, PV = 535.795.

```
actuarial_science_ai999x.solve('act_present_value_cash_flows', amounts=[100.0, 200.0, 300.0],
times=[1.0, 2.0, 3.0], i=0.05)
# -> PV = 535.795, AV_at_last = 620.25   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_present_value_cash_flows', amounts=[-1000.0, 400.0, 400.0,
400.0], times=[0.0, 1.0, 2.0, 3.0], i=0.08)
# -> PV = 30.8388, AV_at_last = 38.848   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Present value of cash flows: amounts = [100, 200, 300], times = [1, 2, 3], i = 0.05	SOLVED	PV = 535.795, AV_at_last = 620.25
Present value of cash flows: amounts = [-1000, 400, 400, 400], times = [0, 1, 2, 3], i = 0.08	SOLVED	PV = 30.8388, AV_at_last = 38.848
Present value of cash flows: times = [1, 2, 3], i = 0.05	NEEDS_INPUT	act_present_value_cash_flows needs amounts (cash flow amounts (a list)). Example: solve('act_present_value_cash_flows',
Present value of cash flows: amounts = [100, 200, 300], times = [1, 2, 3], i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Macaulay and modified duration and convexity of cash flows, `act_duration_convexity`

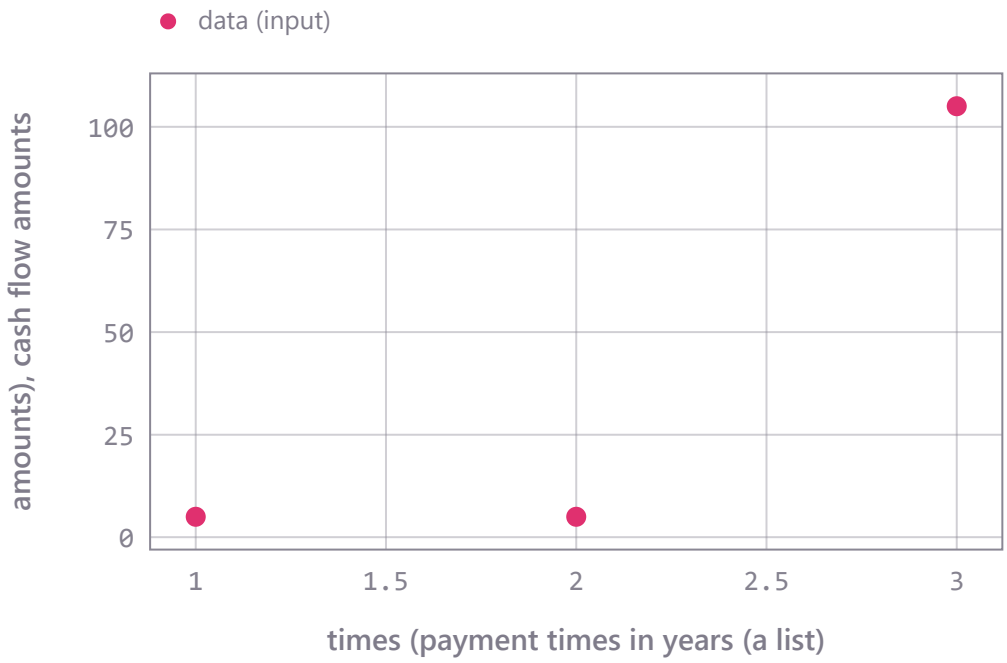
The present value, Macaulay duration, modified duration and convexity (second derivative of price with respect to i over price) of a set of non-negative cash flows.

$$D_{mac} = \frac{\sum t\, C_t v^t}{P}, \quad D_{mod} = \frac{D_{mac}}{1+i}, \quad C = \frac{\sum t(t+1) C_t v^{t+2}}{P}$$

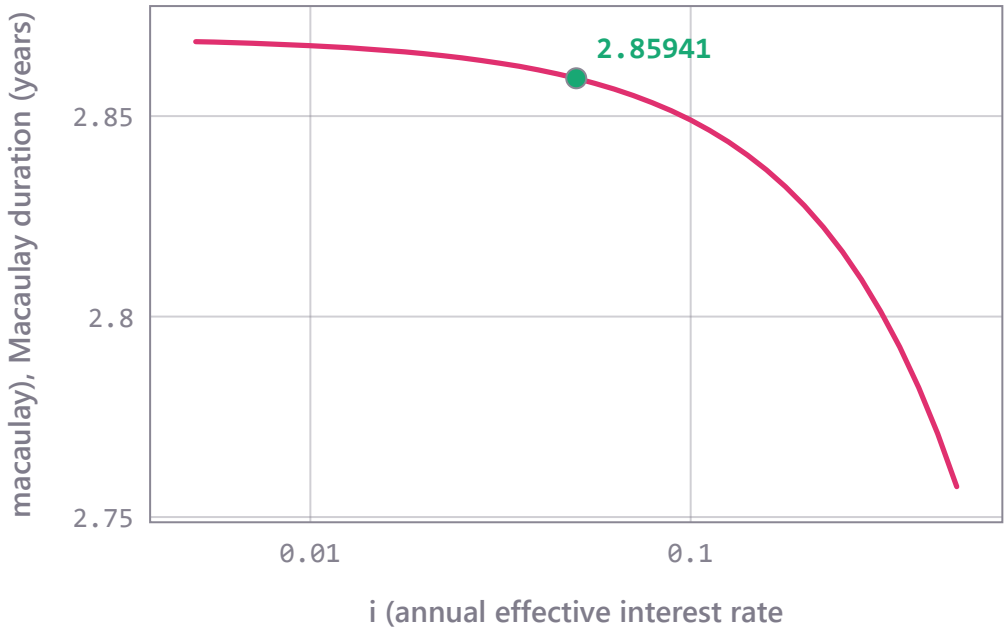
input	meaning	condition
`amounts`	cash flow amounts (a list, non-negative)	
`times`	payment times in years (a list)	
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`PV`	present value
`macaulay`	Macaulay duration (years)
`modified`	modified duration -P'(i)/P
`convexity`	convexity P''(i)/P

- **Method:** D_mac = sum t C v^t/P; D_mod = D_mac/(1+i); convexity = sum t(t+1) C v^{t+2}/P
- **Assumes:** amounts >= 0 with a positive total; one time per amount; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data amounts (cash flow amounts (a list, non-negative)) against times (payment times in years (a list)) as given in the worked example (3 points). Also returned: PV = 100, macaulay = 2.85941, modified = 2.72325, convexity = 10.2056. Other inputs: i = 0.05.



macaulay (Macaulay duration (years)) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: amounts = [5.0, 5.0, 105.0], times = [1.0, 2.0, 3.0]. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, macaulay = 2.85941.

```
actuarial_science_ai999x.solve('act_duration_convexity', amounts=[5.0, 5.0, 105.0], times=[1.0, 2.0, 3.0], i=0.05)
# -> PV = 100, macaulay = 2.85941, modified = 2.72325, convexity = 10.2056   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_duration_convexity', amounts=[1000.0, 2000.0], times=[2.0, 10.0], i=0.04)
# -> PV = 2275.68, macaulay = 6.74979, modified = 6.49018, convexity = 62.6362   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Duration and convexity: amounts = [5, 5, 105], times = [1, 2, 3], i = 0.05	SOLVED	PV = 100, macaulay = 2.85941, modified = 2.72325, convexity = 10.2056
Duration and convexity: amounts = [1000, 2000], times = [2, 10], i = 0.04	SOLVED	PV = 2275.68, macaulay = 6.74979, modified = 6.49018, convexity = 62.6362
Duration and convexity: times = [1, 2, 3], i = 0.05	NEEDS_INPUT	act_duration_convexity needs amounts (cash flow amounts (a list, non-negative)). Example: solve('act_duration_convexity'
Duration and convexity: amounts = [5, 5, 105], times = [1, 2, 3], i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Life table functions dx, qx and px from survivor numbers, `act_life_table_functions`

The numbers dying, the one-year death probabilities and the one-year survival probabilities at each age of a life table given by its survivor numbers lx.

$$d_x = l_x - l_{x+1}, \quad q_x = \frac{d_x}{l_x}, \quad p_x = \frac{l_{x+1}}{l_x}$$

input	meaning	condition
`lx`	survivors at consecutive integer ages (a list)	

output	meaning
`dx`	deaths between ages x and $x+1$ (a list)
`qx`	one-year death probabilities (a list)
`px`	one-year survival probabilities (a list)

- **Method:** Missing superscript or subscript argument(x+1), $q_x = d_x/l_x$, Missing superscript or subscript argument(x+1)/l_x
- **Assumes:** lx non-increasing; only the last lx may be 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



one-year death probabilities (a list) (qx) returned by actuarial-science-ai999x for the worked example, one value for each position in the list (3 values, first 0.05, last 0.113636, largest 0.113636); with dx (deaths between ages x and x+1 (a list)), px (one-year survival probabilities (a list)). Inputs: lx = [1000.0, 950.0, 880.0, 780.0].

```
actuarial_science_ai999x.solve('act_life_table_functions', lx=[1000.0, 950.0, 880.0, 780.0])
# -> dx = [50.0, 70.0, 100.0], qx = [0.05, 0.0736842105263, 0.113636363636], px = [0.95,
0.926315789474, 0.886363636364]   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_life_table_functions', lx=[100000.0, 99500.0, 98900.0, 98200.0,
97300.0])
# -> dx = [500.0, 600.0, 700.0, 900.0], qx = [0.005, 0.00603015075377, 0.00707785642063,
0.0091649694501], px = [0.995, 0.993969849246, 0.992922143579, 0.99083503055]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Life table functions: lx = [1000, 950, 880, 780]	SOLVED	dx = [50.0, 70.0, 100.0], qx = [0.05, 0.0736842105263, 0.113636363636], px = [0.95, 0.926315789474, 0.886363636364]
Life table functions: lx = [100000, 99500, 98900, 98200, 97300]	SOLVED	dx = [500.0, 600.0, 700.0, 900.0], qx = [0.005, 0.00603015075377, 0.00707785642063, 0.0091649694501], px = [0.995, 0.993969849246, 0.992922143579, 0.99083503055]
Life table functions:	NEEDS_INPUT	act_life_table_functions needs lx (survivors at consecutive integer ages (a list)). Example: solve('act_life_table_funct
Life table functions: lx = [1000, 950, 880, 780], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Life table functions: lx = [1000, 1050, 900]	INVALID_INPUT	Life table functions dx, qx and px from survivor numbers: survivor numbers lx must not increase with age.

Probability of surviving t years from a life table, `act_survival_probability`

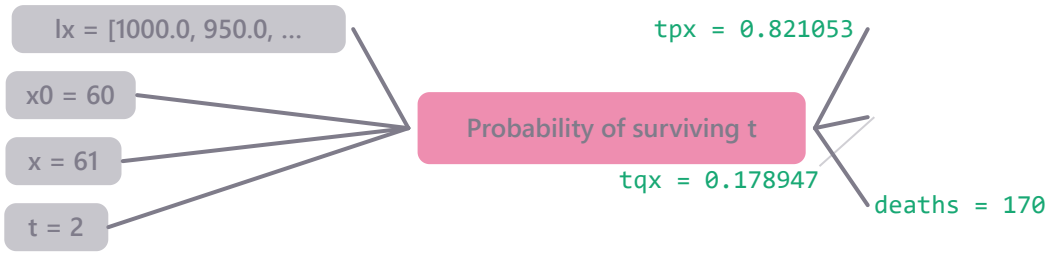
The probability that a life aged x survives t more years, the probability of dying within t years, and the number of deaths between ages x and x+t, from survivor numbers lx starting at age x0.

$$ {}_tp_x = \frac{l_{x+t}}{l_x}, \quad {}_tq_x = \frac{l_x - l_{x+t}}{l_x} $$

input	meaning	condition
`lx`	survivors at consecutive integer ages from x0 (a list)	
`x0`	age of the first entry of lx	
`x`	age of the life	
`t`	number of years	

output	meaning
`tpx`	probability of surviving t years
`tqx`	probability of dying within t years
`deaths`	number dying between ages x and x+t

- **Method:** Double subscripts: use braces to clarify(x+t)/l_x; Double subscripts: use braces to clarify
- **Assumes:** $x_0 \le x$ and x + t inside the table; $l_x > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Probability of surviving t years from a life table as inputs and results: stated lx = [1000.0, 950.0, 880.0, 780.0], x0 = 60, x = 61, t = 2; returned by actuarial-science-ai999x: tpx = 0.821053, tqx = 0.178947, deaths = 170.

```
actuarial_science_ai999x.solve('act_survival_probability', lx=[1000.0, 950.0, 880.0, 780.0], x0=60, x=61, t=2)
# -> tpx = 0.821053, tqx = 0.178947, deaths = 170   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_survival_probability', lx=[100000.0, 99500.0, 98900.0, 98200.0, 97300.0], x0=40, x=40, t=4)
# -> tpx = 0.973, tqx = 0.027, deaths = 2700   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Survival probability from a life table: lx = [1000, 950, 880, 780], x0 = 60, x = 61, t = 2	SOLVED	tpx = 0.821053, tqx = 0.178947, deaths = 170
Survival probability from a life table: lx = [100000, 99500, 98900, 98200, 97300], x0 = 40, x = 40, t = 4	SOLVED	tpx = 0.973, tqx = 0.027, deaths = 2700
Survival probability from a life table: x0 = 60, x = 61, t = 2	NEEDS_INPUT	act_survival_probability needs lx (survivors at consecutive integer ages from x0 (a list)). Example: solve('act_survival
Survival probability from a life table: lx = [1000, 950, 880, 780], x0 = 60, x = 61, t = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Probability of surviving: lx = [1000, 950, 880, 780], x0 = 60, x = 61, t = 3	INVALID_INPUT	Probability of surviving t years from a life table: the ages asked about run past the end of the table.

Deferred death probability u|t q x, `act_deferred_death_probability`

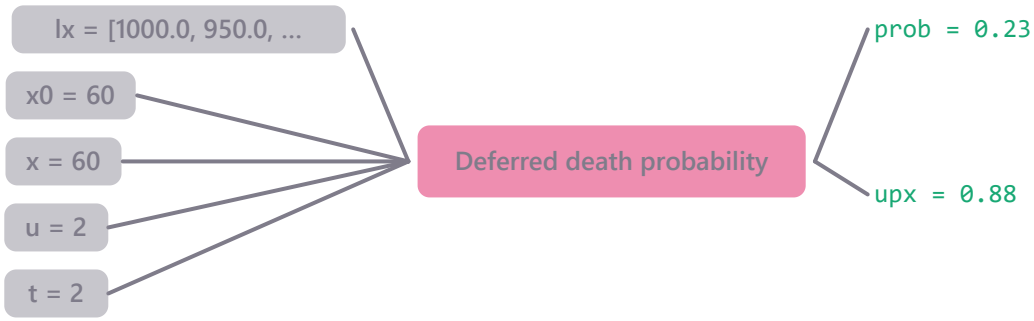
The probability that a life aged x survives u years and then dies within the following t years, from survivor numbers lx starting at age x0.

$$ {}_{u|t}q_x = \frac{l_{x+u} - l_{x+u+t}}{l_x} = {}_{u}p_x \; {}_{t}q_{x+u} $$

input	meaning	condition
`lx`	survivors at consecutive integer ages from x0 (a list)	
`x0`	age of the first entry of lx	
`x`	age of the life	
`u`	years of deferment	
`t`	length of the death interval in years	

output	meaning
`prob`	deferred death probability u|t q_x
`upx`	probability of surviving the deferment

- **Method:** u|t q_x = (l_(x+u) - l_(x+u+t))/Double subscripts: use braces to clarify(x+u)
- **Assumes:** x + u + t inside the table; $t \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Deferred death probability u|t q x as inputs and results: stated lx = [1000.0, 950.0, 880.0, 780.0, 650.0], x0 = 60, x = 60, u = 2, t = 2; returned by actuarial-science-ai999x: prob = 0.23, upx = 0.88.

```
actuarial_science_ai999x.solve('act_deferred_death_probability', lx=[1000.0, 950.0, 880.0, 780.0,
650.0], x0=60, x=60, u=2, t=2)
# -> prob = 0.23, upx = 0.88   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_deferred_death_probability', lx=[100000.0, 99500.0, 98900.0,
98200.0, 97300.0], x0=40, x=41, u=1, t=1)
# -> prob = 0.00703518, upx = 0.99397   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Deferred death probability: lx = [1000, 950, 880, 780, 650], x0 = 60, x = 60, u = 2, t = 2	SOLVED	prob = 0.23, upx = 0.88

question	outcome	result
Deferred death probability: lx = [100000, 99500, 98900, 98200, 97300], x0 = 40, x = 41, u = 1, t = 1	SOLVED	prob = 0.00703518, upx = 0.99397
Deferred death probability: x0 = 60, x = 60, u = 2, t = 2	NEEDS_INPUT	act_deferred_death_probability needs lx (survivors at consecutive integer ages from x0 (a list)). Example: solve('act_de
Deferred death probability: lx = [1000, 950, 880, 780, 650], x0 = 60, x = 60, u = 2, t = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Curtate and complete expectation of life, `act_life_expectancy`

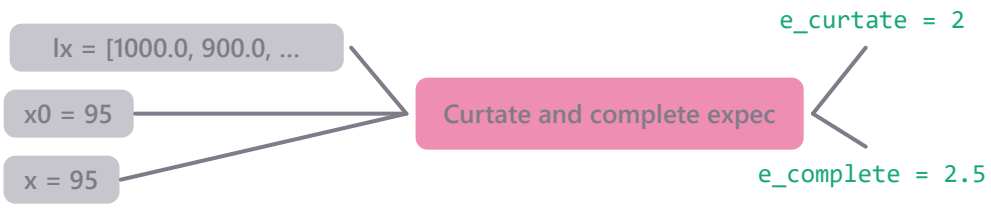
The curtate expectation of life e_x (whole years lived) and the complete expectation e°_x under a uniform distribution of deaths within each year, for a life aged x in a complete life table.

$$e_x = \sum_{k\geq 1} {}_kp_x, \quad \mathring{e}_x = e_x + \tfrac{1}{2}$$

input	meaning	condition
`lx`	survivors at consecutive integer ages from x0, ending with 0 (a list)	
`x0`	age of the first entry of lx	
`x`	age of the life	

output	meaning
`e_curtate`	curtate expectation of life e_x (years)
`e_complete`	complete expectation of life under UDD (years)

- **Method:** e_x = sum_(k>=1) l_(x+k)/l_x; $e°_x = e_x + 1/2$ under UDD
- **Assumes:** the table closes (last lx = 0); x inside the table
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Curtate and complete expectation of life as inputs and results: stated lx = [1000.0, 900.0, 700.0, 400.0, 0.0], x0 = 95, x = 95; returned by actuarial-science-ai999x: e_curtate = 2, e_complete = 2.5.

```
actuarial_science_ai999x.solve('act_life_expectancy', lx=[1000.0, 900.0, 700.0, 400.0, 0.0], x0=95, x=95)
# -> e_curtate = 2, e_complete = 2.5   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_life_expectancy', lx=[1000.0, 900.0, 700.0, 400.0, 0.0], x0=95, x=97)
# -> e_curtate = 0.571429, e_complete = 1.07143   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Expectation of life: lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95	SOLVED	e_curtate = 2, e_complete = 2.5
Expectation of life: lx = [1000, 900, 700, 400, 0], x0 = 95, x = 97	SOLVED	e_curtate = 0.571429, e_complete = 1.07143
Expectation of life: x0 = 95, x = 95	NEEDS_INPUT	act_life_expectancy needs lx (survivors at consecutive integer ages from x0, ending with 0 (a list)). Example: solve('ac
Expectation of life: lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Life expectancy: lx = [1000, 900, 700], x0 = 95, x = 95	INVALID_INPUT	Curtate and complete expectation of life: this needs a complete table: the last survivor number must be 0.

Temporary (n-year) expectation of life, `act_temporary_expectation`

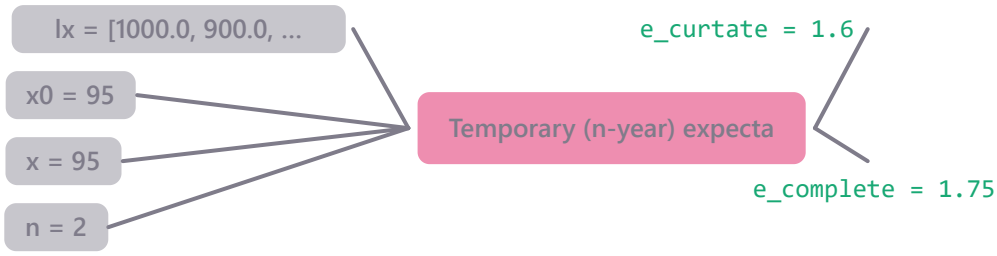
The expected number of whole years (curtate) and of years (complete, UDD) lived within the next n years by a life aged x.

$$e_{x:\overline{n}|} = \sum_{k=1}^{n} {}_kp_x, \quad \mathring{e}_{x:\overline{n}|} = \sum_{k=0}^{n-1} \frac{{}_kp_x + {}_{k+1}p_x}{2}$$

input	meaning	condition
`lx`	survivors at consecutive integer ages from x0 (a list)	
`x0`	age of the first entry of lx	
`x`	age of the life	
`n`	term in years	

output	meaning
`e_curtate`	curtate n-year temporary expectation e_(x:n)
`e_complete`	complete n-year temporary expectation under UDD

- **Method:** e_(x:n) = sum_(k=1..n) k_p_x; e°_(x:n) = sum_(k=0..n-1) (k_p_x + (k+1)_p_x)/2
- **Assumes:** x + n inside the table; $n \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Temporary (n-year) expectation of life as inputs and results: stated lx = [1000.0, 900.0, 700.0, 400.0, 0.0], x0 = 95, x = 95, n = 2; returned by actuarial-science-ai999x: e_curtate = 1.6, e_complete = 1.75.

```
actuarial_science_ai999x.solve('act_temporary_expectation', lx=[1000.0, 900.0, 700.0, 400.0, 0.0],
x0=95, x=95, n=2)
# -> e_curtate = 1.6, e_complete = 1.75   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_temporary_expectation', lx=[100000.0, 99500.0, 98900.0,
98200.0, 97300.0], x0=40, x=40, n=4)
# -> e_curtate = 3.939, e_complete = 3.9525   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Temporary expectation of life: lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95, n = 2	SOLVED	e_curtate = 1.6, e_complete = 1.75
Temporary expectation of life: lx = [100000, 99500, 98900, 98200, 97300], x0 = 40, x = 40, n = 4	SOLVED	e_curtate = 3.939, e_complete = 3.9525
Temporary expectation of life: x0 = 95, x = 95, n = 2	NEEDS_INPUT	act_temporary_expectation needs lx (survivors at consecutive integer ages from x0 (a list)). Example: solve('act_tempora
Temporary expectation of life: lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95, n = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Death probability from the central death rate, `act_central_death_rate`

The one-year death probability q_x corresponding to a central death rate m_x (deaths per year lived), under a uniform distribution of deaths or a constant force of mortality within the year.

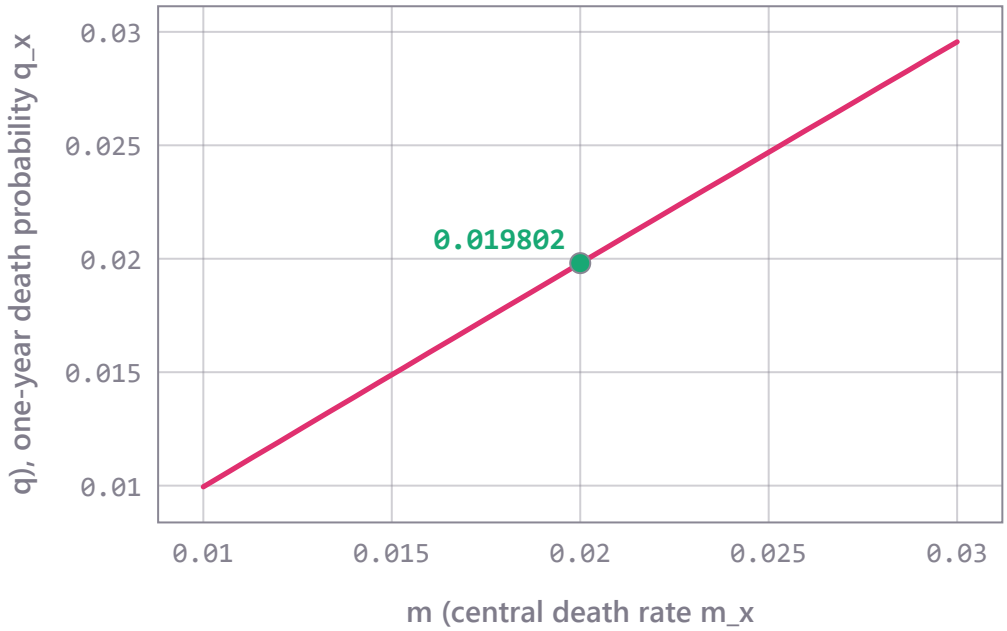
$$\text{UDD: } q_x = \frac{2m_x}{2 + m_x}, \qquad \text{constant force: } q_x = 1 - e^{-m_x}$$

input	meaning	condition
`m`	central death rate m_x (deaths per person-year)	

input	meaning	condition
`assumption`	fractional-age assumption: udd or constant_force	

output	meaning
`q`	one-year death probability q_x
`p`	one-year survival probability p_x

- **Method:** UDD: $q = 2m/(2+m)$; constant force: $q = 1 - e^{-m}$
- **Assumes:** $m > 0$; $m < 2$ under UDD
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



q (one-year death probability q_x) against m (central death rate m_x (deaths per person-year)) from 0.01 to 0.03. Other inputs as in the worked example: assumption = udd. 31 points computed by actuarial-science-ai999x; marked: the worked example, m = 0.02, q = 0.019802.

```
actuarial_science_ai999x.solve('act_central_death_rate', m=0.02, assumption='udd')
# -> q = 0.019802, p = 0.980198   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_central_death_rate', m=0.1, assumption='constant_force')
# -> q = 0.0951626, p = 0.904837   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Death probability from the central death rate: m = 0.02, assumption = udd	SOLVED	q = 0.019802, p = 0.980198
Death probability from the central death rate: m = 0.1, assumption = constant_force	SOLVED	q = 0.0951626, p = 0.904837

question	outcome	result
Death probability from the central death rate: assumption = udd	NEEDS_INPUT	act_central_death_rate needs m (central death rate m_x (deaths per person-year)). Example: solve('act_central_death_rate
Death probability from the central death rate: m = 0.02, assumption = udd, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Survival to a fractional age (UDD, constant force, Balducci), `act_fractional_age`

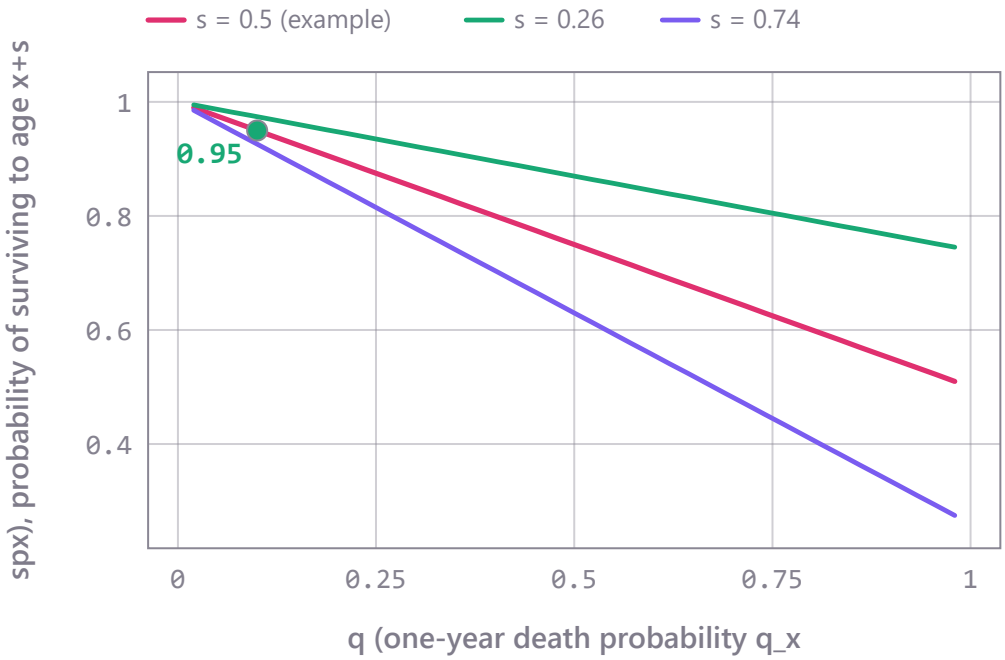
The probability of surviving (or dying within) a fraction s of a year from integer age x, and the force of mortality at x+s, under a uniform distribution of deaths, a constant force of mortality or the Balducci assumption.

$$\text{UDD: } {}_sp_x = 1 - sq_x,\ \mu_{x+s} = \frac{q_x}{1 - sq_x}; \quad \text{CF: } {}_sp_x = p_x^s; \quad \text{Balducci: } {}_sp_x = \frac{p_x}{1 - (1 - s)q_x}$$

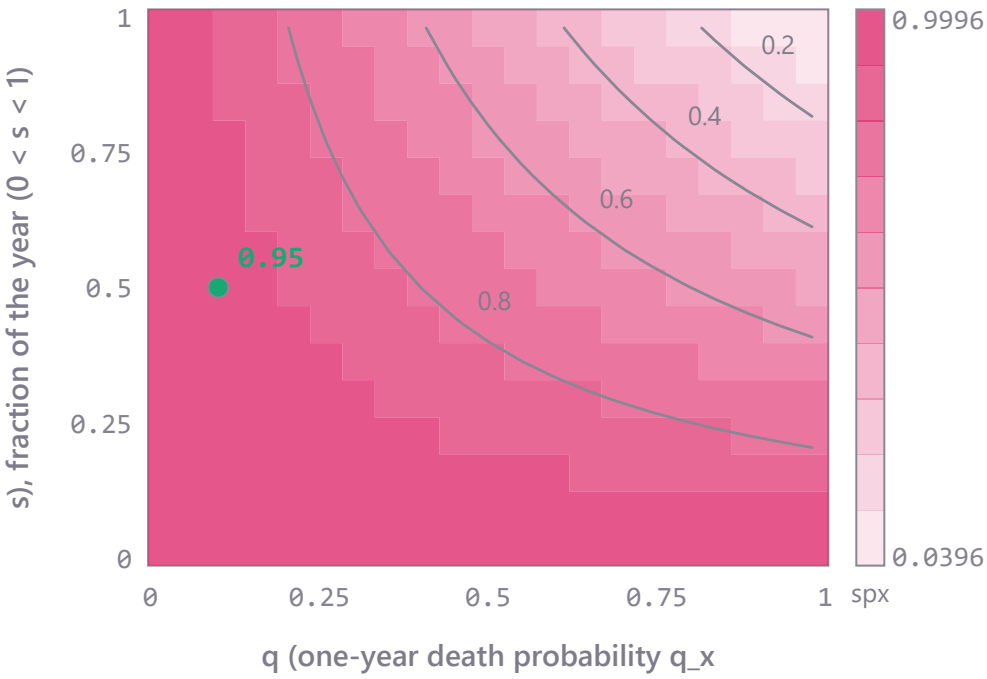
input	meaning	condition
`q`	one-year death probability q_x	
`s`	fraction of the year $(0 < s < 1)$	
`assumption`	fractional-age assumption	

output	meaning
`spx`	probability of surviving to age x+s
`sqx`	probability of dying before age x+s
`mu`	force of mortality at age x+s

- **Method:** UDD: Double subscripts: use braces to clarify q, mu = q/(1 - s q); constant force: $(1 - q)^s$, $mu = -ln$(1-q); Balducci: (1-q)/(1-(1-s)q), mu = q/(1-(1-s)q)
- **Assumes:** $0 < q < 1$; $0 < s < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



spx (probability of surviving to age x+s) against q (one-year death probability q_x) from 0.02 to 0.98. One curve for each value of s (fraction of the year (0 < s < 1)): 0.5, 0.26, 0.74. Other inputs as in the worked example: assumption = udd. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, q = 0.1, spx = 0.95.



spx (probability of surviving to age x+s) over q (one-year death probability q_x) from 0.02 to 0.98 and s (fraction of the year (0 < s < 1)) from 0.02 to 0.98: each of the 21 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 0.0396 to dark 0.9996); iso-lines at spx = 0.2, 0.4, 0.6, 0.8. Other inputs as in the worked example: assumption = udd. Marked: the worked example, q = 0.1, s = 0.5, spx = 0.95.

```
actuarial_science_ai999x.solve('act_fractional_age', q=0.1, s=0.5, assumption='udd')
# -> spx = 0.95, sqx = 0.05, mu = 0.105263   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_fractional_age', q=0.1, s=0.25, assumption='constant_force')
# -> spx = 0.974004, sqx = 0.0259963, mu = 0.105361   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fractional age survival: q = 0.1, s = 0.5, assumption = udd	SOLVED	spx = 0.95, sqx = 0.05, mu = 0.105263
Fractional age survival: q = 0.1, s = 0.25, assumption = constant_force	SOLVED	spx = 0.974004, sqx = 0.0259963, mu = 0.105361
Fractional age survival: s = 0.5, assumption = udd	NEEDS_INPUT	act_fractional_age needs q (one-year death probability q_x). Example: solve('act_fractional_age', q=0.1, s=0.5, assumpti
Fractional age survival: q = 0.1, s = 0.5, assumption = udd, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Fractional age survival: q = 0.1, s = 0.25, assumption = balducci	SOLVED	spx = 0.972973, sqx = 0.027027, mu = 0.108108
Fractional age survival: q = 0.1, s = 1.5, assumption = udd	INVALID_INPUT	Survival to a fractional age (UDD, constant force, Balducci): the fraction of a year s must lie strictly between 0 and 1

Commutation functions D, N, C and M, `act_commutation_functions`

The commutation functions $D_x = v^x$ l_x, N_x, $C_x = v^{x+1}$ d_x and M_x at age x of a complete life table, with the whole life insurance $A_x = M_x/D_x$ and annuity-due ä_x = N_x/D_x.

$$D_x = v^x l_x,\ C_x = v^{x+1} d_x,\ N_x = \sum_{y \geq x} D_y,\ M_x = \sum_{y \geq x} C_y,\ A_x = \frac{M_x}{D_x},\ \ddot{a}_x = \frac{N_x}{D_x}$$

input	meaning	condition
`lx`	survivors at consecutive integer ages from x0, ending with 0 (a list)	
`x0`	age of the first entry of lx	
`x`	age at which the functions are wanted	
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`D_x`	$D_x = v^x$ l_x
`N_x`	N_x = sum of D_y for $y \geq x$
`C_x`	$C_x = v^{x+1}$ d_x
`M_x`	M_x = sum of C_y for $y \geq x$
`A_x`	whole life insurance M_x/D_x
`a_due_x`	whole life annuity-due N_x/D_x

- **Method:** $D_y = v^y$ l_y, $C_y = v^{y+1}$ d_y, N_x = sum D_y, M_x = sum C_y; $A_x = M_x/D_x$, ä_x = N_x/D_x
- **Assumes:** the table closes (last lx = 0); x inside the table; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A_x (whole life insurance M_x/D_x) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: lx = [1000.0, 900.0, 700.0, 400.0, 0.0], x0 = 95, x = 95. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, A_x = 0.864876.

```
actuarial_science_ai999x.solve('act_commutation_functions', lx=[1000.0, 900.0, 700.0, 400.0, 0.0], x0=95, x=95, i=0.05)
# -> D_x = 9.70547, N_x = 27.5402, C_x = 0.924331, M_x = 8.39403, A_x = 0.864876, a_due_x = 2.8376 (SOLVED)
```

```
actuarial_science_ai999x.solve('act_commutation_functions', lx=[1000.0, 900.0, 700.0, 400.0, 0.0], x0=95, x=96, i=0.06)
# -> D_x = 3.34872, N_x = 7.13046, C_x = 0.702039, M_x = 2.94511, A_x = 0.879473, a_due_x = 2.12931 (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Commutation functions: lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95, i = 0.05	SOLVED	D_x = 9.70547, N_x = 27.5402, C_x = 0.924331, M_x = 8.39403, A_x = 0.864876, a_due_x = 2.8376
Commutation functions: lx = [1000, 900, 700, 400, 0], x0 = 95, x = 96, i = 0.06	SOLVED	D_x = 3.34872, N_x = 7.13046, C_x = 0.702039, M_x = 2.94511, A_x = 0.879473, a_due_x = 2.12931
Commutation functions: x0 = 95, x = 95, i = 0.05	NEEDS_INPUT	act_commutation_functions needs lx (survivors at consecutive integer ages from x0, ending with 0 (a list)). Example: sol
Commutation functions: lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95, i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Pure endowment (actuarial discount factor), `act_pure_endowment`

The actuarial present value of 1 paid at time n if a life aged x is then alive, nEx = v^n n_p_x, and the actuarial accumulation factor 1/nEx.

$$ {}_nE_x = v^n \, {}_np_x = v^n \frac{l_{x+n}}{l_x} $$

input	meaning	condition
`lx`	survivors at consecutive integer ages from x0 (a list)	
`x0`	age of the first entry of lx	
`x`	age of the life	
`n`	years until the payment	
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`nEx`	pure endowment Double subscripts: use braces to clarify n_p_x
`accumulation`	actuarial accumulation factor Double subscripts: use braces to clarify

- **Method:** nEx = v^n l_(x+n)/l_x
- **Assumes:** x + n inside the table; l_(x+n) > 0; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



nEx (pure endowment n_E_x = v^n n_p_x) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of n (years until the payment): 4, 2. Other inputs as in the worked example: lx = [100000.0, 99500.0, 98900.0, 98200.0, 97300.0], x0 = 40, x = 40. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, nEx = 0.80049.

```
actuarial_science_ai999x.solve('act_pure_endowment', lx=[100000.0, 99500.0, 98900.0, 98200.0,
97300.0], x0=40, x=40, n=4, i=0.05)
# -> nEx = 0.80049, accumulation = 1.24924   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_pure_endowment', lx=[1000.0, 950.0, 880.0, 780.0], x0=60, x=61,
n=2, i=0.06)
# -> nEx = 0.730734, accumulation = 1.36849   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pure endowment: lx = [100000, 99500, 98900, 98200, 97300], x0 = 40, x = 40, n = 4, i = 0.05	SOLVED	nEx = 0.80049, accumulation = 1.24924
Pure endowment: lx = [1000, 950, 880, 780], x0 = 60, x = 61, n = 2, i = 0.06	SOLVED	nEx = 0.730734, accumulation = 1.36849
Pure endowment: x0 = 40, x = 40, n = 4, i = 0.05	NEEDS_INPUT	act_pure_endowment needs lx (survivors at consecutive integer ages from x0 (a list)). Example: solve('act_pure_endowment
Pure endowment: lx = [100000, 99500, 98900, 98200, 97300], x0 = 40, x = 40, n = 4, i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

De Moivre's law of mortality, `act_de_moivre_law`

Survival and death probabilities, the force of mortality, the complete expectation of life and the variance of the future lifetime under de Moivre's law (uniform future lifetime up to the limiting age omega).

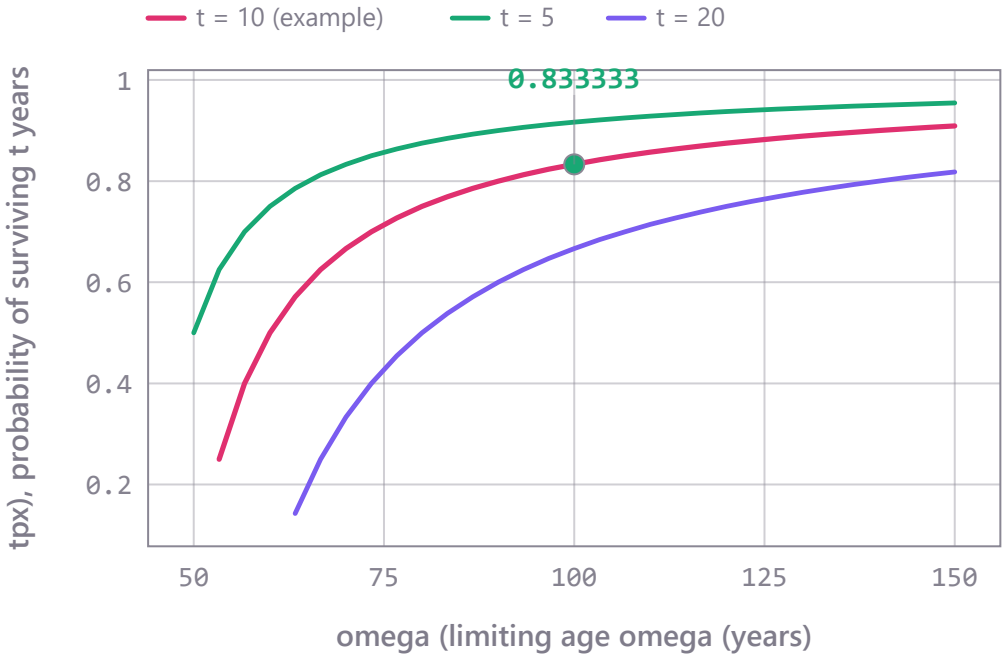
$$ {}_tp_x = \frac{\omega - x - t}{\omega - x},\ \mu_{x+t} = \frac{1}{\omega - x - t},\ \mathring{e}_x = \frac{\omega - x}{2} $$

input	meaning	condition
`omega`	limiting age omega (years)	
`x`	age of the life (years)	
`t`	number of years	

output	meaning
`tpx`	probability of surviving t years
`tqx`	probability of dying within t years
`mu`	force of mortality at age x+t
`e_complete`	complete expectation of life e°_x
`var_T`	variance of the future lifetime

- **Method:** Double subscripts: use braces to clarify; mu_(x+t) = 1/(omega - x - t); $\mathring{e}_x = (\omega - x)/2$; Var $T = (\omega - x)^2/12$
- **Assumes:** $0 \le x < \omega$; $0 \le t < \omega - x$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tpx (probability of surviving t years) against omega (limiting age omega (years)) from 50 to 150. One curve for each value of t (number of years): 10, 5, 20. Other inputs as in the worked example: x = 40.0. 30 points per curve computed by actuarial-science-ai999x; marked: the worked example, omega = 100, tpx = 0.833333.

```
actuarial_science_ai999x.solve('act_de_moivre_law', omega=100.0, x=40.0, t=10.0)
# -> tpx = 0.833333, tqx = 0.166667, mu = 0.02, e_complete = 30, var_T = 300   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_de_moivre_law', omega=110.0, x=65.0, t=20.0)
# -> tpx = 0.555556, tqx = 0.444444, mu = 0.04, e_complete = 22.5, var_T = 168.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
De Moivre's law: omega = 100, x = 40, t = 10	SOLVED	tpx = 0.833333, tqx = 0.166667, mu = 0.02, e_complete = 30, var_T = 300
De Moivre's law: omega = 110, x = 65, t = 20	SOLVED	tpx = 0.555556, tqx = 0.444444, mu = 0.04, e_complete = 22.5, var_T = 168.75
De Moivre's law: x = 40, t = 10	NEEDS_INPUT	act_de_moivre_law needs omega (limiting age omega (years)). Example: solve('act_de_moivre_law', omega=100, x=40, t=10).
De Moivre's law: omega = 100, x = 40, t = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Constant force of mortality (exponential lifetime), `act_constant_force_mortality`

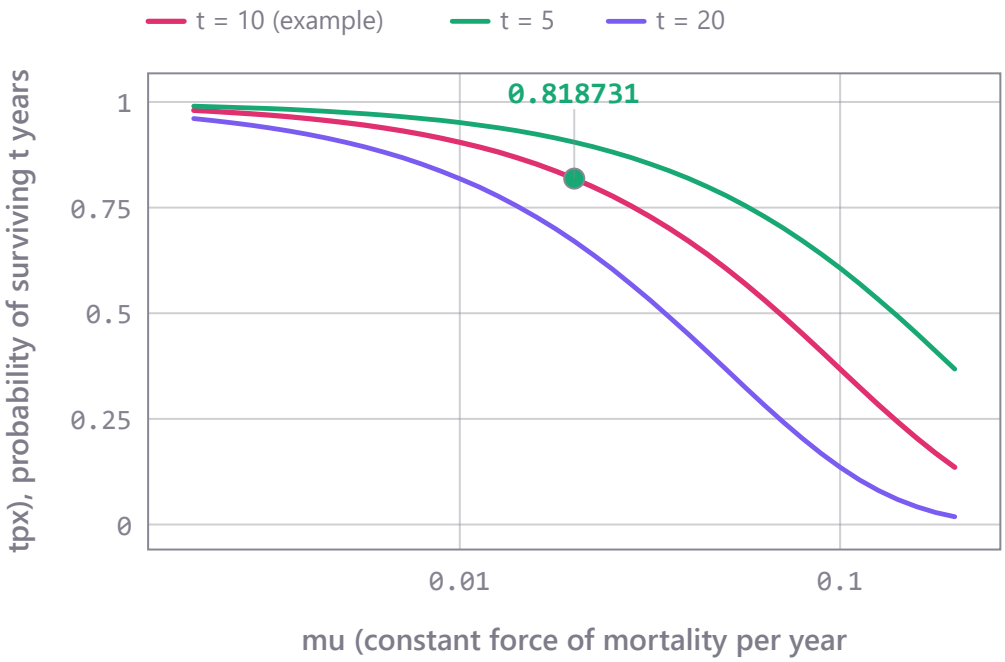
Survival and death probabilities, the complete and curtate expectations of life and the variance of the future lifetime when the force of mortality mu is constant.

$$ {}_tp_x = e^{-\mu t}, \quad \mathring{e}_x = \frac{1}{\mu}, \quad e_x = \frac{e^{-\mu}}{1 - e^{-\mu}} $$

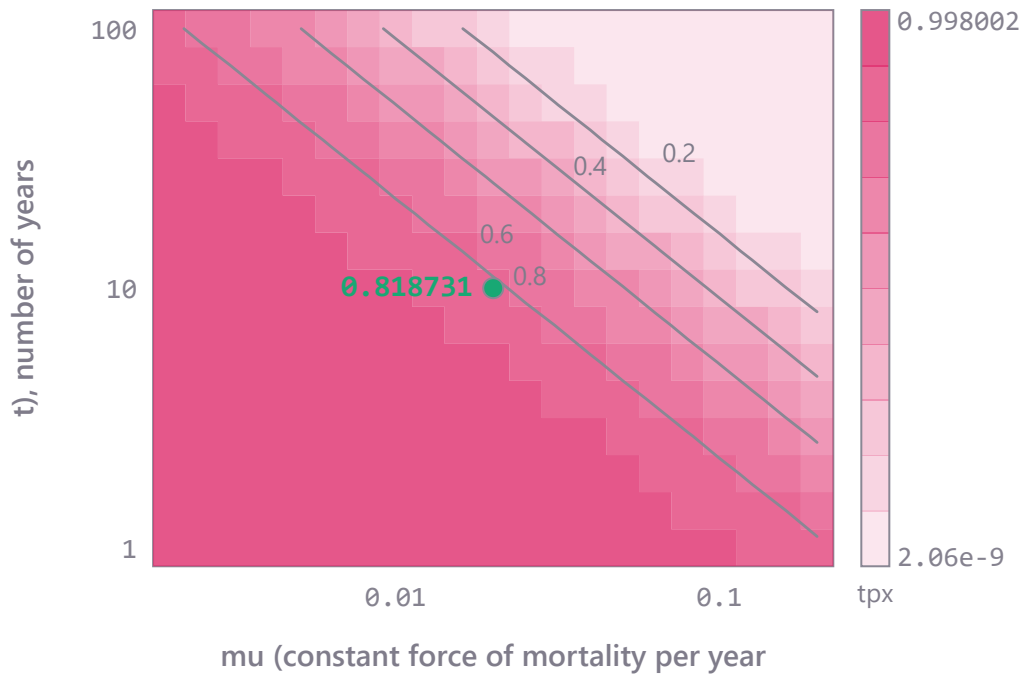
input	meaning	condition
`mu`	constant force of mortality per year	
`t`	number of years	

output	meaning
`tpx`	probability of surviving t years
`tqx`	probability of dying within t years
`e_complete`	complete expectation $1/mu$
`e_curtate`	curtate expectation $p/(1-p)$
`var_T`	variance of the future lifetime $1/mu^2$

- **Method:** t_p_x = e^(-mu t); e° = $1/mu$; e_x = e^(-mu)/(1 - e^(-mu))
- **Assumes:** $0 < mu < 5$; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tpx (probability of surviving t years) against mu (constant force of mortality per year) from 0.002 to 0.2 on a logarithmic axis. One curve for each value of t (number of years): 10, 5, 20. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, mu = 0.02, tpx = 0.818731.



tpx (probability of surviving t years) over mu (constant force of mortality per year) from 0.002 to 0.2 and t (number of years) from 1 to 100 (logarithmic axes): each of the 21 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 2.0612e-09 to dark 0.998002); iso-lines at tpx = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, mu = 0.02, t = 10, tpx = 0.818731.

```
actuarial_science_ai999x.solve('act_constant_force_mortality', mu=0.02, t=10.0)
# -> tpx = 0.818731, tqx = 0.181269, e_complete = 50, e_curtate = 49.5017, var_T = 2500   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_constant_force_mortality', mu=0.1, t=5.0)
# -> tpx = 0.606531, tqx = 0.393469, e_complete = 10, e_curtate = 9.50833, var_T = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Constant force of mortality: mu = 0.02, t = 10	SOLVED	tpx = 0.818731, tqx = 0.181269, e_complete = 50, e_curtate = 49.5017, var_T = 2500
Constant force of mortality: mu = 0.1, t = 5	SOLVED	tpx = 0.606531, tqx = 0.393469, e_complete = 10, e_curtate = 9.50833, var_T = 100
Constant force of mortality: t = 10	NEEDS_INPUT	act_constant_force_mortality needs mu (constant force of mortality per year). Example: solve('act_constant_force_mortali
Constant force of mortality: mu = 0.02, t = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Makeham's law of mortality, `act_makeham_law`

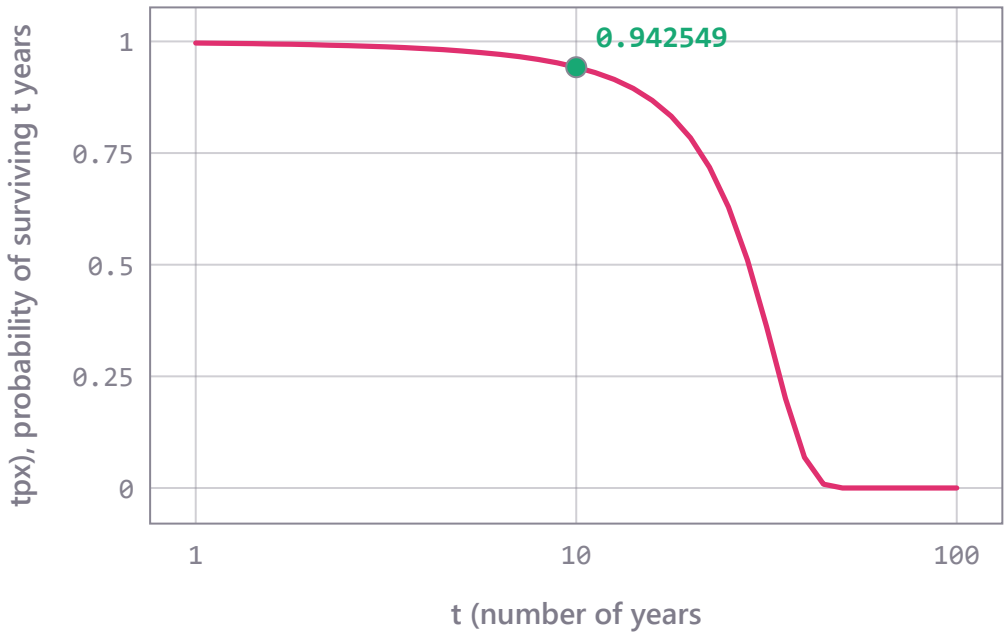
Survival and death probabilities, the force of mortality at x and x+t and the complete expectation of life under Makeham's law $mu_x = A + B\,c^x$.

$$\mu_x = A + Bc^x, \quad {}_tp_x = \exp\left(-At - \frac{Bc^x(c^t-1)}{\ln c}\right), \quad \mathring{e}_x = \int_0^\infty {}_tp_x\,dt$$

input	meaning	condition
`A`	Makeham's age-independent term A (per year)	
`B`	Makeham's B (per year)	
`c`	Makeham's c (> 1)	
`x`	age of the life (years)	
`t`	number of years	

output	meaning
`tpx`	probability of surviving t years
`tqx`	probability of dying within t years
`mu_x`	force of mortality at age x
`mu_xt`	force of mortality at age x+t
`e_complete`	complete expectation of life e°_x

- **Method:** Double subscripts: use braces to clarify(-A t - B c^x $(c^t - 1)/ln$ c); e°_x = integral of t_p_x (Gauss-Legendre)
- **Assumes:** $0 \leq A < 0.1$; $0 < B < 0.1$; $1 < c < 1.5$; $0 \leq x \leq 120$; $0 \leq t \leq 100$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tpx (probability of surviving t years) against t (number of years) from 1 to 100 on a logarithmic axis. Other inputs as in the worked example: A = 0.00022, B = 2.7e-06, c = 1.124, x = 60.0. 41 points computed by actuarial-science-ai999x; marked: the worked example, t = 10, tpx = 0.942549.

```
actuarial_science_ai999x.solve('act_makeham_law', A=0.00022, B=2.7e-06, c=1.124, x=60.0, t=10.0)
# -> tpx = 0.942549, tqx = 0.0574508, mu_x = 0.00322153, mu_xt = 0.00988063, e_complete = 27.2097
(SOLVED)
```

```
actuarial_science_ai999x.solve('act_makeham_law', A=0.0007, B=5e-05, c=1.09, x=40.0, t=20.0)
# -> tpx = 0.906731, tqx = 0.0932687, mu_x = 0.00227047, mu_xt = 0.00950156, e_complete = 40.088
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Makeham's law: A = 0.00022, B = 2.7e-06, c = 1.124, x = 60, t = 10	SOLVED	tpx = 0.942549, tqx = 0.0574508, mu_x = 0.00322153, mu_xt = 0.00988063, e_complete = 27.2097
Makeham's law: A = 0.0007, B = 5e-05, c = 1.09, x = 40, t = 20	SOLVED	tpx = 0.906731, tqx = 0.0932687, mu_x = 0.00227047, mu_xt = 0.00950156, e_complete = 40.088
Makeham's law: B = 2.7e-06, c = 1.124, x = 60, t = 10	NEEDS_INPUT	act_makeham_law needs A (Makeham's age-independent term A (per year)). Example: solve('act_makeham_law', A=0.00022, B=2.
Makeham's law: A = 0.00022, B = 2.7e-06, c = 1.124, x = 60, t = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Makeham's law: A = 0.00022, B = 2.7e-06, c = 0.9, x = 60, t = 10	INVALID_INPUT	Makeham's law of mortality: c must lie between 1 and 1.5.

Gompertz's law of mortality, `act_gompertz_law`

Survival and death probabilities, the force of mortality at x and x+t and the complete expectation of life under Gompertz's law $mu_x = B\,c^x$.

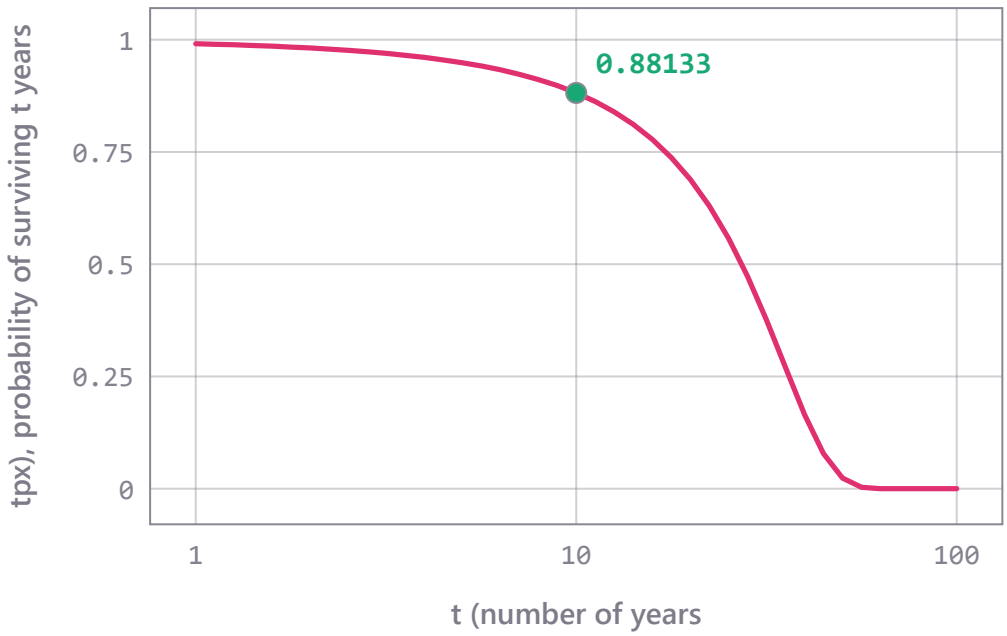
$$\mu_x = Bc^x, \quad {}_tp_x = \exp\left(-\frac{Bc^x(c^t-1)}{\ln c}\right)$$

input	meaning	condition
`B`	Gompertz B (per year)	
`c`	Gompertz c (> 1)	
`x`	age of the life (years)	
`t`	number of years	

output	meaning
`tpx`	probability of surviving t years
`tqx`	probability of dying within t years
`mu_x`	force of mortality at age x
`mu_xt`	force of mortality at age x+t
`e_complete`	complete expectation of life e°_x

- **Method:** Double subscripts: use braces to clarify(-B $c^x\ (c^t-1)/ln$ c); e°_x = integral of t_p_x (Gauss-Legendre)
- **Assumes:** $0 < B < 0.1$; $1 < c < 1.5$; $0 \le x \le 120$; $0 \le t \le 100$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tpx (probability of surviving t years) against t (number of years) from 1 to 100 on a logarithmic axis. Other inputs as in the worked example: B = 0.0003, c = 1.07, x = 50.0. 41 points computed by actuarial-science-ai999x; marked: the worked example, t = 10, tpx = 0.88133.

```
actuarial_science_ai999x.solve('act_gompertz_law', B=0.0003, c=1.07, x=50.0, t=10.0)
# -> tpx = 0.88133, tqx = 0.11867, mu_x = 0.00883711, mu_xt = 0.0173839, e_complete = 26.6911
(SOLVED)
```

```
actuarial_science_ai999x.solve('act_gompertz_law', B=2.7e-06, c=1.124, x=70.0, t=5.0)
# -> tpx = 0.936484, tqx = 0.063516, mu_x = 0.00966063, mu_xt = 0.0173315, e_complete = 18.5554
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gompertz's law: B = 0.0003, c = 1.07, x = 50, t = 10	SOLVED	tpx = 0.88133, tqx = 0.11867, mu_x = 0.00883711, mu_xt = 0.0173839, e_complete = 26.6911
Gompertz's law: B = 2.7e-06, c = 1.124, x = 70, t = 5	SOLVED	tpx = 0.936484, tqx = 0.063516, mu_x = 0.00966063, mu_xt = 0.0173315, e_complete = 18.5554
Gompertz's law: c = 1.07, x = 50, t = 10	NEEDS_INPUT	act_gompertz_law needs B (Gompertz B (per year)). Example: solve('act_gompertz_law', B=0.0003, c=1.07, x=50, t=10).
Gompertz's law: B = 0.0003, c = 1.07, x = 50, t = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Whole life insurance (fully discrete), `act_whole_life_insurance`

The actuarial present value of a whole life insurance paying S at the end of the year of death of a life aged x (A_x), its second moment at double the force of interest, and the standard deviation of the present value of the benefit.

$$A_x = \sum_{k\geq 0} v^{k+1}\,{}_kp_x\,q_{x+k}, \quad \mathrm{Var}(Z) = S^2\left({}^2A_x - A_x^2\right)$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`S`	sum insured	

output	meaning
`A`	A_x per unit sum insured
`A2`	second moment 2A_x (at double the force of interest)
`EPV`	actuarial present value S A_x
`sd`	standard deviation of the present value S sqrt(2A - A^2)

- **Method:** A_x = sum_k v^{k+1} k_p_x q_(x+k); 2A_x with v^2; Var $Z = S^2\left(2A - A^2\right)$
- **Assumes:** the table closes (last q = 1); $0 < i < 1$; $S > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (A_x per unit sum insured) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], S = 1000.0. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, A = 0.865658.

```
actuarial_science_ai999x.solve('act_whole_life_insurance', q=[0.1, 0.2, 0.5, 1.0], i=0.05,
S=1000.0)
# -> A = 0.865658, A2 = 0.751089, EPV = 865.658, sd = 41.5407   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_whole_life_insurance', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3,
1.0], i=0.04, S=100000.0)
# -> A = 0.795949, A2 = 0.635963, EPV = 79594.9, sd = 4928.52   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Whole life insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000	SOLVED	A = 0.865658, A2 = 0.751089, EPV = 865.658, sd = 41.5407
Whole life insurance: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, S = 100000	SOLVED	A = 0.795949, A2 = 0.635963, EPV = 79594.9, sd = 4928.52
Whole life insurance: i = 0.05, S = 1000	NEEDS_INPUT	act_whole_life_insurance needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_who
Whole life insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Whole life insurance: q = [0.1, 0.2, 0.5], i = 5%, S = 1000	INVALID_INPUT	Whole life insurance (fully discrete): a whole-life table must close: its last q must be 1.
Whole-life insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000	SOLVED	A = 0.865658, A2 = 0.751089, EPV = 865.658, sd = 41.5407

Term insurance (fully discrete), `act_term_insurance`

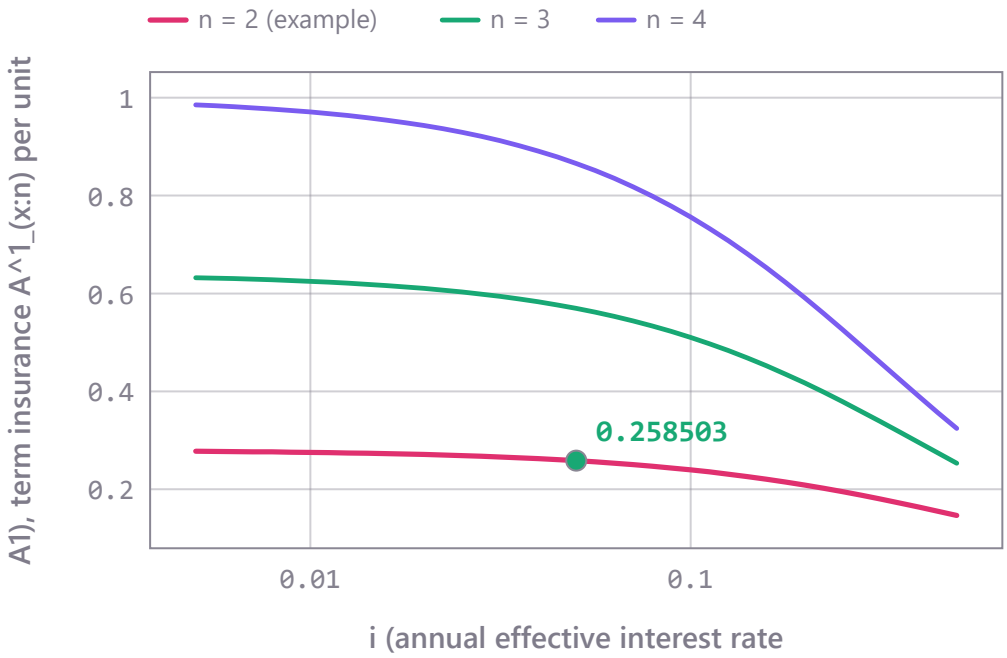
The actuarial present value of an n-year term insurance paying S at the end of the year of death if death occurs within n years, its second moment and the standard deviation of the present value.

$$A^1_{x:\overline{n}|} = \sum_{k=0}^{n-1} v^{k+1}\ {}_kp_x\ q_{x+k}$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... for at least n years (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`n`	term in years	
`S`	sum insured	

output	meaning
`A1`	term insurance Missing superscript or subscript argument(x:n) per unit
`A1_2`	second moment at double the force of interest
`EPV`	actuarial present value S Missing superscript or subscript argument(x:n)
`sd`	standard deviation of the present value

- **Method:** Missing superscript or subscript argument(x:n) = sum_(k<n) v^{k+1} k_p_x q_(x+k)
- **Assumes:** at least n death probabilities; $0 < i < 1$; $S > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A1 (term insurance A^1_(x:n) per unit) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of n (term in years): 2, 3, 4. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], S = 1000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, A1 = 0.258503.

```
actuarial_science_ai999x.solve('act_term_insurance', q=[0.1, 0.2, 0.5, 1.0], i=0.05, n=2, S=1000.0)
# -> A1 = 0.258503, A1_2 = 0.238789, EPV = 258.503, sd = 414.687   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_term_insurance', q=[0.01, 0.012, 0.015, 0.018, 0.022], i=0.06,
n=5, S=50000.0)
# -> A1 = 0.0616161, A1_2 = 0.0511563, EPV = 3080.8, sd = 10881.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Term insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 2, S = 1000	SOLVED	A1 = 0.258503, A1_2 = 0.238789, EPV = 258.503, sd = 414.687
Term insurance: q = [0.01, 0.012, 0.015, 0.018, 0.022], i = 0.06, n = 5, S = 50000	SOLVED	A1 = 0.0616161, A1_2 = 0.0511563, EPV = 3080.8, sd = 10881.2
Term insurance: i = 0.05, n = 2, S = 1000	NEEDS_INPUT	act_term_insurance needs q (death probabilities q_x, q_(x+1), ... for at least n years (a list)). Example: solve('act_te
Term insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 2, S = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Term insurance: q = [0.1, 0.2], i = 5%, n = 3, S = 1000	INVALID_INPUT	Term insurance (fully discrete): the table must cover the whole term (at least n death probabilities).

Endowment insurance (fully discrete), `act_endowment_insurance`

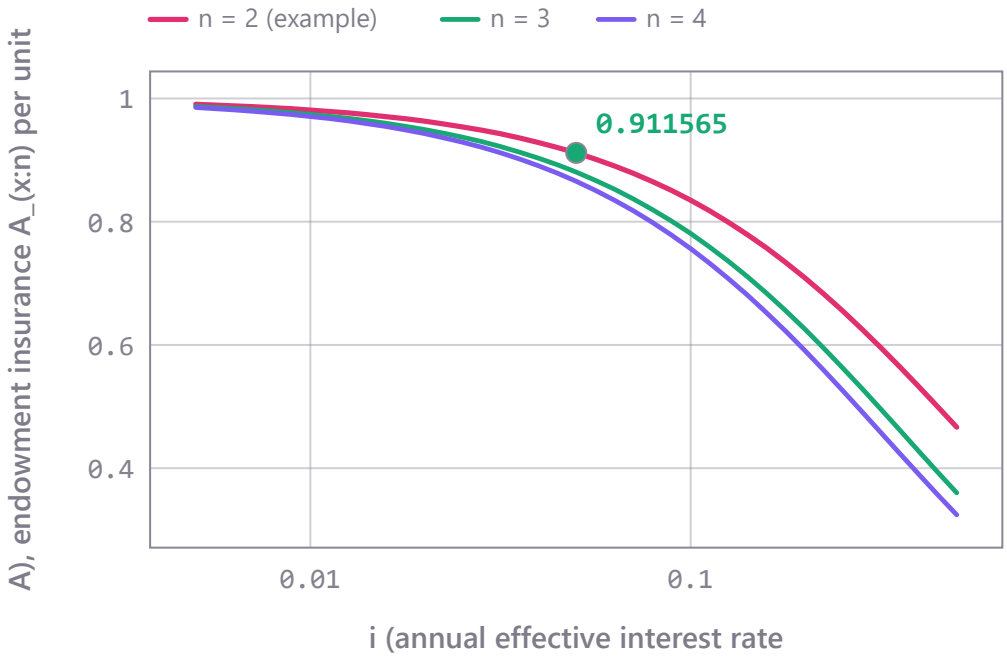
The actuarial present value of an n-year endowment insurance paying S at the end of the year of death within n years or at time n on survival, split into its term insurance and pure endowment parts, with the second moment and standard deviation.

$$A_{x:\overline{n}|} = A^{1}_{x:\overline{n}|} + {}_{n}E_{x}$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... for at least n years (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`n`	term in years	
`S`	sum insured	

output	meaning
`A`	endowment insurance A_(x:n) per unit
`A1`	term insurance part Missing superscript or subscript argument (x:n)
`nEx`	pure endowment part n_E_x
`A2`	second moment at double the force of interest
`EPV`	actuarial present value S A_(x:n)
`sd`	standard deviation of the present value

- **Method:** A_(x:n) = A^1_(x:n) + n_E_x
- **Assumes:** at least n death probabilities; $0 < i < 1$; $S > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (endowment insurance A_(x:n) per unit) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of n (term in years): 2, 3, 4. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], S = 1000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, A = 0.911565.

```
actuarial_science_ai999x.solve('act_endowment_insurance', q=[0.1, 0.2, 0.5, 1.0], i=0.05, n=2,
S=1000.0)
# -> A = 0.911565, A1 = 0.258503, nEx = 0.653061, A2 = 0.831135, EPV = 911.565, sd = 13.6054
(SOLVED)
```

```
actuarial_science_ai999x.solve('act_endowment_insurance', q=[0.01, 0.012, 0.015, 0.018, 0.022],
i=0.06, n=5, S=50000.0)
# -> A = 0.753048, A1 = 0.0616161, nEx = 0.691432, A2 = 0.567834, EPV = 37652.4, sd = 1372.27
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Endowment insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 2, S = 1000	SOLVED	A = 0.911565, A1 = 0.258503, nEx = 0.653061, A2 = 0.831135, EPV = 911.565, sd = 13.6054
Endowment insurance: q = [0.01, 0.012, 0.015, 0.018, 0.022], i = 0.06, n = 5, S = 50000	SOLVED	A = 0.753048, A1 = 0.0616161, nEx = 0.691432, A2 = 0.567834, EPV = 37652.4, sd = 1372.27
Endowment insurance: i = 0.05, n = 2, S = 1000	NEEDS_INPUT	act_endowment_insurance needs q (death probabilities q_x, q_(x+1), ... for at least n years (a list)). Example: solve('a
Endowment insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 2, S = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Deferred whole life insurance, `act_deferred_insurance`

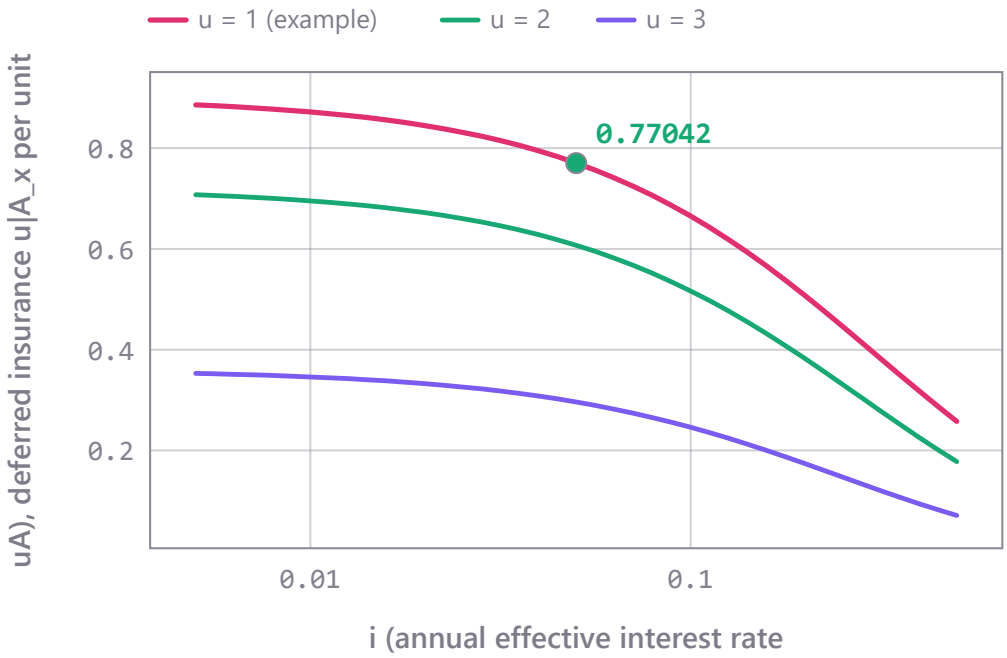
The actuarial present value of a whole life insurance of S that pays only if death occurs after u years (u| $A_x = uE_x$ A_(x+u)).

$$ {}_{u|}A_x = \sum_{k \geq u} v^{k+1}\, {}_kp_x\, q_{x+k} = {}_uE_x\, A_{x+u} $$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`u`	deferment in years	
`S`	sum insured	

output	meaning
`uA`	deferred insurance u|A_x per unit
`EPV`	actuarial present value S u|A_x
`uEx`	pure endowment u_E_x

- **Method:** u|A_x = sum_(k>=u) v^{k+1} k_p_x q_(x+k) = u_E_x A_(x+u)
- **Assumes:** the table closes; u < length of the table; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



uA (deferred insurance u|A_x per unit) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of u (deferment in years): 1, 2, 3. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], S = 1000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, uA = 0.77042.

```
actuarial_science_ai999x.solve('act_deferred_insurance', q=[0.1, 0.2, 0.5, 1.0], i=0.05, u=1,
S=1000.0)
# -> uA = 0.77042, EPV = 770.42, uEx = 0.857143   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_deferred_insurance', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3,
1.0], i=0.04, u=3, S=10000.0)
# -> uA = 0.707282, EPV = 7072.82, uEx = 0.802826   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Deferred insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, u = 1, S = 1000	SOLVED	uA = 0.77042, EPV = 770.42, uEx = 0.857143
Deferred insurance: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, u = 3, S = 10000	SOLVED	uA = 0.707282, EPV = 7072.82, uEx = 0.802826
Deferred insurance: i = 0.05, u = 1, S = 1000	NEEDS_INPUT	act_deferred_insurance needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_defer
Deferred insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, u = 1, S = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Whole life annuity, `act_whole_life_annuity`

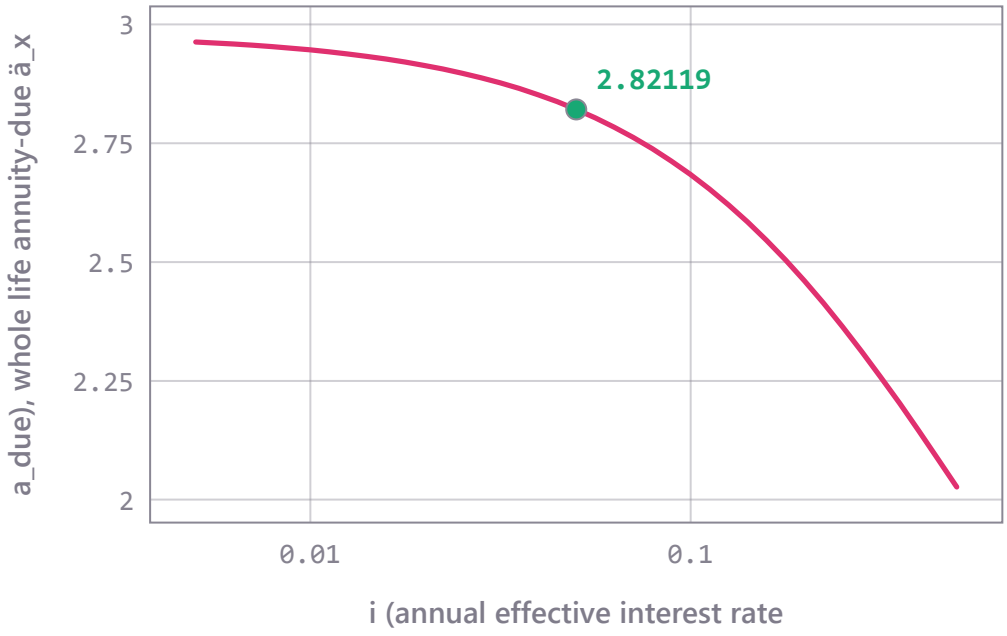
The actuarial present values of a whole life annuity-due (ä_x) and annuity-immediate (a_x) for a life aged x, the value of an annual payment P, and the standard deviation of the present value of the annuity-due payments.

$$\ddot{a}_x = \sum_{k\geq 0} v^k \, {}_kp_x, \quad a_x = \ddot{a}_x - 1, \quad \mathrm{Var}(Y) = \frac{{}^2A_x - A_x^2}{d^2}$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`P`	annual payment	

output	meaning
`a_due`	whole life annuity-due ä_x
`a_imm`	whole life annuity-immediate a_x
`EPV`	actuarial present value P ä_x
`sd`	standard deviation of the annuity-due present value

- **Method:** ä_x = sum_k v^k k_p_x; a_x = ä_x - 1; Var = P^2 $(2A_x - A_x^2)/d^2$
- **Assumes:** the table closes; $0 < i < 1$; $P > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a_due (whole life annuity-due ä_x) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], P = 1000.0. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, a_due = 2.82119.

```
actuarial_science_ai999x.solve('act_whole_life_annuity', q=[0.1, 0.2, 0.5, 1.0], i=0.05, P=1000.0)
# -> a_due = 2.82119, a_imm = 1.82119, EPV = 2821.19, sd = 872.355   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_whole_life_annuity', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3,
1.0], i=0.04, P=12000.0)
# -> a_due = 5.30534, a_imm = 4.30534, EPV = 63664.1, sd = 15377   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Whole life annuity: q = [0.1, 0.2, 0.5, 1], i = 0.05, P = 1000	SOLVED	a_due = 2.82119, a_imm = 1.82119, EPV = 2821.19, sd = 872.355
Whole life annuity: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, P = 12000	SOLVED	a_due = 5.30534, a_imm = 4.30534, EPV = 63664.1, sd = 15377
Whole life annuity: i = 0.05, P = 1000	NEEDS_INPUT	act_whole_life_annuity needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_whole
Whole life annuity: q = [0.1, 0.2, 0.5, 1], i = 0.05, P = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Temporary life annuity, `act_temporary_annuity`

The actuarial present values of an n-year temporary life annuity-due (ä_(x:n)) and annuity-immediate (a_(x:n)), and the value of an annual payment P paid in advance.

$$\ddot{a}_{x:\overline{n}|} = \sum_{k=0}^{n-1} v^k \, {}_kp_x, \quad a_{x:\overline{n}|} = \ddot{a}_{x:\overline{n}|} - 1 + {}_nE_x$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... for at least n years (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`n`	number of years	
`P`	annual payment	

output	meaning
`a_due`	temporary annuity-due ä_(x:n)
`a_imm`	temporary annuity-immediate a_(x:n)
`EPV`	actuarial present value P ä_(x:n)

- **Method:** ä_(x:n) = sum_(k<n) v^k k_p_x; a_(x:n) = ä_(x:n) - 1 + n_E_x
- **Assumes:** at least n death probabilities; $0 < i < 1$; $P > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a_due (temporary annuity-due ä_(x:n)) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of n (number of years): 3, 4. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], P = 1000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, a_due = 2.5102.

```
actuarial_science_ai999x.solve('act_temporary_annuity', q=[0.1, 0.2, 0.5, 1.0], i=0.05, n=3,
P=1000.0)
# -> a_due = 2.5102, a_imm = 1.82119, EPV = 2510.2   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_temporary_annuity', q=[0.01, 0.012, 0.015, 0.018, 0.022],
i=0.06, n=5, P=2000.0)
# -> a_due = 4.36282, a_imm = 4.05425, EPV = 8725.64   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Temporary life annuity: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, P = 1000	SOLVED	a_due = 2.5102, a_imm = 1.82119, EPV = 2510.2
Temporary life annuity: q = [0.01, 0.012, 0.015, 0.018, 0.022], i = 0.06, n = 5, P = 2000	SOLVED	a_due = 4.36282, a_imm = 4.05425, EPV = 8725.64
Temporary life annuity: i = 0.05, n = 3, P = 1000	NEEDS_INPUT	act_temporary_annuity needs q (death probabilities q_x, q_(x+1), ... for at least n years (a list)). Example: solve('act
Temporary life annuity: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, P = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Deferred whole life annuity-due, `act_deferred_annuity`

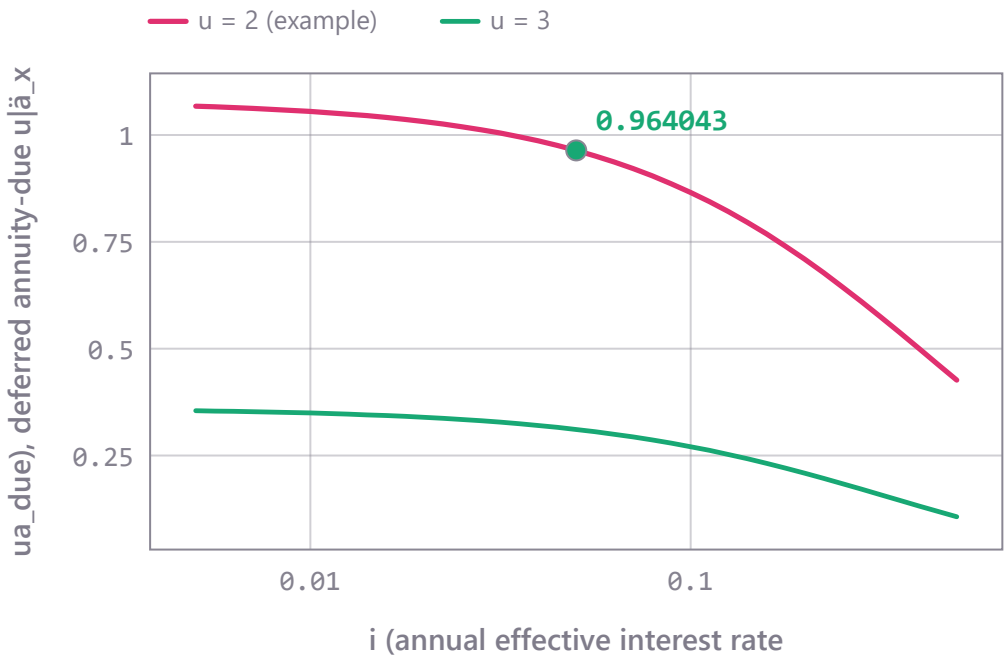
The actuarial present value of a whole life annuity-due of P a year starting u years from now if the life is then alive (u|ä_x = uE_x ä_(x+u)).

$$ {}_{u|}\ddot{a}_x = \sum_{k \geq u} v^k \, {}_kp_x = {}_uE_x \, \ddot{a}_{x+u} $$

input	meaning	condition
q	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
i	annual effective interest rate (5% or 0.05)	
u	deferment in years	
P	annual payment	

output	meaning
ua_due	deferred annuity-due u|ä_x
EPV	actuarial present value P u|ä_x

- **Method:** u|ä_x = sum_(k>=u) v^k Double subscripts: use braces to clarify ä_(x+u)
- **Assumes:** the table closes; u < length of the table; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ua_due (deferred annuity-due u|ä_x) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of u (deferment in years): 2, 3. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], P = 1000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, ua_due = 0.964043.

```
actuarial_science_ai999x.solve('act_deferred_annuity', q=[0.1, 0.2, 0.5, 1.0], i=0.05, u=2,
P=1000.0)
# -> ua_due = 0.964043, EPV = 964.043   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_deferred_annuity', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1.0],
i=0.04, u=3, P=5000.0)
# -> ua_due = 2.48415, EPV = 12420.7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Deferred annuity: q = [0.1, 0.2, 0.5, 1], i = 0.05, u = 2, P = 1000	SOLVED	ua_due = 0.964043, EPV = 964.043
Deferred annuity: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, u = 3, P = 5000	SOLVED	ua_due = 2.48415, EPV = 12420.7
Deferred annuity: i = 0.05, u = 2, P = 1000	NEEDS_INPUT	act_deferred_annuity needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_deferre
Deferred annuity: q = [0.1, 0.2, 0.5, 1], i = 0.05, u = 2, P = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Continuous whole life insurance and annuity under UDD, `act_continuous_insurance_udd`

The actuarial present value of a whole life insurance paid at the moment of death (Ā_x = (i/delta) A_x) and of a continuous whole life annuity, from an annual life table with deaths uniformly distributed within each year.

$$\bar{A}_x = \frac{i}{\delta} A_x, \qquad \bar{a}_x = \frac{1 - \bar{A}_x}{\delta}$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`A_bar`	insurance payable at the moment of death Ā_x
`A`	insurance payable at the end of the year of death A_x
`a_bar`	continuous whole life annuity ā_x

- **Method:** Ā_x = (i/delta) A_x; ā_x = (1 - Ā_x)/delta
- **Assumes:** the table closes; $0 < i < 1$; uniform distribution of deaths within each year
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A_bar (insurance payable at the moment of death Ā_x) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0]. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, A_bar = 0.887123.

```
actuarial_science_ai999x.solve('act_continuous_insurance_udd', q=[0.1, 0.2, 0.5, 1.0], i=0.05)
# -> A_bar = 0.887123, A = 0.865658, a_bar = 2.31351   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_continuous_insurance_udd', q=[0.02, 0.03, 0.05, 0.08, 0.15,
0.3, 1.0], i=0.06)
# -> A_bar = 0.734604, A = 0.71341, a_bar = 4.55467   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Continuous insurance under UDD: q = [0.1, 0.2, 0.5, 1], i = 0.05	SOLVED	A_bar = 0.887123, A = 0.865658, a_bar = 2.31351
Continuous insurance under UDD: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.06	SOLVED	A_bar = 0.734604, A = 0.71341, a_bar = 4.55467
Continuous insurance under UDD: i = 0.05	NEEDS_INPUT	act_continuous_insurance_udd needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act
Continuous insurance under UDD: q = [0.1, 0.2, 0.5, 1], i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

m-thly whole life annuity-due under UDD, `act_mthly_annuity_udd`

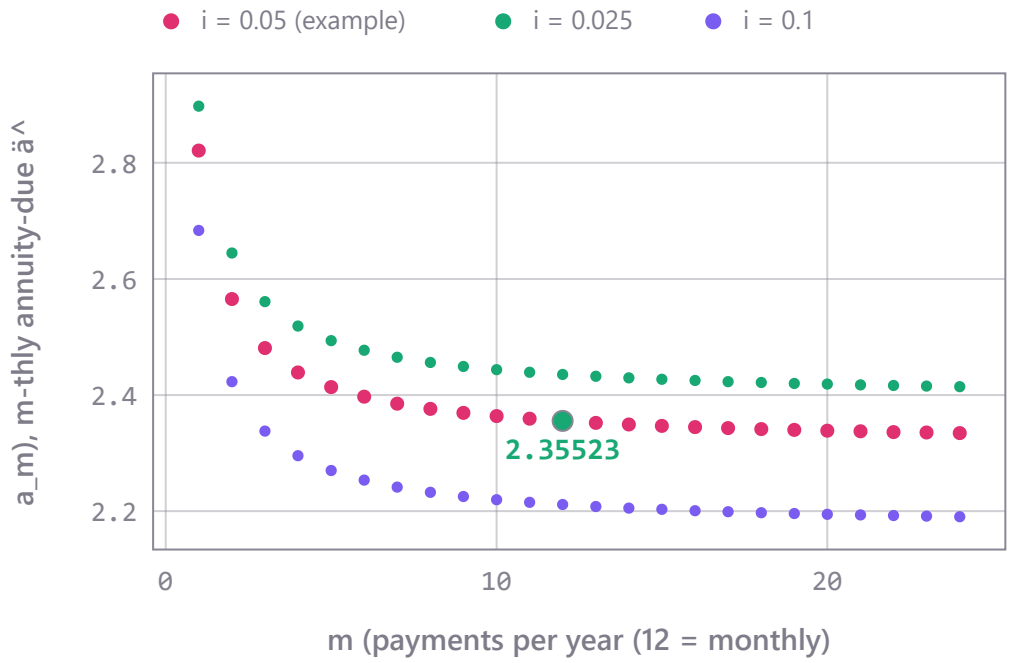
The actuarial present value of a whole life annuity-due paying $1/m$ at the start of every $1/m$ of a year, from an annual life table under the uniform distribution of deaths (ä^(m)$_x$ = α(m) ä_x - beta(m)).

$$\ddot{a}_x^{(m)} = \alpha(m)\,\ddot{a}_x - \beta(m), \quad \alpha(m) = \frac{id}{i^{(m)}d^{(m)}},\ \beta(m) = \frac{i - i^{(m)}}{i^{(m)}d^{(m)}}$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`m`	payments per year (12 = monthly)	

output	meaning
`a_m`	m-thly annuity-due ä^(m)_x (total 1 a year)
`alpha`	alpha(m) = i d/(i(m) d(m))
`beta`	beta(m) = (i − i(m))/(i(m) d(m))
`a_due`	annual annuity-due ä_x

- **Method:** ä^(m)$_x$ = α(m) ä_x - beta(m)
- **Assumes:** the table closes; $0 < i < 1$; $1 \leq m \leq 365$; UDD within each year
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a_m (m-thly annuity-due ä^(m)_x (total 1 a year)) against m (payments per year (12 = monthly)) from 1 to 24. One curve for each value of i (annual effective interest rate (5% or 0.05)): 0.05, 0.025, 0.1. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0]. 24 points per curve computed by actuarial-science-ai999x; marked: the worked example, m = 12, a_m = 2.35523.



a_m (m-thly annuity-due ä^(m)_x (total 1 a year)) over i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 and m (payments per year (12 = monthly)) from 1 to 24 (logarithmic horizontal axis): each of the 21 × 12 cells is one run of actuarial-science-ai999x, shaded by its value (light 1.50048 to dark 2.96303); iso-lines at a_m = 1.75, 2, 2.25, 2.5, 2.75. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0]. Marked: the worked example, i = 0.05, m = 12, a_m = 2.35523.

```
actuarial_science_ai999x.solve('act_mthly_annuity_udd', q=[0.1, 0.2, 0.5, 1.0], i=0.05, m=12)
# -> a_m = 2.35523, alpha = 1.0002, beta = 0.466508, a_due = 2.82119   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_mthly_annuity_udd', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1.0],
i=0.04, m=4)
# -> a_m = 4.92479, alpha = 1.00012, beta = 0.381189, a_due = 5.30534   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
m-thly annuity under UDD: q = [0.1, 0.2, 0.5, 1], i = 0.05, m = 12	SOLVED	a_m = 2.35523, alpha = 1.0002, beta = 0.466508, a_due = 2.82119
m-thly annuity under UDD: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, m = 4	SOLVED	a_m = 4.92479, alpha = 1.00012, beta = 0.381189, a_due = 5.30534
m-thly annuity under UDD: i = 0.05, m = 12	NEEDS_INPUT	act_mthly_annuity_udd needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_mthly_
m-thly annuity under UDD: q = [0.1, 0.2, 0.5, 1], i = 0.05, m = 12, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Woolhouse approximation for an m-thly annuity, `act_woolhouse_approximation`

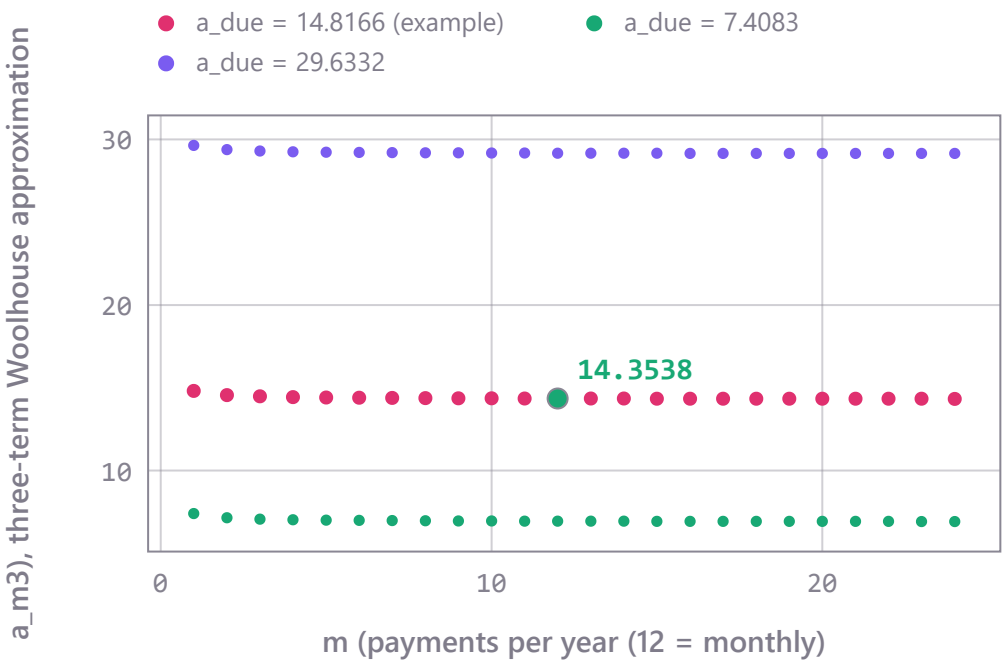
Woolhouse's two- and three-term approximations of the m-thly whole life annuity-due ä^(m)_x from the annual annuity-due ä_x, the interest rate and the force of mortality at x.

$$\ddot{a}_x^{(m)} \approx \ddot{a}_x - \frac{m-1}{2m} - \frac{m^2-1}{12m^2}(\delta + \mu_x)$$

input	meaning	condition
`a_due`	annual whole life annuity-due ä_x	
`m`	payments per year (12 = monthly)	
`i`	annual effective interest rate (5% or 0.05)	
`mu`	force of mortality at age x (per year)	

output	meaning
`a_m3`	three-term Woolhouse approximation
`a_m2`	two-term approximation ä_x - $(m-1)/(2m)$

- **Method:** ä^(m)_x ≈ ä_x - (m-1)/(2m) - (m^2-1)/(12 m^2) (delta + mu_x)
- **Assumes:** ä_x >= 1; $1 \leq m \leq 365$; $0 < i < 1$; $0 \leq mu < 5$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a_m3 (three-term Woolhouse approximation) against m (payments per year (12 = monthly)) from 1 to 24. One curve for each value of a_due (annual whole life annuity-due ä_x): 14.8166, 7.4083, 29.6332. Other inputs as in the worked example: i = 0.05, mu = 0.00473. 24 points per curve computed by actuarial-science-ai999x; marked: the worked example, m = 12, a_m3 = 14.3538.

```
actuarial_science_ai999x.solve('act_woolhouse_approximation', a_due=14.8166, m=12, i=0.05,
mu=0.00473)
# -> a_m3 = 14.3538, a_m2 = 14.3583   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_woolhouse_approximation', a_due=10.0, m=4, i=0.06, mu=0.02)
# -> a_m3 = 9.61889, a_m2 = 9.625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Woolhouse approximation: a_due = 14.8166, m = 12, i = 0.05, mu = 0.00473	SOLVED	a_m3 = 14.3538, a_m2 = 14.3583
Woolhouse approximation: a_due = 10, m = 4, i = 0.06, mu = 0.02	SOLVED	a_m3 = 9.61889, a_m2 = 9.625
Woolhouse approximation: m = 12, i = 0.05, mu = 0.00473	NEEDS_INPUT	act_woolhouse_approximation needs a_due (annual whole life annuity-due ä_x). Example: solve('act_woolhouse_approximation
Woolhouse approximation: a_due = 14.8166, m = 12, i = 0.05, mu = 0.00473, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Net annual premium for whole life insurance, `act_net_premium_whole_life`

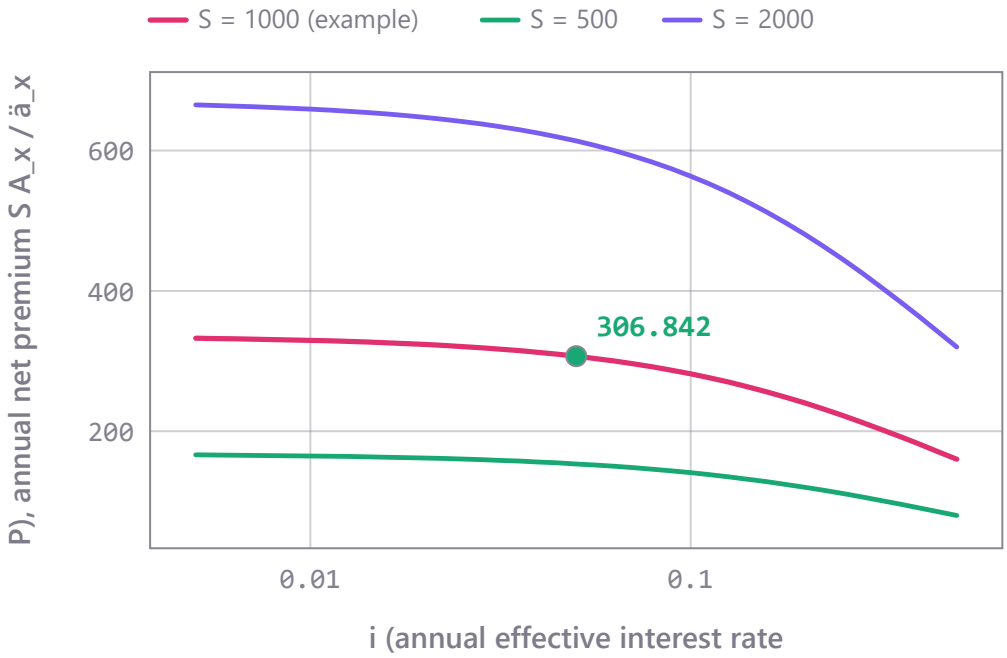
The level annual net premium, payable at the start of each year while alive, for a fully discrete whole life insurance of S, by the equivalence principle.

$$P = S\,\frac{A_x}{\ddot{a}_x} = S\left(\frac{1}{\ddot{a}_x} - d\right)$$

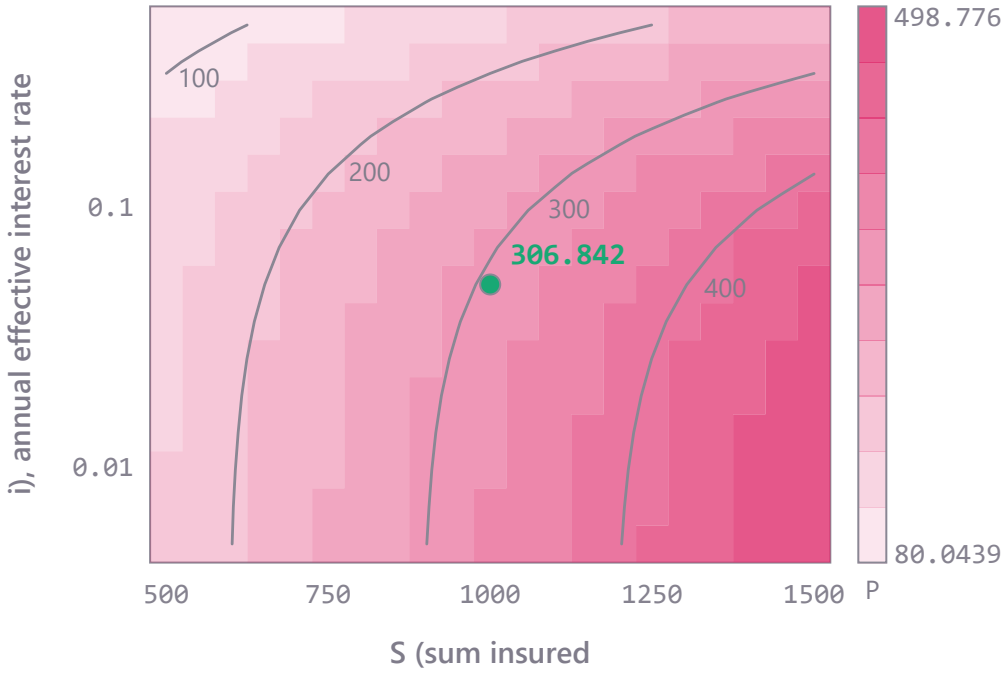
input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`S`	sum insured	

output	meaning
`P`	annual net premium S A_x / ä_x
`A`	whole life insurance A_x
`a_due`	whole life annuity-due ä_x

- **Method:** equivalence principle: P ä_x = S A_x
- **Assumes:** the table closes; $0 < i < 1$; $S > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (annual net premium S A_x / ä_x) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0]. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, P = 306.842.



P (annual net premium S A_x / ä_x) over S (sum insured) from 500 to 1500 and i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 (logarithmic vertical axis): each of the 21 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 80.0439 to dark 498.776); iso-lines at P = 100, 200, 300, 400. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0]. Marked: the worked example, S = 1000, i = 0.05, P = 306.842.

```
actuarial_science_ai999x.solve('act_net_premium_whole_life', q=[0.1, 0.2, 0.5, 1.0], i=0.05,
S=1000.0)
# -> P = 306.842, A = 0.865658, a_due = 2.82119   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_net_premium_whole_life', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3,
1.0], i=0.04, S=100000.0)
# -> P = 15002.8, A = 0.795949, a_due = 5.30534   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Net premium for whole life insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000	SOLVED	P = 306.842, A = 0.865658, a_due = 2.82119
Net premium for whole life insurance: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, S = 100000	SOLVED	P = 15002.8, A = 0.795949, a_due = 5.30534
Net premium for whole life insurance: i = 0.05, S = 1000	NEEDS_INPUT	act_net_premium_whole_life needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_n
Net premium for whole life insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Net annual premium for term insurance, `act_net_premium_term`

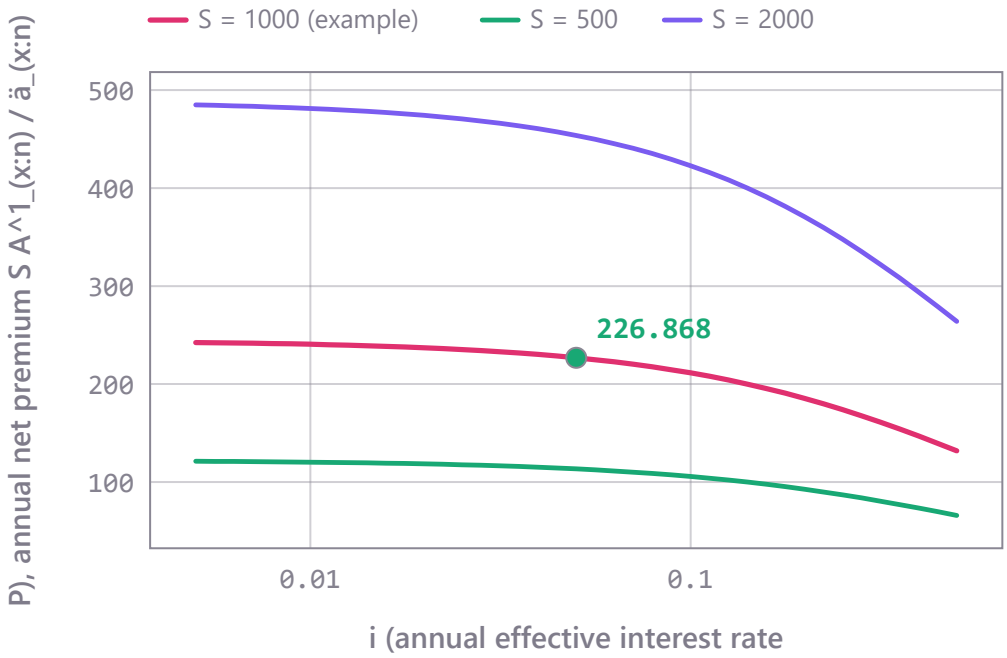
The level annual net premium, payable for n years while alive, for a fully discrete n-year term insurance of S, by the equivalence principle.

$$P = S\,\frac{A^{1}_{x:\overline{n}|}}{\ddot{a}_{x:\overline{n}|}}$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... for at least n years (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`n`	term in years	
`S`	sum insured	

output	meaning
`P`	annual net premium S Missing superscript or subscript argument(x:n) / ä_(x:n)
`A1`	term insurance Missing superscript or subscript argument(x:n)
`a_due`	temporary annuity-due ä_(x:n)

- **Method:** P ä_(x:n) = S Missing superscript or subscript argument(x:n)
- **Assumes:** at least n death probabilities; $0 < i < 1$; $S > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (annual net premium S A^1_(x:n) / ä_(x:n)) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], n = 3. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, P = 226.868.

```
actuarial_science_ai999x.solve('act_net_premium_term', q=[0.1, 0.2, 0.5, 1.0], i=0.05, n=3,
S=1000.0)
# -> P = 226.868, A1 = 0.569485, a_due = 2.5102   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_net_premium_term', q=[0.01, 0.012, 0.015, 0.018, 0.022],
i=0.06, n=5, S=50000.0)
# -> P = 706.15, A1 = 0.0616161, a_due = 4.36282   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Net premium for term insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000	SOLVED	P = 226.868, A1 = 0.569485, a_due = 2.5102
Net premium for term insurance: q = [0.01, 0.012, 0.015, 0.018, 0.022], i = 0.06, n = 5, S = 50000	SOLVED	P = 706.15, A1 = 0.0616161, a_due = 4.36282
Net premium for term insurance: i = 0.05, n = 3, S = 1000	NEEDS_INPUT	act_net_premium_term needs q (death probabilities q_x, q_(x+1), ... for at least n years (a list)). Example: solve('act_
Net premium for term insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Net annual premium for endowment insurance, `act_net_premium_endowment`

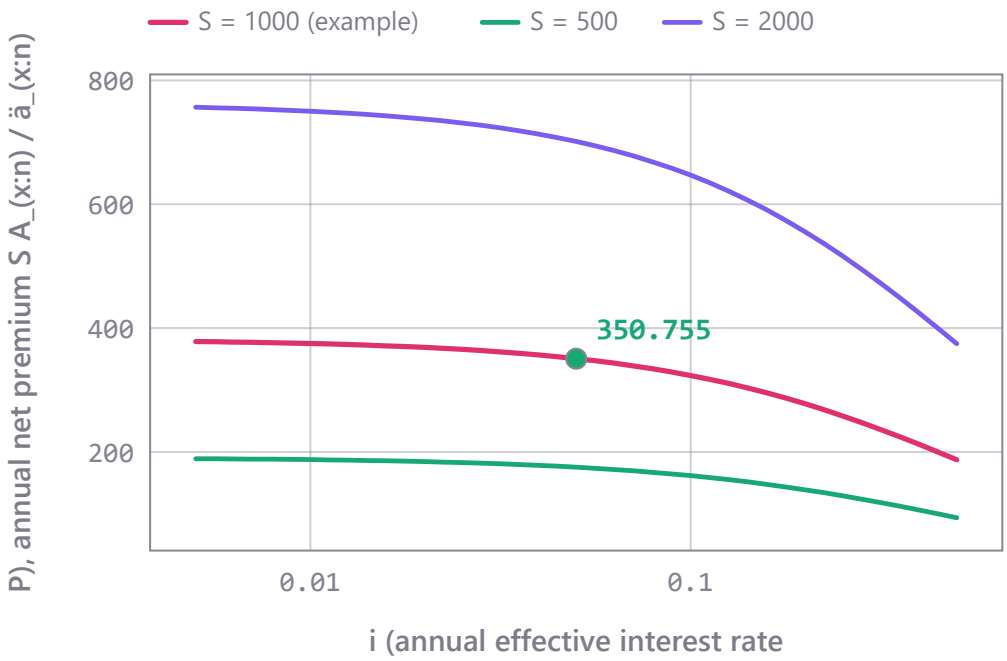
The level annual net premium, payable for n years while alive, for a fully discrete n-year endowment insurance of S, by the equivalence principle.

$$P = S\frac{A_{x:\overline{n}|}}{\ddot{a}_{x:\overline{n}|}} = S\left(\frac{1}{\ddot{a}_{x:\overline{n}|}} - d\right)$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... for at least n years (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`n`	term in years	
`S`	sum insured	

output	meaning
`P`	annual net premium S A_(x:n) / ä_(x:n)
`A`	endowment insurance A_(x:n)
`a_due`	temporary annuity-due ä_(x:n)

- **Method:** P ä_(x:n) = S A_(x:n); equivalently $P = S$ (1/ä_(x:n) - d)
- **Assumes:** at least n death probabilities; $0 < i < 1$; $S > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (annual net premium S A_(x:n) / ä_(x:n)) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], n = 3. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, P = 350.755.

```
actuarial_science_ai999x.solve('act_net_premium_endowment', q=[0.1, 0.2, 0.5, 1.0], i=0.05, n=3,
S=1000.0)
# -> P = 350.755, A = 0.880466, a_due = 2.5102   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_net_premium_endowment', q=[0.01, 0.012, 0.015, 0.018, 0.022],
i=0.06, n=5, S=50000.0)
# -> P = 8630.29, A = 0.753048, a_due = 4.36282   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Net premium for endowment insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000	SOLVED	P = 350.755, A = 0.880466, a_due = 2.5102
Net premium for endowment insurance: q = [0.01, 0.012, 0.015, 0.018, 0.022], i = 0.06, n = 5, S = 50000	SOLVED	P = 8630.29, A = 0.753048, a_due = 4.36282
Net premium for endowment insurance: i = 0.05, n = 3, S = 1000	NEEDS_INPUT	act_net_premium_endowment needs q (death probabilities q_x, q_(x+1), ... for at least n years (a list)). Example: solve(
Net premium for endowment insurance: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Net annual premium for limited-payment whole life insurance, `act_net_premium_limited_pay`

The level annual net premium for a fully discrete whole life insurance of S with premiums payable for at most h years.

$$ {}_hP_x = S\,\frac{A_x}{\ddot{a}_{x:\overline{h}|}} $$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`h`	premium paying period in years	
`S`	sum insured	

output	meaning
`P`	annual net premium S A_x / ä_(x:h)
`A`	whole life insurance A_x
`a_due_h`	h-year temporary annuity-due

- **Method:** P ä_(x:h) = S A_x
- **Assumes:** the table closes; 1 <= h <= length of the table; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (annual net premium S A_x / ä_(x:h)) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], h = 2. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, P = 466.123.

```
actuarial_science_ai999x.solve('act_net_premium_limited_pay', q=[0.1, 0.2, 0.5, 1.0], i=0.05, h=2, S=1000.0)
# -> P = 466.123, A = 0.865658, a_due_h = 1.85714   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_net_premium_limited_pay', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1.0], i=0.04, h=3, S=100000.0)
# -> P = 28213.2, A = 0.795949, a_due_h = 2.82119   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Limited payment whole life premium: q = [0.1, 0.2, 0.5, 1], i = 0.05, h = 2, S = 1000	SOLVED	P = 466.123, A = 0.865658, a_due_h = 1.85714
Limited payment whole life premium: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, h = 3, S = 100000	SOLVED	P = 28213.2, A = 0.795949, a_due_h = 2.82119
Limited payment whole life premium: i = 0.05, h = 2, S = 1000	NEEDS_INPUT	act_net_premium_limited_pay needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_
Limited payment whole life premium: q = [0.1, 0.2, 0.5, 1], i = 0.05, h = 2, S = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Gross annual premium for whole life insurance with expenses, `act_gross_premium`

The level annual gross premium for a fully discrete whole life insurance of S by the equivalence principle with expenses: an initial fraction e0 of the first premium, a renewal fraction r of later premiums, a per-policy

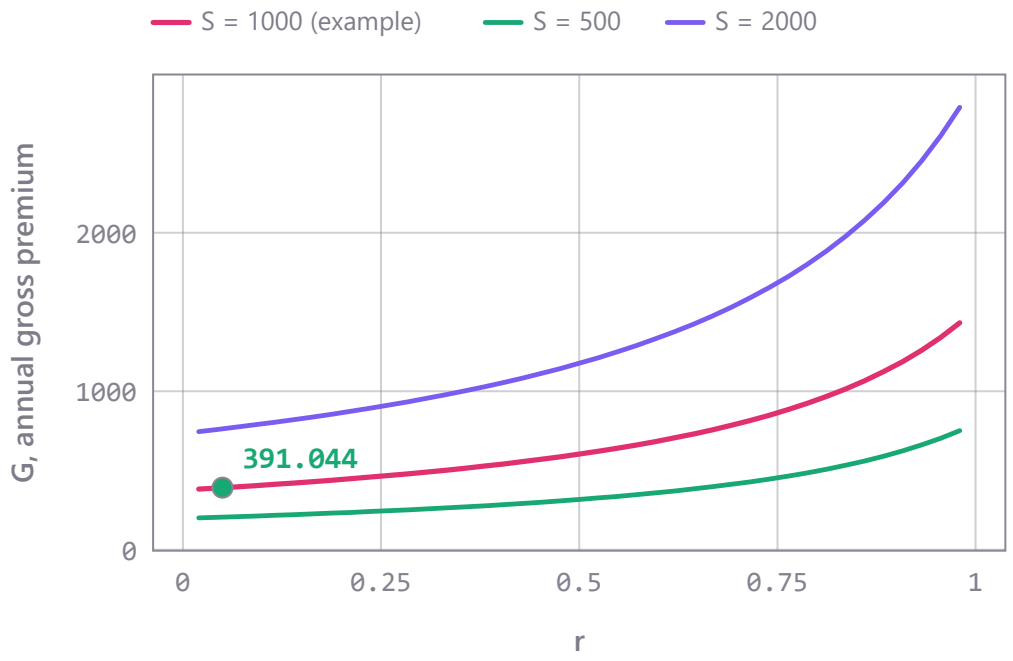
expense c every year (first year included) and a settlement expense E per claim; with the net premium and the loading.

$$G = \frac{(S+E)A_x + c\,\ddot{a}_x}{(1-r)\ddot{a}_x - (e_0 - r)}$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`S`	sum insured	
`e0`	initial expense as a fraction of the first gross premium (e.g. 40%)	
`r`	renewal expense as a fraction of each later gross premium (e.g. 5%)	
`c`	per-policy expense at the start of every year	
`E`	settlement expense per claim	

output	meaning
`G`	annual gross premium
`P`	annual net premium
`loading`	expense loading G - P

- **Method:** G ä_x = (S + E) $A_x + e_0$ G + r G (ä_x - 1) + c ä_x
- **Assumes:** the table closes; $0 < i < 1$; $0 \le e_0, r < 1$; c, $E \ge 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



G (annual gross premium) against r (renewal expense as a fraction of each later gross premium (e.g. 5%)) from 0.02 to 0.98. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], i = 0.05, e0 = 0.4, c = 10.0, E = 20.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, r = 0.05, G = 391.044.

```
actuarial_science_ai999x.solve('act_gross_premium', q=[0.1, 0.2, 0.5, 1.0], i=0.05, S=1000.0,
e0=0.4, r=0.05, c=10.0, E=20.0)
# -> G = 391.044, P = 306.842, loading = 84.2025   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_gross_premium', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1.0],
i=0.04, S=100000.0, e0=0.6, r=0.03, c=50.0, E=200.0)
# -> G = 17486.1, P = 15002.8, loading = 2483.27   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gross premium: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, e0 = 0.4, r = 0.05, c = 10, E = 20	SOLVED	G = 391.044, P = 306.842, loading = 84.2025
Gross premium: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, S = 100000, e0 = 0.6, r = 0.03, c = 50, E = 200	SOLVED	G = 17486.1, P = 15002.8, loading = 2483.27
Gross premium: i = 0.05, S = 1000, e0 = 0.4, r = 0.05, c = 10, E = 20	NEEDS_INPUT	act_gross_premium needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_gross_prem
Gross premium: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, e0 = 0.4, r = 0.05, c = 10, E = 20, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Gross premium: q = [0.1, 0.2, 0.5, 1], i = 5%, S = 1000, e0 = 40%, r = 5%, c = 10, E = 20	SOLVED	G = 391.044, P = 306.842, loading = 84.2025

Loss at issue of whole life insurance: mean, standard deviation, probability of loss, `act_loss_at_issue`

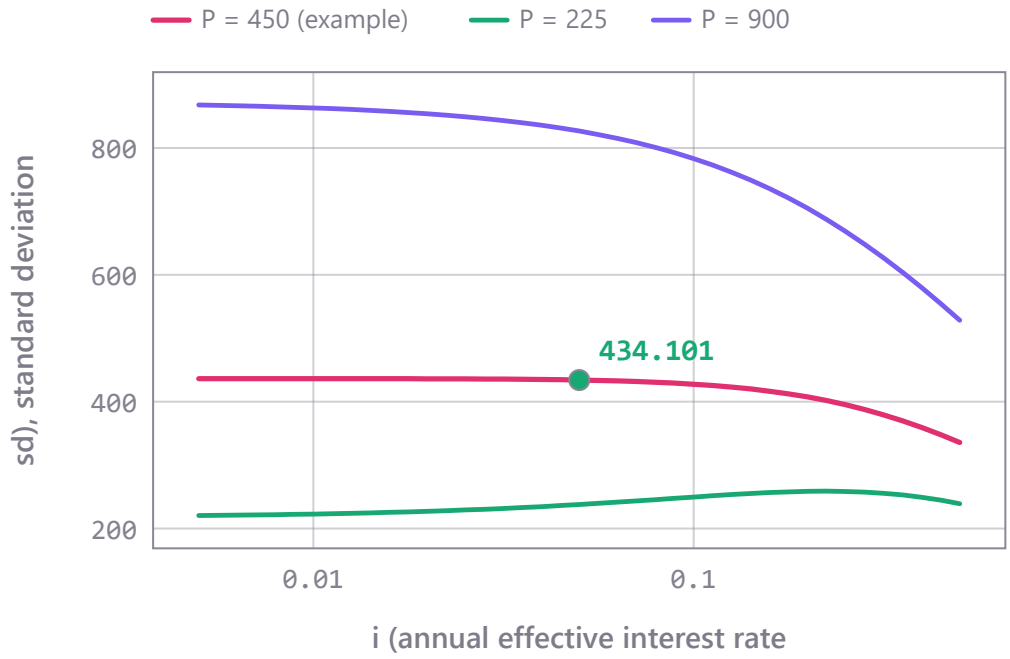
The mean and standard deviation of the insurer's loss at issue $L_0 = S\,v^{K+1} - P$ ä_(K+1) for a fully discrete whole life insurance of S with annual premium P, and the probability that the loss is positive.

$$L_0 = \left(S + \frac{P}{d}\right) v^{K+1} - \frac{P}{d}, \quad \mathrm{Var}(L_0) = \left(S + \frac{P}{d}\right)^2 \left({}^2A_x - A_x^2\right)$$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`S`	sum insured	
`P`	annual premium	

output	meaning
`mean`	expected loss at issue S A_x - P ä_x
`sd`	standard deviation (S + P/d) sqrt(2A_x - A_x^2)
`prob_loss`	probability that the loss at issue is positive

- **Method:** $L_0 = (S + P/d)\, v^{K+1} - P/d$; $E = S$ A - P ä; Var = (S + P/d)^2 $(2A - A^2)$
- **Assumes:** the table closes; $0 < i < 1$; S, $P > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sd (standard deviation (S + P/d) sqrt(2A_x - A_x^2)) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of P (annual premium): 450, 225, 900. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], S = 1000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, sd = 434.101.

```
actuarial_science_ai999x.solve('act_loss_at_issue', q=[0.1, 0.2, 0.5, 1.0], i=0.05, S=1000.0,
P=450.0)
# -> mean = -403.876, sd = 434.101, prob_loss = 0.28   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_loss_at_issue', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1.0],
i=0.04, S=100000.0, P=15000.0)
# -> mean = 14.7655, sd = 24149.8, prob_loss = 0.293799   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Loss at issue: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, P = 450	SOLVED	mean = -403.876, sd = 434.101, prob_loss = 0.28
Loss at issue: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, S = 100000, P = 15000	SOLVED	mean = 14.7655, sd = 24149.8, prob_loss = 0.293799
Loss at issue: i = 0.05, S = 1000, P = 450	NEEDS_INPUT	act_loss_at_issue needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_loss_at_is
Loss at issue: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, P = 450, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Net premium reserve for whole life insurance, `act_reserve_whole_life`

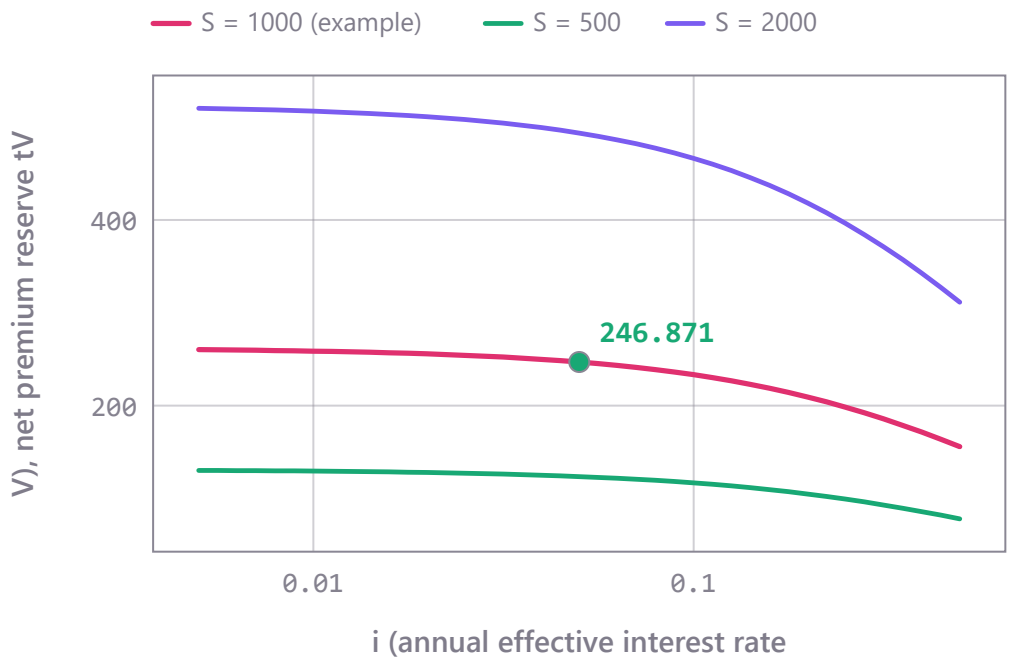
The net premium reserve (policy value) at integer duration t of a fully discrete whole life insurance of S with the equivalence-principle net premium, by the prospective method.

$$ {}_tV = S\,A_{x+t} - P\,\ddot{a}_{x+t} = S\left(1 - \frac{\ddot{a}_{x+t}}{\ddot{a}_x}\right) $$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`S`	sum insured	
`t`	policy duration in years	

output	meaning
`V`	net premium reserve tV
`P`	annual net premium
`A_t`	A_(x+t)
`a_due_t`	ä_(x+t)

- **Method:** $tV = S$ A_(x+t) - P ä_(x+t) = S (1 - ä_(x+t)/ä_x)
- **Assumes:** the table closes; 0 <= t < length of the table; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (net premium reserve tV) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], t = 1. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, V = 246.871.

```
actuarial_science_ai999x.solve('act_reserve_whole_life', q=[0.1, 0.2, 0.5, 1.0], i=0.05, S=1000.0,
t=1)
# -> V = 246.871, P = 306.842, A_t = 0.898823, a_due_t = 2.12472   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_reserve_whole_life', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3,
1.0], i=0.04, S=100000.0, t=3)
# -> V = 41676.6, P = 15002.8, A_t = 0.88099, a_due_t = 3.09425   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Whole life reserve: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, t = 1	SOLVED	V = 246.871, P = 306.842, A_t = 0.898823, a_due_t = 2.12472
Whole life reserve: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, S = 100000, t = 3	SOLVED	V = 41676.6, P = 15002.8, A_t = 0.88099, a_due_t = 3.09425
Whole life reserve: i = 0.05, S = 1000, t = 1	NEEDS_INPUT	act_reserve_whole_life needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_reser
Whole life reserve: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Net premium reserve for endowment insurance, `act_reserve_endowment`

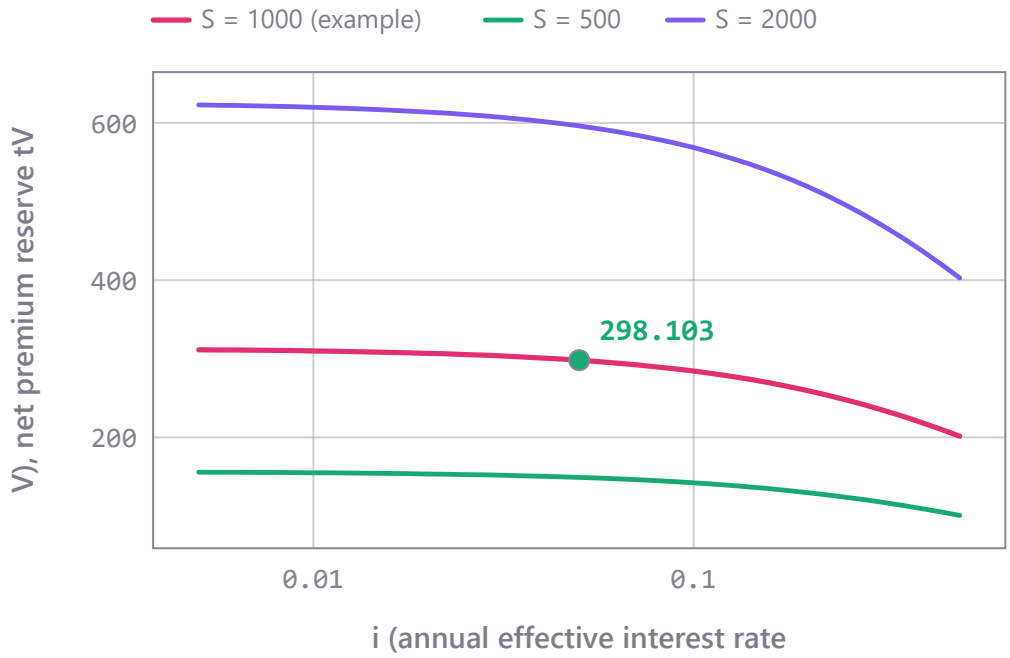
The net premium reserve at integer duration t of a fully discrete n-year endowment insurance of S with level net premiums payable for n years, by the prospective method (S at maturity).

$$ {}_tV = S\,A_{x+t:\overline{n-t}|} - P\,\ddot{a}_{x+t:\overline{n-t}|} $$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... for at least n years (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`n`	term in years	
`S`	sum insured	
`t`	policy duration in years	

output	meaning
`V`	net premium reserve tV
`P`	annual net premium

- **Method:** $tV = S$ A_(x+t:n-t) - P ä_(x+t:n-t)
- **Assumes:** at least n death probabilities; $0 \le t \le n$; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (net premium reserve tV) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], n = 3, t = 1. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, V = 298.103.

```
actuarial_science_ai999x.solve('act_reserve_endowment', q=[0.1, 0.2, 0.5, 1.0], i=0.05, n=3,
S=1000.0, t=1)
# -> V = 298.103, P = 350.755   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_reserve_endowment', q=[0.01, 0.012, 0.015, 0.018, 0.022],
i=0.06, n=5, S=50000.0, t=3)
# -> V = 27922.4, P = 8630.29   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Endowment reserve: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000, t = 1	SOLVED	V = 298.103, P = 350.755
Endowment reserve: q = [0.01, 0.012, 0.015, 0.018, 0.022], i = 0.06, n = 5, S = 50000, t = 3	SOLVED	V = 27922.4, P = 8630.29
Endowment reserve: i = 0.05, n = 3, S = 1000, t = 1	NEEDS_INPUT	act_reserve_endowment needs q (death probabilities q_x, q_(x+1), ... for at least n years (a list)). Example: solve('act
Endowment reserve: q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000, t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Net premium reserve for term insurance, `act_reserve_term`

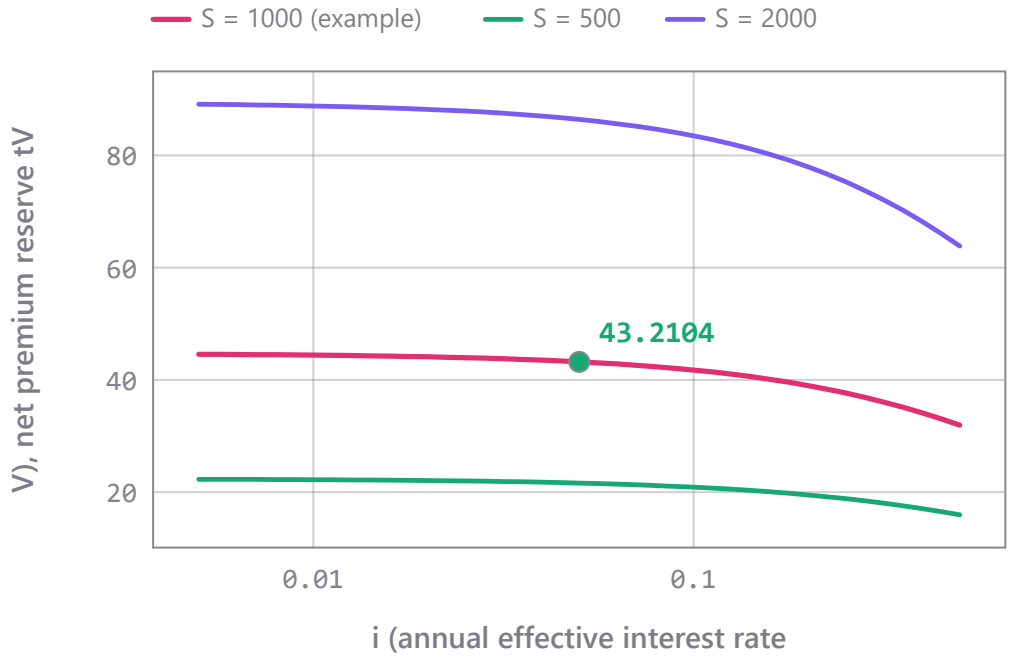
The net premium reserve at integer duration t of a fully discrete n-year term insurance of S with level net premiums payable for n years, by the prospective method.

$$ {}_tV = S\,A^{1}_{x+t:\overline{n-t}|} - P\,\ddot{a}_{x+t:\overline{n-t}|} $$

input	meaning	condition
q	death probabilities q_x, q_(x+1), ... for at least n years (a list)	
i	annual effective interest rate (5% or 0.05)	
n	term in years	
S	sum insured	
t	policy duration in years	

output	meaning
V	net premium reserve tV
P	annual net premium

- **Method:** $tV = S$ Missing superscript or subscript argument(x+t:n-t) - P ä_(x+t:n-t)
- **Assumes:** at least n death probabilities; $0 \le t \le n$; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (net premium reserve tV) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.01, 0.02, 0.04, 0.08], n = 4, t = 2. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, V = 43.2104.

```
actuarial_science_ai999x.solve('act_reserve_term', q=[0.01, 0.02, 0.04, 0.08], i=0.05, n=4,
S=1000.0, t=2)
# -> V = 43.2104, P = 33.7174   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_reserve_term', q=[0.01, 0.012, 0.015, 0.018, 0.022], i=0.06,
n=5, S=50000.0, t=3)
# -> V = 450.094, P = 706.15   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Term insurance reserve: q = [0.01, 0.02, 0.04, 0.08], i = 0.05, n = 4, S = 1000, t = 2	SOLVED	V = 43.2104, P = 33.7174
Term insurance reserve: q = [0.01, 0.012, 0.015, 0.018, 0.022], i = 0.06, n = 5, S = 50000, t = 3	SOLVED	V = 450.094, P = 706.15
Term insurance reserve: i = 0.05, n = 4, S = 1000, t = 2	NEEDS_INPUT	act_reserve_term needs q (death probabilities q_x, q_(x+1), ... for at least n years (a list)). Example: solve('act_rese
Term insurance reserve: q = [0.01, 0.02, 0.04, 0.08], i = 0.05, n = 4, S = 1000, t = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Retrospective reserve (accumulated fund per survivor), `act_retrospective_reserve`

The retrospective reserve at integer duration t of a fully discrete life insurance of S with annual premium P: the actuarial accumulated value of the premiums received less the cost of insurance, per survivor.

$$ {}_tV = \frac{P\,\ddot{a}_{x:\overline{t}|} - S\,A^{1}_{x:\overline{t}|}}{{}_tE_x} $$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... for at least t years (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`S`	sum insured	
`P`	annual premium	
`t`	policy duration in years	

output	meaning
`V`	retrospective reserve tV
`tEx`	pure endowment t_E_x

- **Method:** tV = (P ä_(x:t) - S Missing superscript or subscript argument (x:t)) / t_E_x
- **Assumes:** at least t death probabilities; Double subscripts: use braces to clarify; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (retrospective reserve tV) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of P (annual premium): 450, 225, 900. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], S = 1000.0, t = 2. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, V = 883.854.

```
actuarial_science_ai999x.solve('act_retrospective_reserve', q=[0.1, 0.2, 0.5, 1.0], i=0.05,
S=1000.0, P=450.0, t=2)
# -> V = 883.854, tEx = 0.653061   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_retrospective_reserve', q=[0.01, 0.012, 0.015, 0.018, 0.022],
i=0.06, S=50000.0, P=8000.0, t=3)
# -> V = 25737.2, tEx = 0.80893   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Retrospective reserve: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, P = 450, t = 2	SOLVED	V = 883.854, tEx = 0.653061
Retrospective reserve: q = [0.01, 0.012, 0.015, 0.018, 0.022], i = 0.06, S = 50000, P = 8000, t = 3	SOLVED	V = 25737.2, tEx = 0.80893
Retrospective reserve: i = 0.05, S = 1000, P = 450, t = 2	NEEDS_INPUT	act_retrospective_reserve needs q (death probabilities q_x, q_(x+1), ... for at least t years (a list)). Example: solve(
Retrospective reserve: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, P = 450, t = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Policy value recursion (one year), `act_policy_value_recursion`

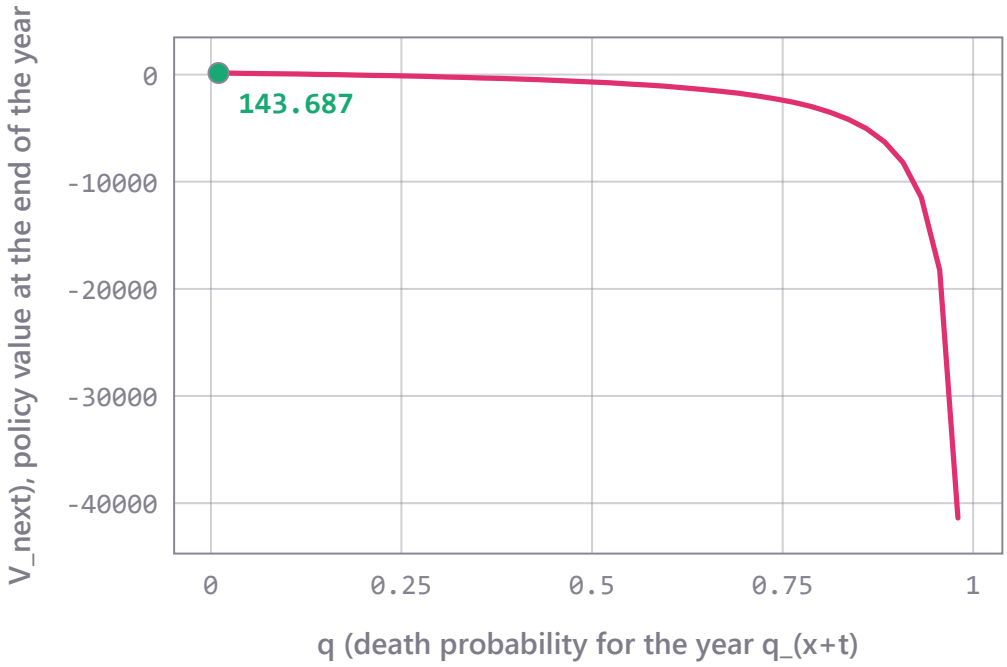
The policy value at the end of a policy year from the value at its start, the premium and expense at the start of the year, the interest rate, the death probability, the sum insured and the settlement expense: (tV + P - e) $(1+i) = q$ (S + E) + p (t+1)V.

$$({}_tV + P - e)(1+i) = q_{x+t}\,(S+E) + p_{x+t}\;{}_{t+1}V$$

input	meaning	condition
`V`	policy value at the start of the year tV	
`P`	premium at the start of the year	
`e`	expense at the start of the year	
`i`	annual effective interest rate (5% or 0.05)	
`q`	death probability for the year q_(x+t)	
`S`	death benefit	
`E`	settlement expense per claim	

output	meaning
`V_next`	policy value at the end of the year (t+1)V

- **Method:** (t+1)V = [(tV + P - e)$(1+i) - q$ (S + E)] / $(1-q)$
- **Assumes:** $0 \le q < 1$; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V_next (policy value at the end of the year (t+1)V) against q (death probability for the year q_(x+t)) from 0.02 to 0.98. Other inputs as in the worked example: V = 100.0, P = 50.0, e = 5.0, i = 0.05, S = 1000.0, E = 0.0. 41 points computed by actuarial-science-ai999x; marked: the worked example, q = 0.01, V_next = 143.687.

```
actuarial_science_ai999x.solve('act_policy_value_recursion', V=100.0, P=50.0, e=5.0, i=0.05,
q=0.01, S=1000.0, E=0.0)
# -> V_next = 143.687   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_policy_value_recursion', V=2500.0, P=400.0, e=20.0, i=0.04,
q=0.03, S=10000.0, E=100.0)
# -> V_next = 2775.46   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Policy value recursion: V = 100, P = 50, e = 5, i = 0.05, q = 0.01, S = 1000, E = 0	SOLVED	V_next = 143.687
Policy value recursion: V = 2500, P = 400, e = 20, i = 0.04, q = 0.03, S = 10000, E = 100	SOLVED	V_next = 2775.46
Policy value recursion: P = 50, e = 5, i = 0.05, q = 0.01, S = 1000, E = 0	NEEDS_INPUT	act_policy_value_recursion needs V (policy value at the start of the year tV). Example: solve('act_policy_value_recursio
Policy value recursion: V = 100, P = 50, e = 5, i = 0.05, q = 0.01, S = 1000, E = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Gross premium reserve for whole life insurance with expenses, `act_gross_premium_reserve`

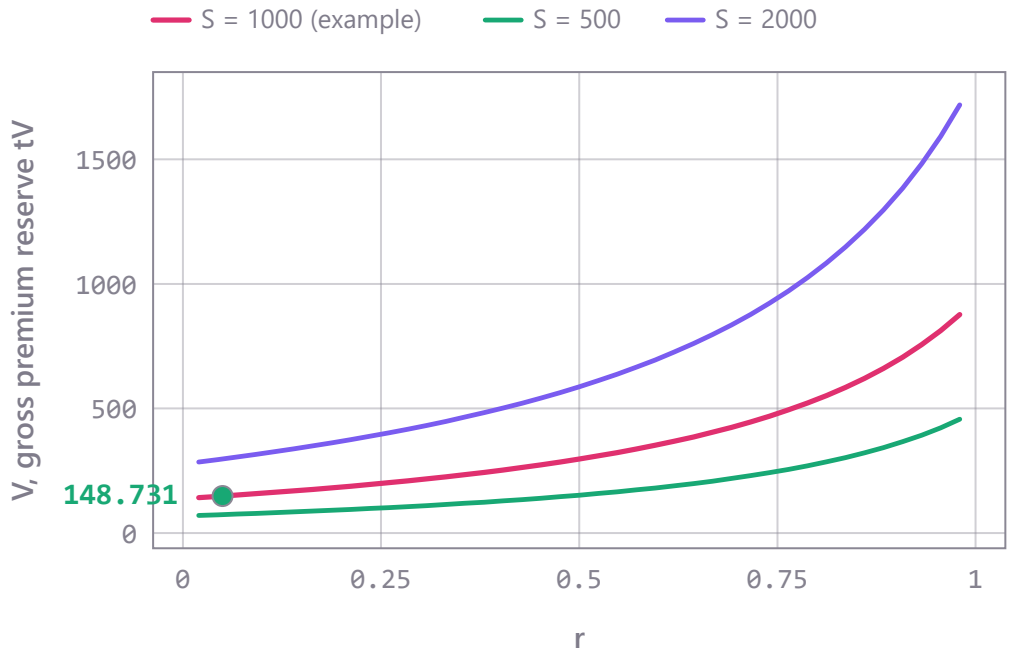
The gross premium policy value at integer duration t of a fully discrete whole life insurance of S, with the gross premium set by the equivalence principle with initial, renewal, per-policy and settlement expenses.

$$ {}_tV^g = (S+E)A_{x+t} + c\,\ddot{a}_{x+t} - (1-r)G\,\ddot{a}_{x+t} $$

input	meaning	condition
`q`	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`S`	sum insured	
`e0`	initial expense as a fraction of the first gross premium	
`r`	renewal expense as a fraction of each later gross premium	
`c`	per-policy expense at the start of every year	
`E`	settlement expense per claim	
`t`	policy duration in years	

output	meaning
`V`	gross premium reserve tV
`G`	annual gross premium

- **Method:** $tV = (S+E)$ A_(x+t) + c ä_(x+t) - G $(1-r)$ ä_(x+t) for $t \geq 1$ (0 at issue by the equivalence principle)
- **Assumes:** the table closes; 0 <= t < length of the table; $0 \leq e_0, r < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (gross premium reserve tV) against r (renewal expense as a fraction of each later gross premium) from 0.02 to 0.98. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: q = [0.1, 0.2, 0.5, 1.0], i = 0.05, e0 = 0.4, c = 10.0, E = 20.0, t = 1. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, r = 0.05, V = 148.731.

```
actuarial_science_ai999x.solve('act_gross_premium_reserve', q=[0.1, 0.2, 0.5, 1.0], i=0.05,
S=1000.0, e0=0.4, r=0.05, c=10.0, E=20.0, t=1)
# -> V = 148.731, G = 391.044   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_gross_premium_reserve', q=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3,
1.0], i=0.04, S=100000.0, e0=0.6, r=0.03, c=50.0, E=200.0, t=2)
# -> V = 20741.9, G = 17486.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gross premium reserve: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, e0 = 0.4, r = 0.05, c = 10, E = 20, t = 1	SOLVED	V = 148.731, G = 391.044
Gross premium reserve: q = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], i = 0.04, S = 100000, e0 = 0.6, r = 0.03, c = 50, E = 200, t = 2	SOLVED	V = 20741.9, G = 17486.1
Gross premium reserve: i = 0.05, S = 1000, e0 = 0.4, r = 0.05, c = 10, E = 20, t = 1	NEEDS_INPUT	act_gross_premium_reserve needs q (death probabilities q_x, q_(x+1), ... ending with 1 (a list)). Example: solve('act_gr
Gross premium reserve: q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, e0 = 0.4, r = 0.05, c = 10, E = 20, t = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Thiele's equation: continuous endowment insurance reserve (constant forces), `act_thiele_endowment`

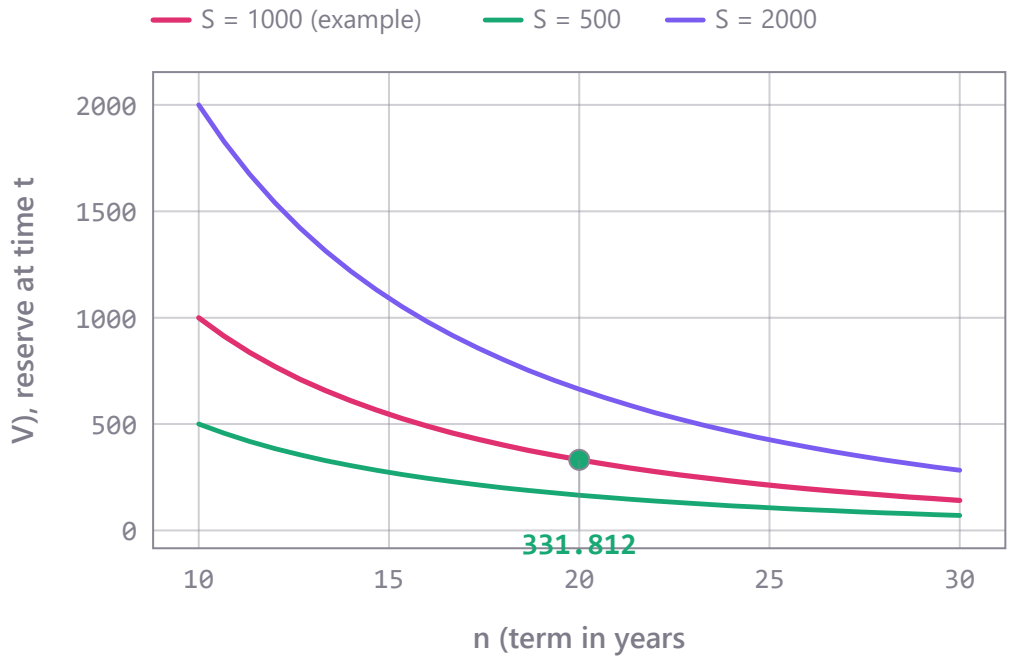
The continuous net premium rate and the reserve at time t of an n-year endowment insurance of S paid at the moment of death or at maturity, with a constant force of mortality mu and a constant force of interest delta (the solution of Thiele's equation $dV/dt = \delta$ V + P - mu (S - V), V(n) = S).

$$\frac{d\,{}_tV}{dt} = \delta\,{}_tV + \bar{P} - \mu\,(S - {}_tV), \quad {}_nV = S$$

input	meaning	condition
`mu`	constant force of mortality per year	
`delta`	constant force of interest per year	
`n`	term in years	
`S`	sum insured	
`t`	time since issue in years	

output	meaning
`P`	continuous annual net premium rate
`A_bar`	endowment insurance Ā_(x:n)
`a_bar`	continuous temporary annuity ā_(x:n)
`V`	reserve at time t

- **Method:** Ā = mu/(mu+delta)(1 - e^(-(mu+delta)n)) + e^(-(mu+delta)n); ā = (1 - e^(-(mu+delta)n))/(mu+delta); $P = S$ Ā/ā; ${}_tV = S$ Ā_(n-t) - P ā_(n-t)
- **Assumes:** $0 < mu$, $\delta < 1$; $0 < n \leq 100$; $0 \leq t \leq n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (reserve at time t) against n (term in years) from 10 to 30. One curve for each value of S (sum insured): 1000, 500, 2000. Other inputs as in the worked example: mu = 0.02, delta = 0.05, t = 10.0. 31 points per curve computed by actuarial-science-ai999x; marked: the worked example, n = 20, V = 331.812.

```
actuarial_science_ai999x.solve('act_thiele_endowment', mu=0.02, delta=0.05, n=20.0, S=1000.0,
t=10.0)
# -> P = 42.9118, A_bar = 0.461855, a_bar = 10.7629, V = 331.812   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_thiele_endowment', mu=0.05, delta=0.04, n=10.0, S=50000.0,
t=3.0)
# -> P = 5583.03, A_bar = 0.736253, a_bar = 6.59367, V = 10618.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thiele's equation for endowment insurance: mu = 0.02, delta = 0.05, n = 20, S = 1000, t = 10	SOLVED	P = 42.9118, A_bar = 0.461855, a_bar = 10.7629, V = 331.812
Thiele's equation for endowment insurance: mu = 0.05, delta = 0.04, n = 10, S = 50000, t = 3	SOLVED	P = 5583.03, A_bar = 0.736253, a_bar = 6.59367, V = 10618.1
Thiele's equation for endowment insurance: delta = 0.05, n = 20, S = 1000, t = 10	NEEDS_INPUT	act_thiele_endowment needs mu (constant force of mortality per year). Example: solve('act_thiele_endowment', mu=0.02, de
Thiele's equation for endowment insurance: mu = 0.02, delta = 0.05, n = 20, S = 1000, t = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Continuous whole life insurance, premium and reserve under Makeham's law, `act_makeham_continuous_whole_life`

The continuous whole life annuity, the insurance paid at the moment of death, the continuous net premium rate and the reserve at time t for a life aged x following Makeham's law, at a constant interest rate.

$$\bar{a}_x = \int_0^\infty e^{-\delta s}\, {}_sp_x\, ds, \quad \bar{A}_x = 1 - \delta \bar{a}_x, \quad {}_t\bar{V} = S\left(1 - \frac{\bar{a}_{x+t}}{\bar{a}_x}\right)$$

input	meaning	condition
`A`	Makeham's A (per year)	
`B`	Makeham's B (per year)	
`c`	Makeham's c (> 1)	
`x`	age at issue (years)	
`i`	annual effective interest rate (5% or 0.05)	
`S`	sum insured	
`t`	time since issue in years	

output	meaning
`a_bar`	continuous whole life annuity ā_x
`A_bar`	insurance payable at death Ā_x
`P`	continuous annual net premium rate
`V`	reserve at time t

- **Method:** ā_x = integral of e^(-delta s) s_p_x (Gauss-Legendre); Ā_x = $1 - \delta$ ā_x; $P = S$ Ā_x/ā_x; ${}_tV = S$ (1 - ā_(x+t)/ā_x)
- **Assumes:** Makeham parameters $0 \le A < 0.1$, $0 < B < 0.1$, $1 < c < 1.5$; $0 \le x \le 120$; $0 \le t \le 100$; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (reserve at time t) against t (time since issue in years) from 1 to 100 on a logarithmic axis. One curve for each value of S (sum insured): 100000, 50000, 200000. Other inputs as in the worked example: A = 0.00022, B = 2.7e-06, c = 1.124, x = 50.0, i = 0.05. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, t = 10, V = 12836.5.

```
actuarial_science_ai999x.solve('act_makeham_continuous_whole_life', A=0.00022, B=2.7e-06, c=1.124,
x=50.0, i=0.05, S=100000.0, t=10.0)
# -> a_bar = 16.5204, A_bar = 0.193968, P = 1174.12, V = 12836.5   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_makeham_continuous_whole_life', A=0.0007, B=5e-05, c=1.09,
x=40.0, i=0.04, S=1000.0, t=5.0)
# -> a_bar = 19.3095, A_bar = 0.242667, P = 12.5672, V = 55.1886   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Continuous whole life under Makeham: A = 0.00022, B = 2.7e-06, c = 1.124, x = 50, i = 0.05, S = 100000, t = 10	SOLVED	a_bar = 16.5204, A_bar = 0.193968, P = 1174.12, V = 12836.5
Continuous whole life under Makeham: A = 0.0007, B = 5e-05, c = 1.09, x = 40, i = 0.04, S = 1000, t = 5	SOLVED	a_bar = 19.3095, A_bar = 0.242667, P = 12.5672, V = 55.1886
Continuous whole life under Makeham: B = 2.7e-06, c = 1.124, x = 50, i = 0.05, S = 100000, t = 10	NEEDS_INPUT	act_makeham_continuous_whole_life needs A (Makeham's A (per year)). Example: solve('act_makeham_continuous_whole_life',
Continuous whole life under Makeham: A = 0.00022, B = 2.7e-06, c = 1.124, x = 50, i = 0.05, S = 100000, t = 10, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Joint life and last survivor annuities and insurances, `act_joint_life_last_survivor`

The joint life and last survivor annuities-due and whole life insurances (end of year of the first or second death) for two independent lives aged x and y with their own life tables.

$$\ddot{a}_{xy} = \sum_{k\geq 0} v^k \, {}_kp_x \, {}_kp_y, \quad \ddot{a}_{\overline{xy}} = \ddot{a}_x + \ddot{a}_y - \ddot{a}_{xy}, \quad A = 1 - d\,\ddot{a}$$

input	meaning	condition
`qx`	death probabilities of the first life from age x, ending with 1 (a list)	
`qy`	death probabilities of the second life from age y, ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	

output	meaning
`a_joint`	joint life annuity-due ä_xy
`a_last`	last survivor annuity-due
`A_joint`	joint life insurance A_xy
`A_last`	last survivor insurance

- **Method:** ä_xy = sum v^k k_p_x k_p_y; last survivor ä = ä_x + ä_y - ä_xy; $A = 1 - d$ ä

- **Assumes:** both tables close; independent lives; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a_joint (joint life annuity-due ä_xy) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: qx = [0.1, 0.2, 0.5, 1.0], qy = [0.05, 0.1, 0.3, 0.6, 1.0]. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, a_joint = 2.55878.

```
actuarial_science_ai999x.solve('act_joint_life_last_survivor', qx=[0.1, 0.2, 0.5, 1.0], qy=[0.05, 0.1, 0.3, 0.6, 1.0], i=0.05)
# -> a_joint = 2.55878, a_last = 3.65664, A_joint = 0.878154, A_last = 0.825874   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_joint_life_last_survivor', qx=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1.0], qy=[0.01, 0.02, 0.04, 0.07, 0.12, 0.25, 0.5, 1.0], i=0.04)
# -> a_joint = 4.81428, a_last = 6.20396, A_joint = 0.814835, A_last = 0.761386   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Joint life and last survivor: qx = [0.1, 0.2, 0.5, 1], qy = [0.05, 0.1, 0.3, 0.6, 1], i = 0.05	SOLVED	a_joint = 2.55878, a_last = 3.65664, A_joint = 0.878154, A_last = 0.825874
Joint life and last survivor: qx = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], qy = [0.01, 0.02, 0.04, 0.07, 0.12, 0.25, 0.5, 1], i = 0.04	SOLVED	a_joint = 4.81428, a_last = 6.20396, A_joint = 0.814835, A_last = 0.761386
Joint life and last survivor: qy = [0.05, 0.1, 0.3, 0.6, 1], i = 0.05	NEEDS_INPUT	act_joint_life_last_survivor needs qx (death probabilities of the first life from age x, ending with 1 (a list)). Exampl
Joint life and last survivor: qx = [0.1, 0.2, 0.5, 1], qy = [0.05, 0.1, 0.3, 0.6, 1], i = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Reversionary annuity, `act_reversionary_annuity`

The actuarial present value of an annuity-immediate of P a year paid to a life aged y after the death of a life aged x (a_(x|y) = a_y - a_xy), independent lives.

$$a_{x|y} = a_y - a_{xy} = \sum_{k\geq 1} v^k \, {}_kp_y \, (1 - {}_kp_x)$$

input	meaning	condition
`qx`	death probabilities of the life whose death starts the annuity, ending with 1 (a list)	
`qy`	death probabilities of the annuitant from age y, ending with 1 (a list)	
`i`	annual effective interest rate (5% or 0.05)	
`P`	annual payment	

output	meaning
`a_rev`	reversionary annuity a_(x|y) per unit
`EPV`	actuarial present value P a_(x|y)

- **Method:** a_(x|y) = a_y - a_xy = sum_(k>=1) v^k k_p_y Double subscripts: use braces to clarify
- **Assumes:** both tables close; independent lives; $0 < i < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



a_rev (reversionary annuity a_(x|y) per unit) against i (annual effective interest rate (5% or 0.05)) from 0.005 to 0.5 on a logarithmic axis. Other inputs as in the worked example: qx = [0.1, 0.2, 0.5, 1.0], qy = [0.05, 0.1, 0.3, 0.6, 1.0], P = 1000.0. 41 points computed by actuarial-science-ai999x; marked: the worked example, i = 0.05, a_rev = 0.835458.

```
actuarial_science_ai999x.solve('act_reversionary_annuity', qx=[0.1, 0.2, 0.5, 1.0], qy=[0.05, 0.1,
0.3, 0.6, 1.0], i=0.05, P=1000.0)
# -> a_rev = 0.835458, EPV = 835.458   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_reversionary_annuity', qx=[0.02, 0.03, 0.05, 0.08, 0.15, 0.3,
1.0], qy=[0.01, 0.02, 0.04, 0.07, 0.12, 0.25, 0.5, 1.0], i=0.04, P=5000.0)
# -> a_rev = 0.898623, EPV = 4493.11   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reversionary annuity: qx = [0.1, 0.2, 0.5, 1], qy = [0.05, 0.1, 0.3, 0.6, 1], i = 0.05, P = 1000	SOLVED	a_rev = 0.835458, EPV = 835.458
Reversionary annuity: qx = [0.02, 0.03, 0.05, 0.08, 0.15, 0.3, 1], qy = [0.01, 0.02, 0.04, 0.07, 0.12, 0.25, 0.5, 1], i = 0.04, P = 5000	SOLVED	a_rev = 0.898623, EPV = 4493.11
Reversionary annuity: qy = [0.05, 0.1, 0.3, 0.6, 1], i = 0.05, P = 1000	NEEDS_INPUT	act_reversionary_annuity needs qx (death probabilities of the life whose death starts the annuity, ending with 1 (a list
Reversionary annuity: qx = [0.1, 0.2, 0.5, 1], qy = [0.05, 0.1, 0.3, 0.6, 1], i = 0.05, P = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Associated single decrement rates from a multiple decrement table (UDD), `act_multiple_decrement_to_single`

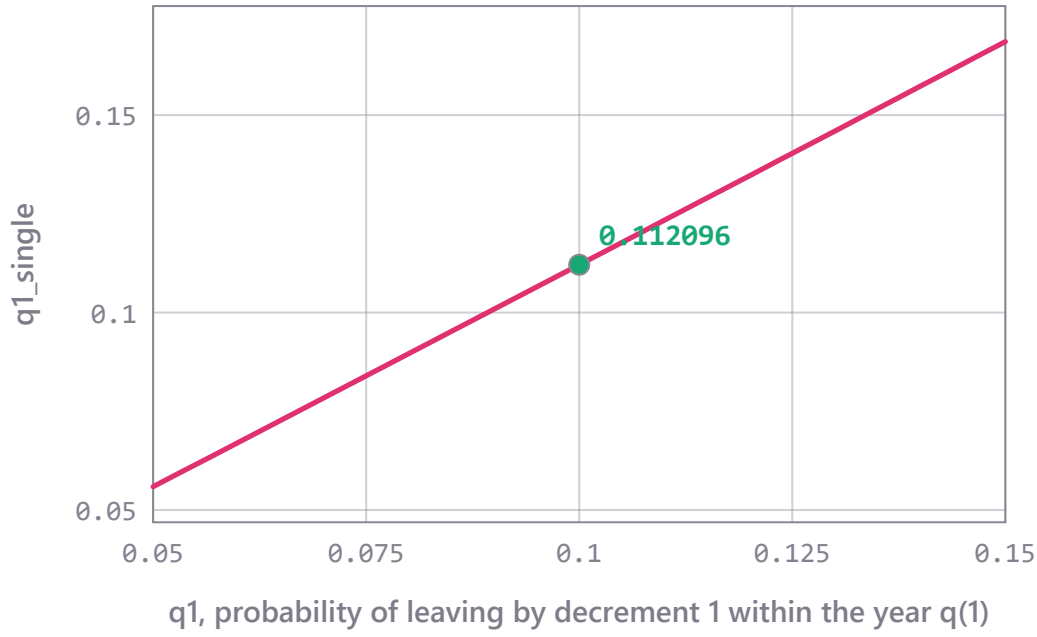
The associated single decrement probabilities q'(1) and q'(2) from the two decrement probabilities of a double decrement table, assuming each decrement is uniformly distributed within the year in the multiple decrement table.

$$q_x'^{(j)} = 1 - \left(p_x^{(\tau)}\right)^{q_x^{(j)}/q_x^{(\tau)}}$$

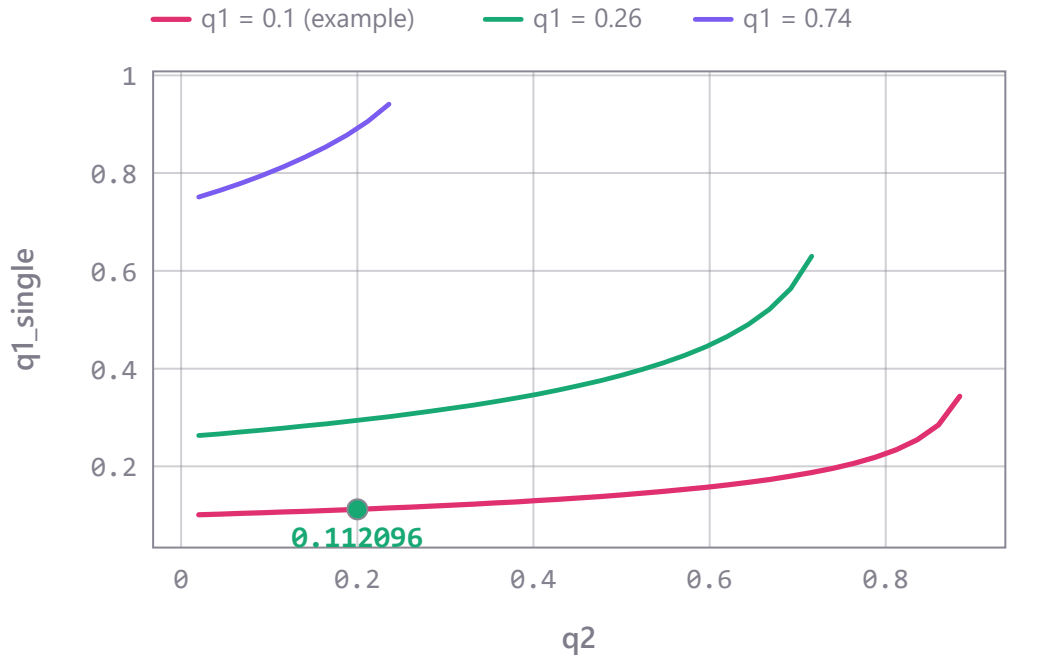
input	meaning	condition
`q1`	probability of leaving by decrement 1 within the year q(1)	
`q2`	probability of leaving by decrement 2 within the year q(2)	

output	meaning
`q1_single`	associated single decrement probability q'(1)
`q2_single`	associated single decrement probability q'(2)
`q_total`	total decrement probability q(tau)

- **Method:** q'(j) = 1 - (p(tau))^(q(j)/q(tau))
- **Assumes:** q1, $q_2 > 0$; $q_1 + q_2 < 1$; UDD in the multiple decrement table
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



q1_single (associated single decrement probability q'(1)) against q1 (probability of leaving by decrement 1 within the year q(1)), with q2 = 0.2. 31 points computed by actuarial-science-ai999x; marked: q1 = 0.1, q1_single = 0.112096.



q1_single (associated single decrement probability q'(1)) against q2 (probability of leaving by decrement 2 within the year q(2)) from 0.02 to 0.98. One curve for each value of q1 (probability of leaving by decrement 1 within the year q(1)): 0.1, 0.26, 0.74. 37 points per curve computed by actuarial-science-ai999x; marked: the worked example, q2 = 0.2, q1_single = 0.112096.

```
actuarial_science_ai999x.solve('act_multiple_decrement_to_single', q1=0.1, q2=0.2)
# -> q1_single = 0.112096, q2_single = 0.211626, q_total = 0.3   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_multiple_decrement_to_single', q1=0.02, q2=0.15)
# -> q1_single = 0.0216826, q2_single = 0.151605, q_total = 0.17   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Associated single decrement rates: q1 = 0.1, q2 = 0.2	SOLVED	q1_single = 0.112096, q2_single = 0.211626, q_total = 0.3
Associated single decrement rates: q1 = 0.02, q2 = 0.15	SOLVED	q1_single = 0.0216826, q2_single = 0.151605, q_total = 0.17
Associated single decrement rates: q2 = 0.2	NEEDS_INPUT	act_multiple_decrement_to_single needs q1 (probability of leaving by decrement 1 within the year q(1)). Example: solve('
Associated single decrement rates: q1 = 0.1, q2 = 0.2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Multiple decrement probabilities from single decrement rates (UDD),

`act_single_to_multiple_decrement`

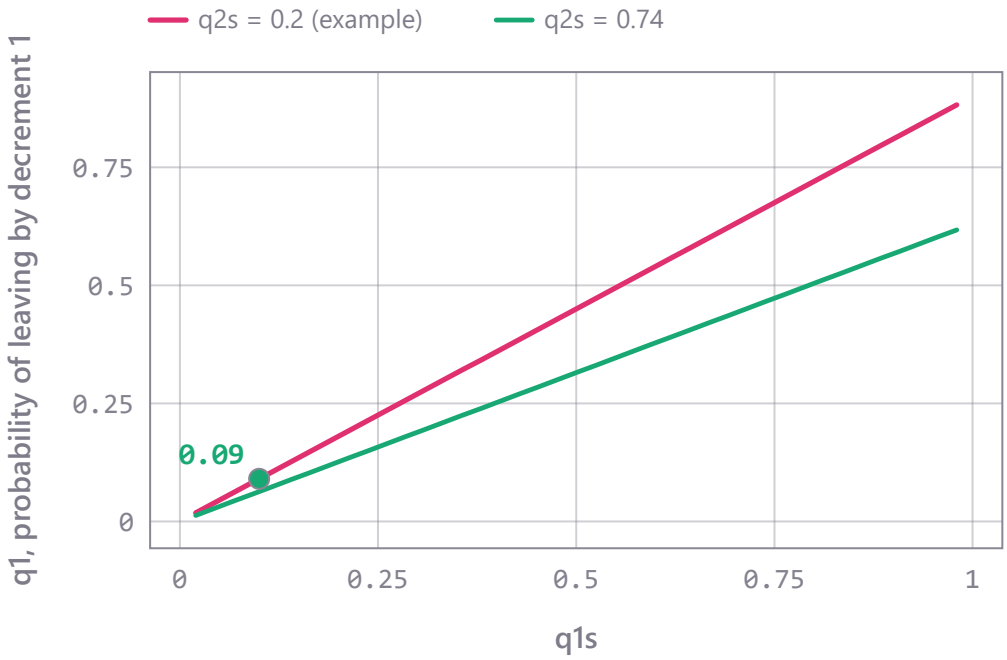
The double decrement probabilities q(1) and q(2) from the associated single decrement probabilities q'(1) and q'(2), each uniformly distributed within the year in its single decrement table.

$$q_x^{(1)} = q_x'^{(1)}\left(1 - \tfrac{1}{2}q_x'^{(2)}\right), \quad q_x^{(2)} = q_x'^{(2)}\left(1 - \tfrac{1}{2}q_x'^{(1)}\right)$$

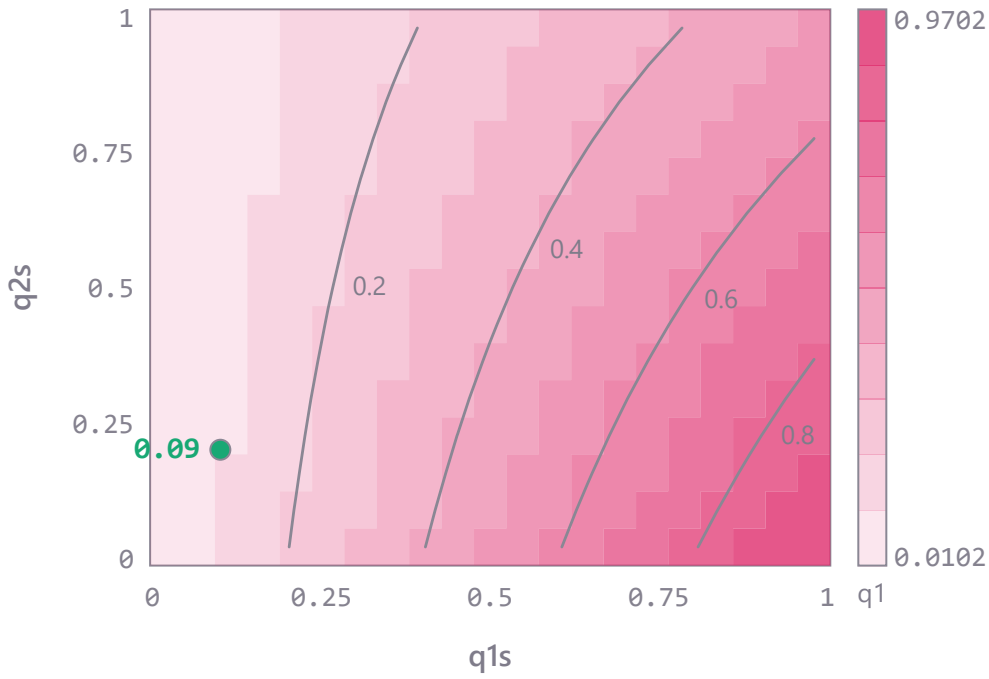
input	meaning	condition
`q1s`	associated single decrement probability q'(1)	
`q2s`	associated single decrement probability q'(2)	

output	meaning
`q1`	probability of leaving by decrement 1, q(1)
`q2`	probability of leaving by decrement 2, q(2)
`q_total`	total decrement probability q(tau)

- **Method:** q(1) = q'(1) (1 - q'(2)/2), q(2) = q'(2) (1 - q'(1)/2)
- **Assumes:** $0 < q$'(1), q'$(2) < 1$; UDD in each single decrement table
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



q1 (probability of leaving by decrement 1, q(1)) against q1s (associated single decrement probability q'(1)) from 0.02 to 0.98. One curve for each value of q2s (associated single decrement probability q'(2)): 0.2, 0.74. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, q1s = 0.1, q1 = 0.09.



q1 (probability of leaving by decrement 1, q(1)) over q1s (associated single decrement probability q'(1)) from 0.02 to 0.98 and q2s (associated single decrement probability q'(2)) from 0.02 to 0.98: each of the 21 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 0.0102 to dark 0.9702); iso-lines at q1 = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, q1s = 0.1, q2s = 0.2, q1 = 0.09.

```
actuarial_science_ai999x.solve('act_single_to_multiple_decrement', q1s=0.1, q2s=0.2)
# -> q1 = 0.09, q2 = 0.19, q_total = 0.28   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_single_to_multiple_decrement', q1s=0.05, q2s=0.3)
# -> q1 = 0.0425, q2 = 0.2925, q_total = 0.335   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiple decrement probabilities: q1s = 0.1, q2s = 0.2	SOLVED	q1 = 0.09, q2 = 0.19, q_total = 0.28
Multiple decrement probabilities: q1s = 0.05, q2s = 0.3	SOLVED	q1 = 0.0425, q2 = 0.2925, q_total = 0.335
Multiple decrement probabilities: q2s = 0.2	NEEDS_INPUT	act_single_to_multiple_decrement needs q1s (associated single decrement probability q'(1)). Example: solve('act_single_t
Multiple decrement probabilities: q1s = 0.1, q2s = 0.2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Poisson claim frequency probabilities, `act_poisson_frequency`

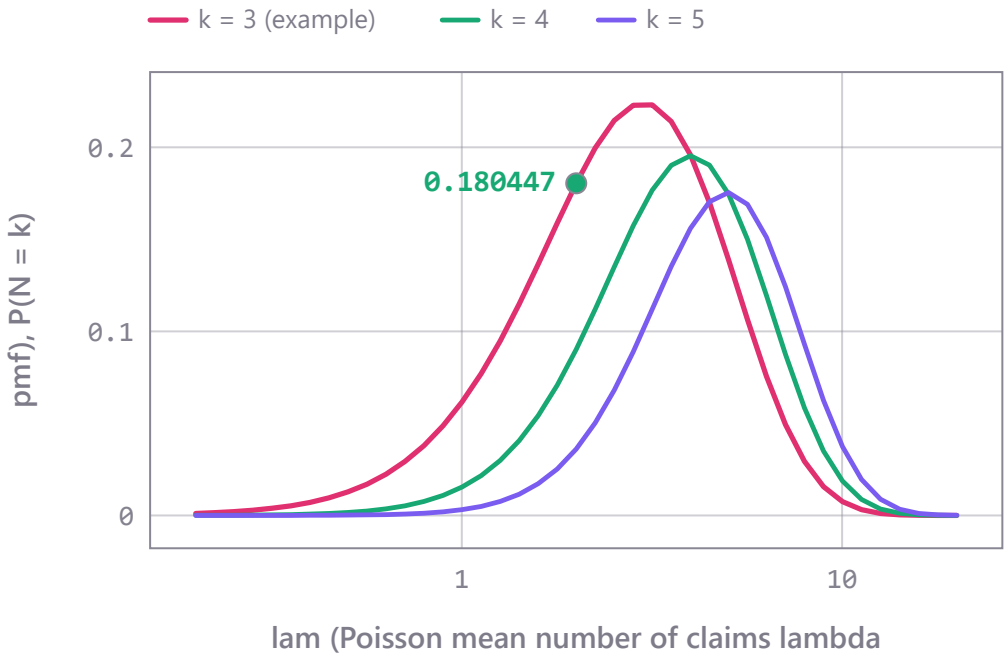
The probability of exactly k claims and of at most k claims when the number of claims is Poisson with mean lambda.

$$P(N = k) = \frac{e^{-\lambda}\lambda^{k}}{k!}$$

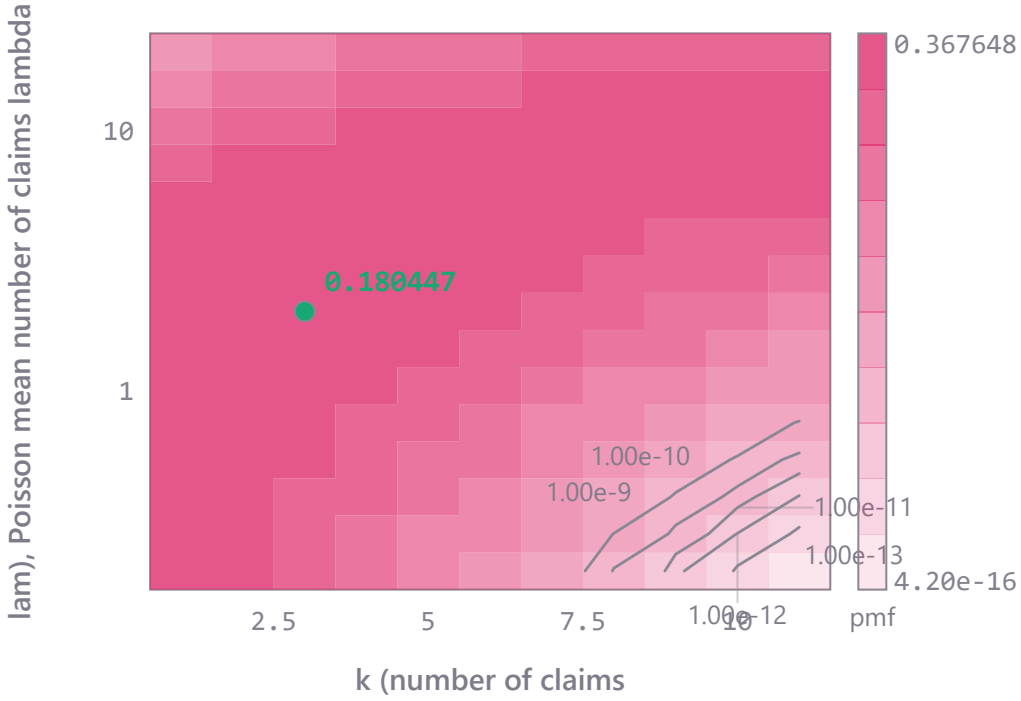
input	meaning	condition
`lam`	Poisson mean number of claims lambda	
`k`	number of claims	

output	meaning
`pmf`	P(N = k)
`cdf`	P(N <= k)
`mean`	mean number of claims
`var`	variance of the number of claims

- **Method:** P(N = k) = e^(-lambda) $\lambda^{k}/k!$
- **Assumes:** $0 < \lambda \leq 500$; $k \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pmf (P(N = k)) against lam (Poisson mean number of claims lambda) from 0.2 to 20 on a logarithmic axis. One curve for each value of k (number of claims): 3, 4, 5. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, lam = 2, pmf = 0.180447.



pmf (P(N = k)) over k (number of claims) from 1 to 11 and lam (Poisson mean number of claims lambda) from 0.2 to 20 (logarithmic vertical axis): each of the 11 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 4.2006e-16 to dark 0.367648, logarithmic shading); iso-lines at pmf = 1.00e-13, 1.00e-12, 1.00e-11, 1.00e-10, 1.00e-9. Marked: the worked example, k = 3, lam = 2, pmf = 0.180447.

```
actuarial_science_ai999x.solve('act_poisson_frequency', lam=2.0, k=3)
# -> pmf = 0.180447, cdf = 0.857123, mean = 2, var = 2   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_poisson_frequency', lam=0.5, k=0)
# -> pmf = 0.606531, cdf = 0.606531, mean = 0.5, var = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Poisson claim frequency: lam = 2, k = 3	SOLVED	pmf = 0.180447, cdf = 0.857123, mean = 2, var = 2
Poisson claim frequency: lam = 0.5, k = 0	SOLVED	pmf = 0.606531, cdf = 0.606531, mean = 0.5, var = 0.5
Poisson claim frequency: k = 3	NEEDS_INPUT	act_poisson_frequency needs lam (Poisson mean number of claims lambda). Example: solve('act_poisson_frequency', lam=2, k
Poisson claim frequency: lam = 2, k = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Poisson claim count: lambda = 2, k = 3	SOLVED	pmf = 0.180447, cdf = 0.857123, mean = 2, var = 2

Negative binomial claim frequency probabilities, `act_negative_binomial_frequency`

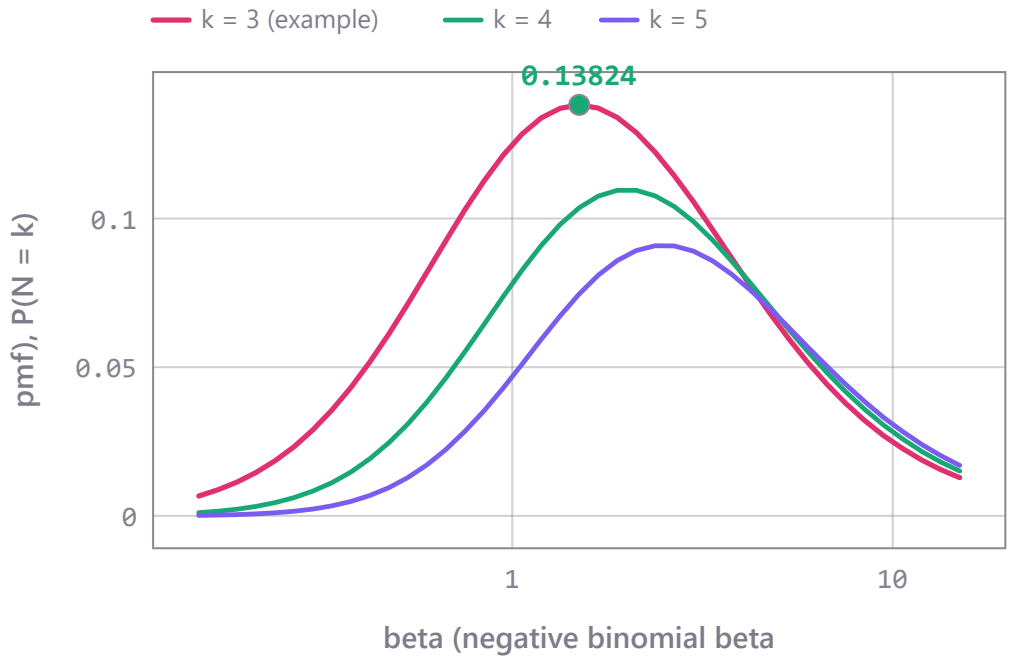
The probability of exactly k claims and of at most k claims, with the mean and variance, for a negative binomial claim count with parameters r and beta (mean r beta).

$$P(N = k) = \binom{r + k - 1}{k} \left(\frac{1}{1+\beta}\right)^{r} \left(\frac{\beta}{1+\beta}\right)^{k}$$

input	meaning	condition
`r`	negative binomial r	
`beta`	negative binomial beta	
`k`	number of claims	

output	meaning
`pmf`	P(N = k)
`cdf`	P(N <= k)
`mean`	mean r beta
`var`	variance r beta $(1+\beta)$

- **Method:** P(N = k) = Gamma(r+k)/(Gamma(r) k!) (1/(1+beta))^r (beta/(1+beta))^k
- **Assumes:** r, $\beta > 0$; $k \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pmf (P(N = k)) against beta (negative binomial beta) from 0.15 to 15 on a logarithmic axis. One curve for each value of k (number of claims): 3, 4, 5. Other inputs as in the worked example: r = 2.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, beta = 1.5, pmf = 0.13824.

```
actuarial_science_ai999x.solve('act_negative_binomial_frequency', r=2.0, beta=1.5, k=3)
# -> pmf = 0.13824, cdf = 0.66304, mean = 3, var = 7.5   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_negative_binomial_frequency', r=0.5, beta=4.0, k=0)
# -> pmf = 0.447214, cdf = 0.447214, mean = 2, var = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Negative binomial claim frequency: r = 2, beta = 1.5, k = 3	SOLVED	pmf = 0.13824, cdf = 0.66304, mean = 3, var = 7.5
Negative binomial claim frequency: r = 0.5, beta = 4, k = 0	SOLVED	pmf = 0.447214, cdf = 0.447214, mean = 2, var = 10
Negative binomial claim frequency: beta = 1.5, k = 3	NEEDS_INPUT	act_negative_binomial_frequency needs r (negative binomial r). Example: solve('act_negative_binomial_frequency', r=2, be
Negative binomial claim frequency: r = 2, beta = 1.5, k = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Mean and variance of aggregate losses (compound distribution),

`act_compound_distribution_moments`

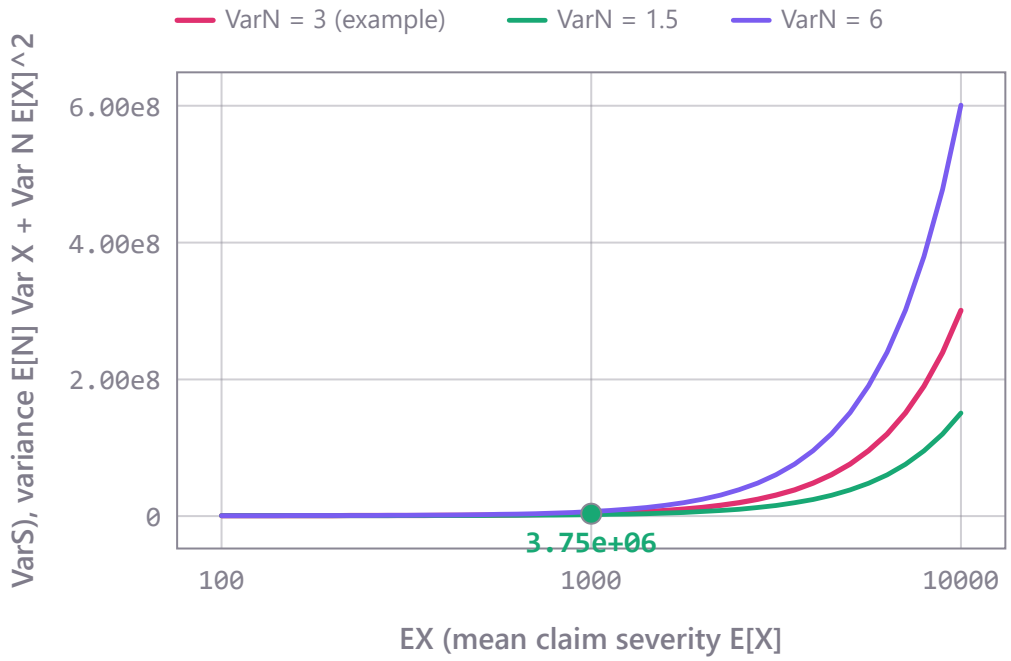
The mean and variance of aggregate losses $S = X_1 + \ldots + X_N$ from the mean and variance of the claim count N and of the claim severity X (independent, identically distributed severities).

$$E[S] = E[N]\,E[X], \quad \operatorname{Var}(S) = E[N]\operatorname{Var}(X) + \operatorname{Var}(N)\,E[X]^2$$

input	meaning	condition
`EN`	mean number of claims E[N]	
`VarN`	variance of the number of claims	
`EX`	mean claim severity E[X]	
`VarX`	variance of the claim severity	

output	meaning
`ES`	mean aggregate loss E[N] E[X]
`VarS`	variance E[N] Var X + Var N E[X]^2

- **Method:** E[S] = E[N] E[X]; Var $S = E$[N] Var X + Var N (E[X])^2
- **Assumes:** moments non-negative; N and the X's independent
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



VarS (variance E[N] Var X + Var N E[X]^2) against EX (mean claim severity E[X]) from 100 to 10000 on a logarithmic axis. One curve for each value of VarN (variance of the number of claims): 3, 1.5, 6. Other inputs as in the worked example: EN = 3.0, VarX = 250000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, EX = 1000, VarS = 3.75e+06.

```
actuarial_science_ai999x.solve('act_compound_distribution_moments', EN=3.0, VarN=3.0, EX=1000.0,
VarX=250000.0)
# -> ES = 3000, VarS = 3.75e+06   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_compound_distribution_moments', EN=10.0, VarN=25.0, EX=500.0,
VarX=40000.0)
# -> ES = 5000, VarS = 6.65e+06   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Aggregate loss mean and variance: EN = 3, VarN = 3, EX = 1000, VarX = 250000	SOLVED	ES = 3000, VarS = 3.75e+06
Aggregate loss mean and variance: EN = 10, VarN = 25, EX = 500, VarX = 40000	SOLVED	ES = 5000, VarS = 6.65e+06
Aggregate loss mean and variance: VarN = 3, EX = 1000, VarX = 250000	NEEDS_INPUT	act_compound_distribution_moments needs EN (mean number of claims E[N]). Example: solve('act_compound_distribution_momen
Aggregate loss mean and variance: EN = 3, VarN = 3, EX = 1000, VarX = 250000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Compound Poisson mean, variance and skewness, `act_compound_poisson_moments`

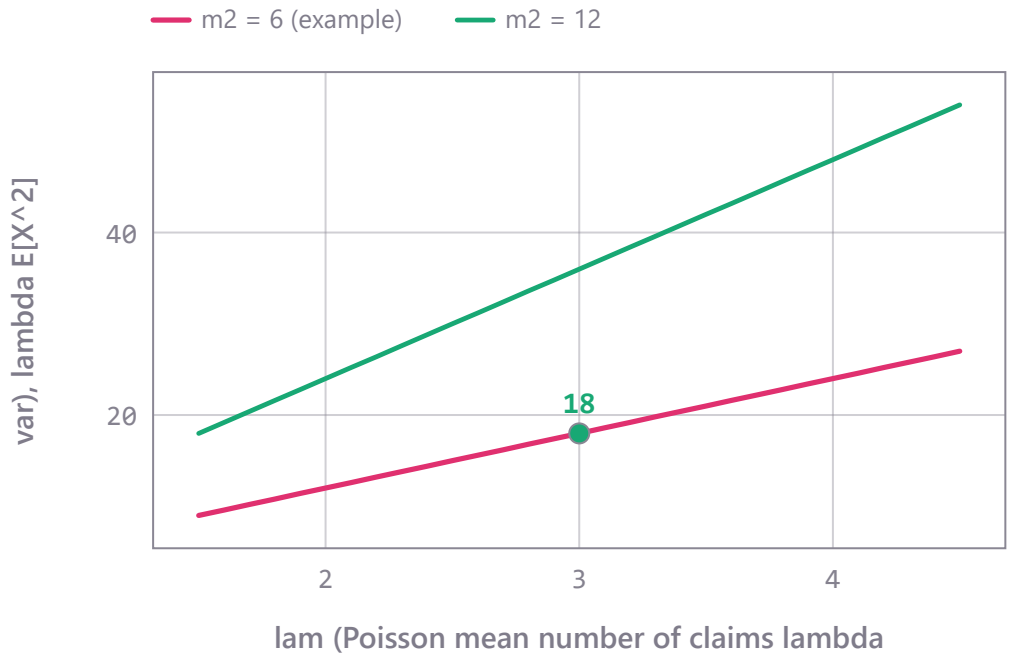
The mean, variance and coefficient of skewness of aggregate losses when the claim count is Poisson(lambda), from the first three raw moments of the claim severity.

$$\kappa_n(S) = \lambda\, E[X^n], \quad \gamma_S = \frac{\lambda E[X^3]}{(\lambda E[X^2])^{3/2}}$$

input	meaning	condition
`lam`	Poisson mean number of claims lambda	
`m1`	E[X]	
`m2`	E[X^2]	
`m3`	E[X^3]	

output	meaning
`mean`	lambda E[X]
`var`	lambda E[X^2]
`skew`	lambda E[X^3] / (lambda E[X^2])^$(3/2)$

- **Method:** cumulants of a compound Poisson sum: $kappa_n = \lambda$ E[X^n]
- **Assumes:** $\lambda > 0$; moments positive; E[X^2] >= E[X]^2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



var (lambda E[X^2]) against lam (Poisson mean number of claims lambda) from 1.5 to 4.5. One curve for each value of m2 (E[X^2]): 6, 12. Other inputs as in the worked example: m1 = 2.0, m3 = 24.0. 31 points per curve computed by actuarial-science-ai999x; marked: the worked example, lam = 3, var = 18.

```
actuarial_science_ai999x.solve('act_compound_poisson_moments', lam=3.0, m1=2.0, m2=6.0, m3=24.0)
# -> mean = 6, var = 18, skew = 0.942809   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_compound_poisson_moments', lam=10.0, m1=100.0, m2=20000.0,
m3=6000000.0)
# -> mean = 1000, var = 200000, skew = 0.67082   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compound Poisson moments: lam = 3, m1 = 2, m2 = 6, m3 = 24	SOLVED	mean = 6, var = 18, skew = 0.942809
Compound Poisson moments: lam = 10, m1 = 100, m2 = 20000, m3 = 6000000	SOLVED	mean = 1000, var = 200000, skew = 0.67082
Compound Poisson moments: m1 = 2, m2 = 6, m3 = 24	NEEDS_INPUT	act_compound_poisson_moments needs lam (Poisson mean number of claims lambda). Example: solve('act_compound_poisson_mome
Compound Poisson moments: lam = 3, m1 = 2, m2 = 6, m3 = 24, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Individual risk model: aggregate mean and variance, `act_individual_risk_model`

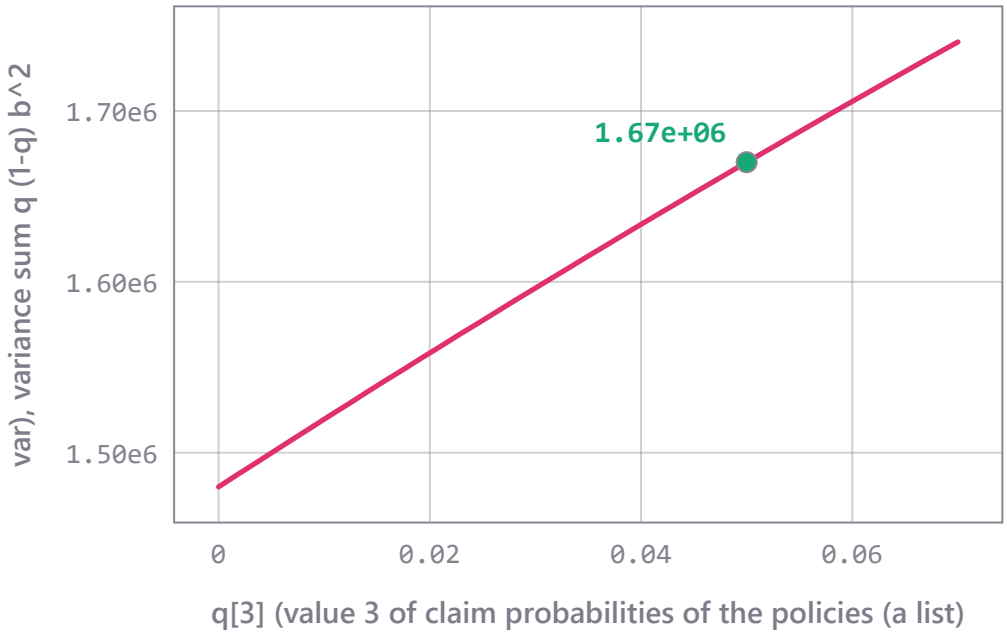
The mean and variance of total claims, and the probability of no claim, for a portfolio of independent policies each paying a fixed benefit b_i with probability q_i.

$$E[S] = \sum_i q_i b_i, \quad \mathrm{Var}(S) = \sum_i q_i (1 - q_i)\, b_i^2$$

input	meaning	condition
`q`	claim probabilities of the policies (a list)	
`b`	benefit amounts of the policies (a list)	

output	meaning
`mean`	expected total claims sum q b
`var`	variance sum q $(1 - q)\, b^2$
`prob_no_claim`	probability of no claim at all

- **Method:** E[S] = sum q_i b_i; Var S = sum q_i $(1 - q_i)\, b_i^2$; P(S = 0) = prod $(1 - q_i)$
- **Assumes:** one benefit per probability; $0 \le q \le 1$; $b > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



var (variance sum q (1-q) b^2) as value 3 of q (claim probabilities of the policies (a list)) moves from 0 to 0.07, the other values as given ([0.01, 0.02, 0.05]); 41 runs of actuarial-science-ai999x. Other inputs: b = [10000.0, 5000.0, 2000.0]. Marked: the worked example, q[3] = 0.05, var = 1.67e+06.

```
actuarial_science_ai999x.solve('act_individual_risk_model', q=[0.01, 0.02, 0.05], b=[10000.0,
5000.0, 2000.0])
# -> mean = 300, var = 1.67e+06, prob_no_claim = 0.92169   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_individual_risk_model', q=[0.1, 0.1, 0.2, 0.05], b=[1000.0,
1000.0, 3000.0, 5000.0])
# -> mean = 1050, var = 2.8075e+06, prob_no_claim = 0.6156   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Individual risk model: q = [0.01, 0.02, 0.05], b = [10000, 5000, 2000]	SOLVED	mean = 300, var = 1.67e+06, prob_no_claim = 0.92169
Individual risk model: q = [0.1, 0.1, 0.2, 0.05], b = [1000, 1000, 3000, 5000]	SOLVED	mean = 1050, var = 2.8075e+06, prob_no_claim = 0.6156
Individual risk model: b = [10000, 5000, 2000]	NEEDS_INPUT	act_individual_risk_model needs q (claim probabilities of the policies (a list)). Example: solve('act_individual_risk_mo
Individual risk model: q = [0.01, 0.02, 0.05], b = [10000, 5000, 2000], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Normal approximation to aggregate losses, `act_normal_approximation_aggregate`

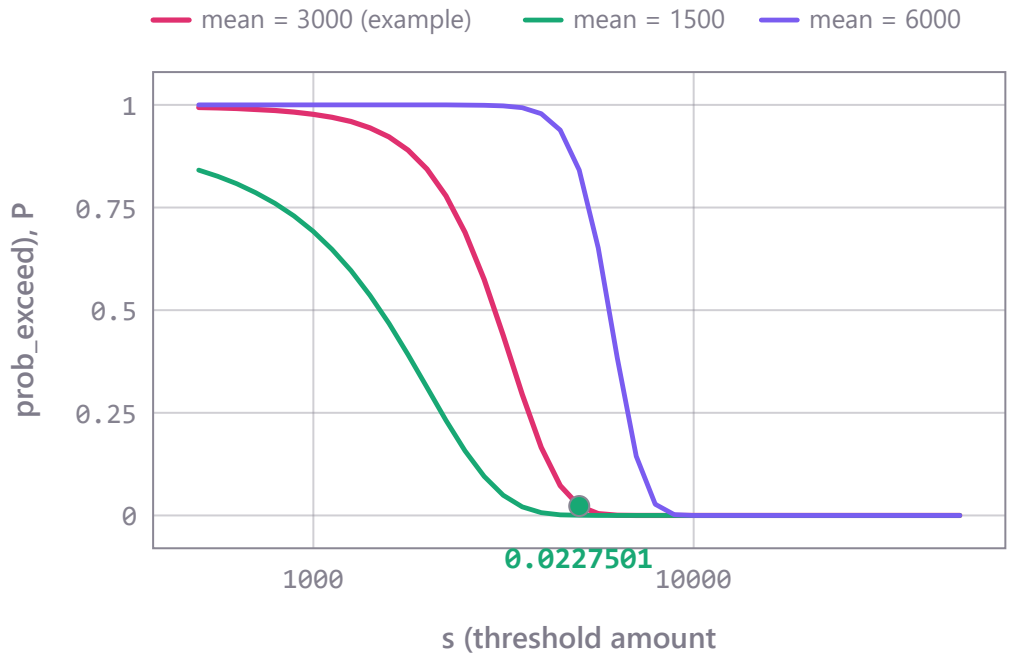
The probability that aggregate losses exceed an amount s under the normal approximation with the given mean and variance.

$$P(S > s) \approx 1 - \Phi\left(\frac{s - E[S]}{\sqrt{\mathrm{Var}(S)}}\right)$$

input	meaning	condition
`mean`	mean of aggregate losses	
`var`	variance of aggregate losses	
`s`	threshold amount	

output	meaning
`prob_exceed`	P(S > s) by the normal approximation
`z`	standardized threshold (s - mean)/sd

- **Method:** P(S > s) = 1 - Phi((s - E[S])/sd(S))
- **Assumes:** variance > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



prob_exceed (P(S > s) by the normal approximation) against s (threshold amount) from 500 to 50000 on a logarithmic axis. One curve for each value of mean (mean of aggregate losses): 3000, 1500, 6000. Other inputs as in the worked example: var = 1000000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, s = 5000, prob_exceed = 0.0227501.

```
actuarial_science_ai999x.solve('act_normal_approximation_aggregate', mean=3000.0, var=1000000.0,
s=5000.0)
# -> prob_exceed = 0.0227501, z = 2   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_normal_approximation_aggregate', mean=100000.0,
var=400000000.0, s=120000.0)
# -> prob_exceed = 0.158655, z = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal approximation to aggregate losses: mean = 3000, var = 1000000, s = 5000	SOLVED	prob_exceed = 0.0227501, z = 2
Normal approximation to aggregate losses: mean = 100000, var = 400000000, s = 120000	SOLVED	prob_exceed = 0.158655, z = 1
Normal approximation to aggregate losses: var = 1000000, s = 5000	NEEDS_INPUT	act_normal_approximation_aggregate needs mean (mean of aggregate losses). Example: solve('act_normal_approximation_aggre
Normal approximation to aggregate losses: mean = 3000, var = 1000000, s = 5000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Panjer recursion for a compound Poisson distribution, `act_panjer_poisson`

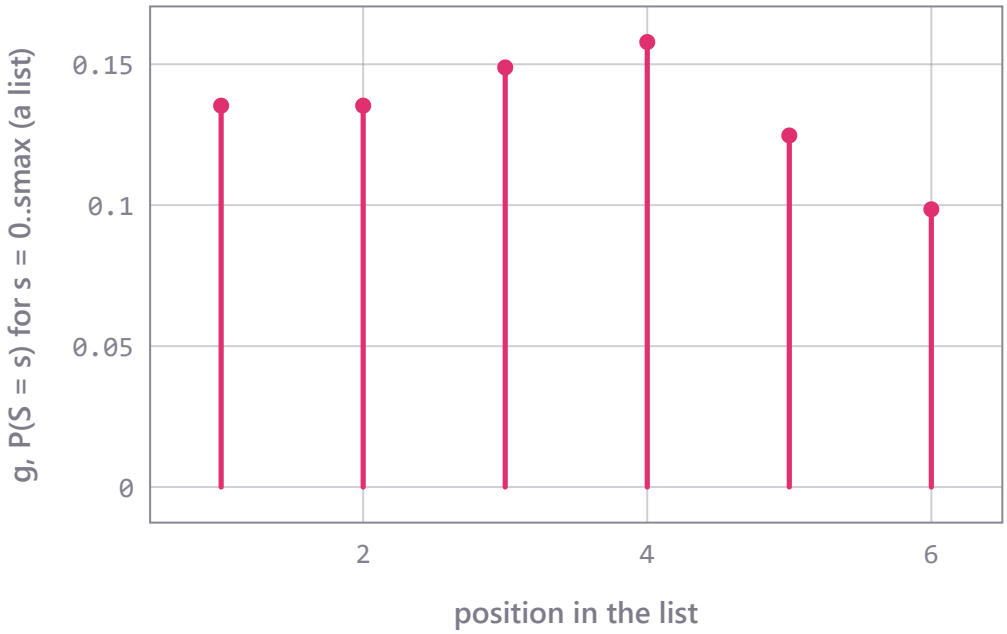
The probabilities of aggregate losses 0, 1, ..., smax (in monetary units) for a Poisson(lambda) number of claims with claim severities on 0, 1, 2, ... units, by Panjer's recursion, and P(S <= smax).

$$g_0 = e^{-\lambda(1-f_0)}, \quad g_s = \frac{\lambda}{s}\sum_{j=1}^{s} j\, f_j\, g_{s-j}$$

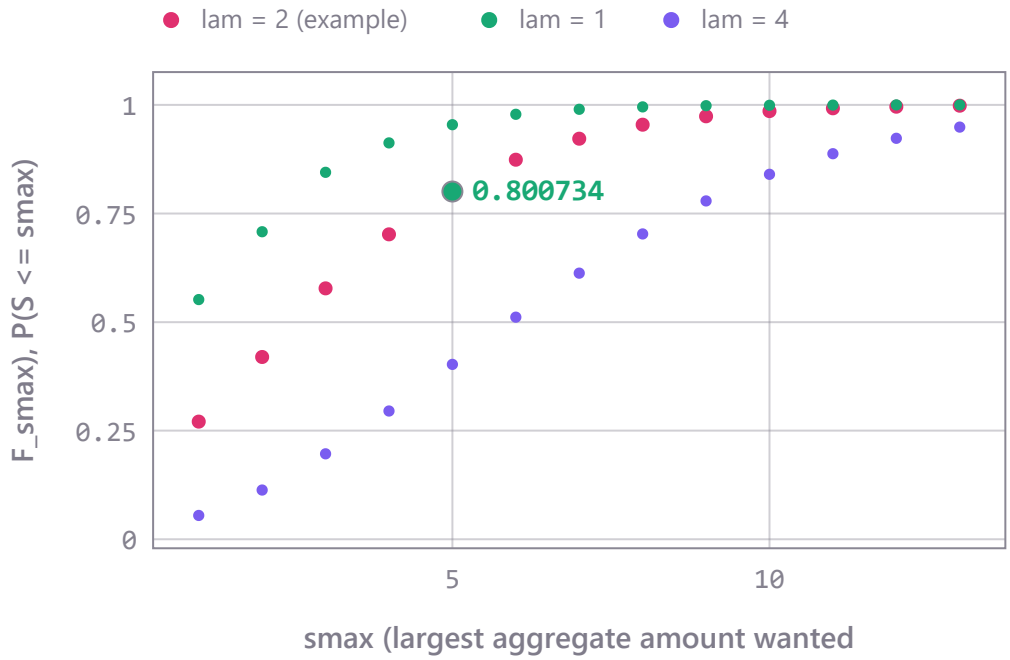
input	meaning	condition
`lam`	Poisson mean number of claims lambda	
`f`	severity probabilities of 0, 1, 2, ... units (a list adding to 1)	
`smax`	largest aggregate amount wanted	

output	meaning
`g`	P(S = s) for s = 0..smax (a list)
`F_smax`	P(S <= smax)

- **Method:** g_0 = e^(-lambda(1 - f_0)); $g_s = (1/s)$ sum_(j=1..s) lambda j f_j g_(s-j)
- **Assumes:** $0 < \lambda \leq 200$; severity probabilities add to 1; smax <= 300
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P(S = s) for s = 0..smax (a list) (g) returned by actuarial-science-ai999x for the worked example, one value for each position in the list (6 values, first 0.135335, last 0.0985692, largest 0.157891). Inputs: lam = 2.0, f = [0.0, 0.5, 0.3, 0.2], smax = 5.



F_smax (P(S <= smax)) against smax (largest aggregate amount wanted) from 1 to 13. One curve for each value of lam (Poisson mean number of claims lambda): 2, 1, 4. Other inputs as in the worked example: f = [0.0, 0.5, 0.3, 0.2]. 13 points per curve computed by actuarial-science-ai999x; marked: the worked example, smax = 5, F_smax = 0.800734.

```
actuarial_science_ai999x.solve('act_panjer_poisson', lam=2.0, f=[0.0, 0.5, 0.3, 0.2], smax=5)
# -> g = [0.135335283237, 0.135335283237, 0.14886881156, 0.157891163776, 0.124734019383,
0.0985691979573], F_smax = 0.800734   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_panjer_poisson', lam=1.0, f=[0.2, 0.4, 0.4], smax=4)
# -> g = [0.449328964117, 0.179731585647, 0.215677902776, 0.0766854765427, 0.0508041282095], F_smax
= 0.972228   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Panjer recursion for compound Poisson: lam = 2, f = [0, 0.5, 0.3, 0.2], smax = 5	SOLVED	g = [0.135335283237, 0.135335283237, 0.14886881156, 0.157891163776, 0.124734019383, 0.0985691979573], F_smax = 0.800734
Panjer recursion for compound Poisson: lam = 1, f = [0.2, 0.4, 0.4], smax = 4	SOLVED	g = [0.449328964117, 0.179731585647, 0.215677902776, 0.0766854765427, 0.0508041282095], F_smax = 0.972228
Panjer recursion for compound Poisson: f = [0, 0.5, 0.3, 0.2], smax = 5	NEEDS_INPUT	act_panjer_poisson needs lam (Poisson mean number of claims lambda). Example: solve('act_panjer_poisson', lam=2, f=[0.0,
Panjer recursion for compound Poisson: lam = 2, f = [0, 0.5, 0.3, 0.2], smax = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Panjer recursion for compound Poisson: lam = 2, f = [0, 0.5, 0.3, 0.3], smax = 5	INVALID_INPUT	Panjer recursion for a compound Poisson distribution: the severity probabilities must add up to 1.

Panjer recursion for a compound negative binomial distribution, `act_panjer_negative_binomial`

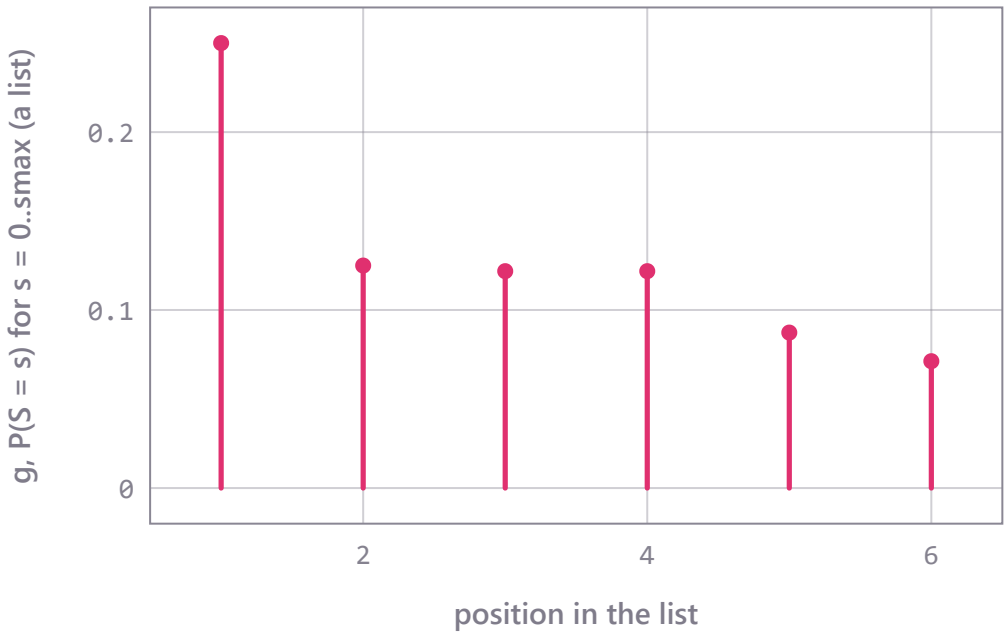
The probabilities of aggregate losses 0, 1, ..., smax units for a negative binomial (r, beta) number of claims with claim severities on 0, 1, 2, ... units, by Panjer's recursion, and P(S <= smax).

$$g_s = \frac{1}{1 - a f_0} \sum_{j=1}^{s} \left(a + \frac{bj}{s} \right) f_j \, g_{s-j}, \quad a = \frac{\beta}{1+\beta}, \; b = (r-1)a$$

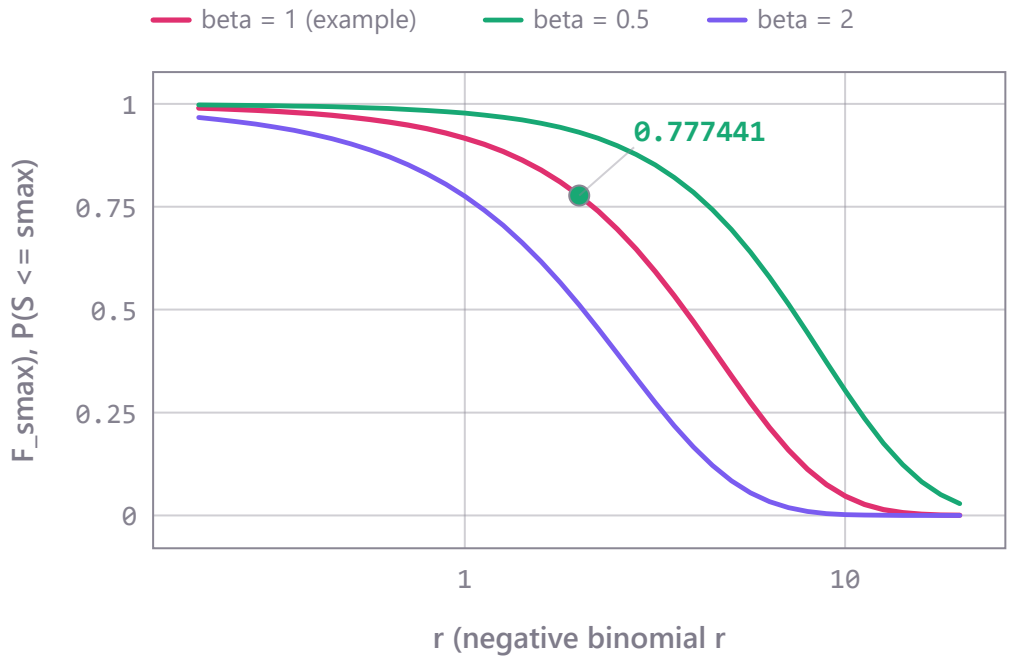
input	meaning	condition
`r`	negative binomial r	
`beta`	negative binomial beta	
`f`	severity probabilities of 0, 1, 2, ... units (a list adding to 1)	
`smax`	largest aggregate amount wanted	

output	meaning
`g`	P(S = s) for s = 0..smax (a list)
`F_smax`	P(S <= smax)

- **Method:** (a, b, 0) class: $a = \beta/(1+\beta)$, $b = (r-1)$ a; g_0 = (1 + beta(1 - f_0))^(-r); g_s = [sum (a + b j/s) f_j g_(s-j)]/(1 - a f_0)
- **Assumes:** r, $\beta > 0$; severity probabilities add to 1; smax <= 300
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P(S = s) for s = 0..smax (a list) (g) returned by actuarial-science-ai999x for the worked example, one value for each position in the list (6 values, first 0.25, last 0.0713086, largest 0.25). Inputs: r = 2.0, beta = 1.0, f = [0.0, 0.5, 0.3, 0.2], smax = 5.



F_smax (P(S <= smax)) against r (negative binomial r) from 0.2 to 20 on a logarithmic axis. One curve for each value of beta (negative binomial beta): 1, 0.5, 2. Other inputs as in the worked example: f = [0.0, 0.5, 0.3, 0.2], smax = 5. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, r = 2, F_smax = 0.777441.

```
actuarial_science_ai999x.solve('act_panjer_negative_binomial', r=2.0, beta=1.0, f=[0.0, 0.5, 0.3, 0.2], smax=5)
# -> g = [0.25, 0.125, 0.121875, 0.121875, 0.0873828125, 0.07130859375], F_smax = 0.777441 (SOLVED)
```

```
actuarial_science_ai999x.solve('act_panjer_negative_binomial', r=0.5, beta=2.0, f=[0.2, 0.4, 0.4], smax=4)
# -> g = [0.620173672946, 0.0954113342994, 0.117429334522, 0.0496816415287, 0.0404749037629], F_smax = 0.923171   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Panjer recursion for compound negative binomial: r = 2, beta = 1, f = [0, 0.5, 0.3, 0.2], smax = 5	SOLVED	g = [0.25, 0.125, 0.121875, 0.121875, 0.0873828125, 0.07130859375], F_smax = 0.777441
Panjer recursion for compound negative binomial: r = 0.5, beta = 2, f = [0.2, 0.4, 0.4], smax = 4	SOLVED	g = [0.620173672946, 0.0954113342994, 0.117429334522, 0.0496816415287, 0.0404749037629], F_smax = 0.923171
Panjer recursion for compound negative binomial: beta = 1, f = [0, 0.5, 0.3, 0.2], smax = 5	NEEDS_INPUT	act_panjer_negative_binomial needs r (negative binomial r). Example: solve('act_panjer_negative_binomial', r=2, beta=1,
Panjer recursion for compound negative binomial: r = 2, beta = 1, f = [0, 0.5, 0.3, 0.2], smax = 5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Stop-loss premium for a compound Poisson distribution, `act_stop_loss_premium`

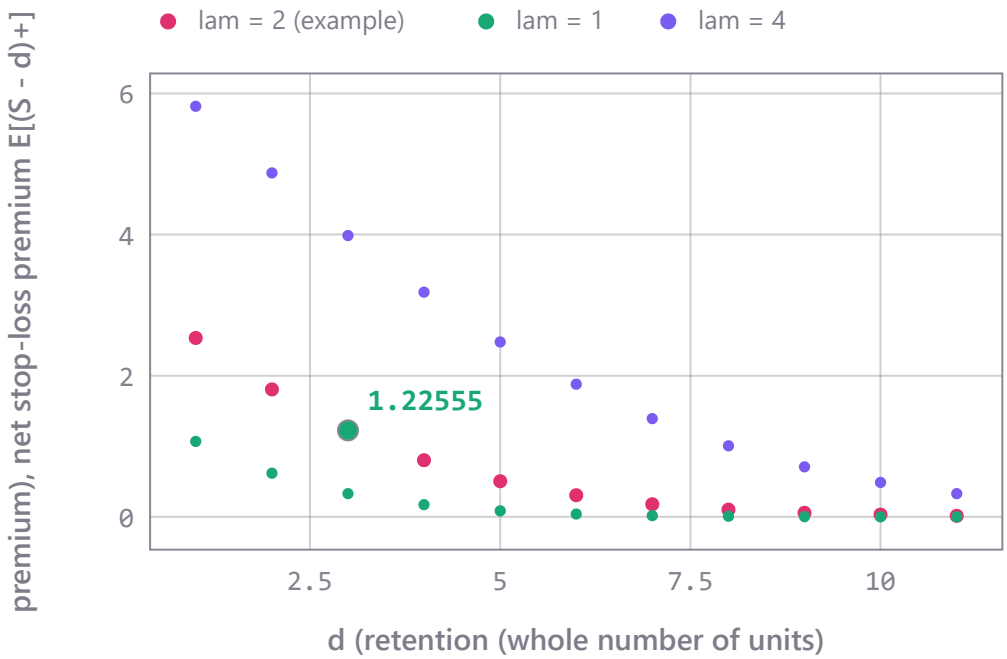
The net stop-loss premium E[(S - d)+] for a retention of d units, with E[S] and the limited expectation E[S ∧ d], when S is compound Poisson with claim severities on 0, 1, 2, ... units.

$$E[(S-d)_+] = E[S] - \sum_{s=0}^{d-1} (1 - F_S(s))$$

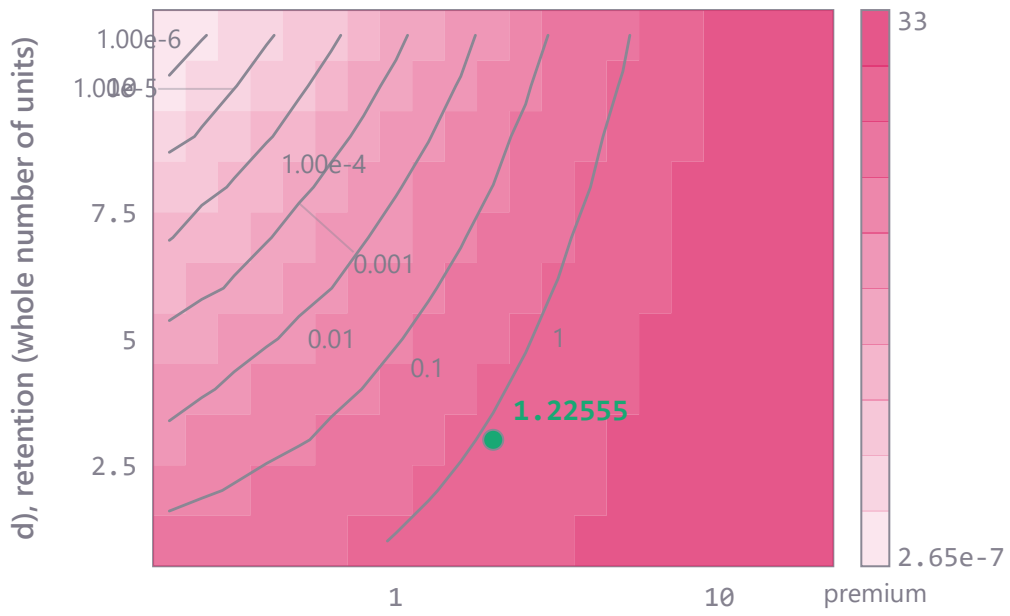
input	meaning	condition
`lam`	Poisson mean number of claims lambda	
`f`	severity probabilities of 0, 1, 2, ... units (a list adding to 1)	
`d`	retention (whole number of units)	

output	meaning
`premium`	net stop-loss premium E[(S - d)+]
`ES`	expected aggregate loss E[S]
`limited`	limited expectation E[S ∧ d]

- **Method:** E[(S - d)+] = E[S] - sum_(s=0..d-1) P(S > s), with P(S = s) from Panjer's recursion
- **Assumes:** $0 < \lambda \leq 200$; severity probabilities add to 1; whole-number retention
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



premium (net stop-loss premium E[(S - d)+]) against d (retention (whole number of units)) from 1 to 11. One curve for each value of lam (Poisson mean number of claims lambda): 2, 1, 4. Other inputs as in the worked example: f = [0.0, 0.5, 0.3, 0.2]. 11 points per curve computed by actuarial-science-ai999x; marked: the worked example, d = 3, premium = 1.22555.



premium (net stop-loss premium E[(S - d)+]) over lam (Poisson mean number of claims lambda) from 0.2 to 20 and d (retention (whole number of units)) from 1 to 11 (logarithmic horizontal axis): each of the 21 × 11 cells is one run of actuarial-science-ai999x, shaded by its value (light 2.6505e-07 to dark 33, logarithmic shading); iso-lines at premium = 1.00e-6, 1.00e-5, 1.00e-4, 0.001, 0.01, 0.1, 1. Other inputs as in the worked example: f = [0.0, 0.5, 0.3, 0.2]. Marked: the worked example, lam = 2, d = 3, premium = 1.22555.

```
actuarial_science_ai999x.solve('act_stop_loss_premium', lam=2.0, f=[0.0, 0.5, 0.3, 0.2], d=3)
# -> premium = 1.22555, ES = 3.4, limited = 2.17445   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_stop_loss_premium', lam=1.0, f=[0.2, 0.4, 0.4], d=2)
# -> premium = 0.27839, ES = 1.2, limited = 0.92161   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stop-loss premium: lam = 2, f = [0, 0.5, 0.3, 0.2], d = 3	SOLVED	premium = 1.22555, ES = 3.4, limited = 2.17445
Stop-loss premium: lam = 1, f = [0.2, 0.4, 0.4], d = 2	SOLVED	premium = 0.27839, ES = 1.2, limited = 0.92161
Stop-loss premium: f = [0, 0.5, 0.3, 0.2], d = 3	NEEDS_INPUT	act_stop_loss_premium needs lam (Poisson mean number of claims lambda). Example: solve('act_stop_loss_premium', lam=2, f
Stop-loss premium: lam = 2, f = [0, 0.5, 0.3, 0.2], d = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Ordinary deductible with exponential losses, `act_exponential_deductible`

The expected payment per loss and per payment, the limited expected value and the loss elimination ratio of an ordinary deductible d when losses are exponential with mean theta.

$$E[X \wedge d] = \theta\left(1 - e^{-d/\theta}\right), \quad E[(X-d)_+] = \theta e^{-d/\theta}, \quad e(d) = \theta$$

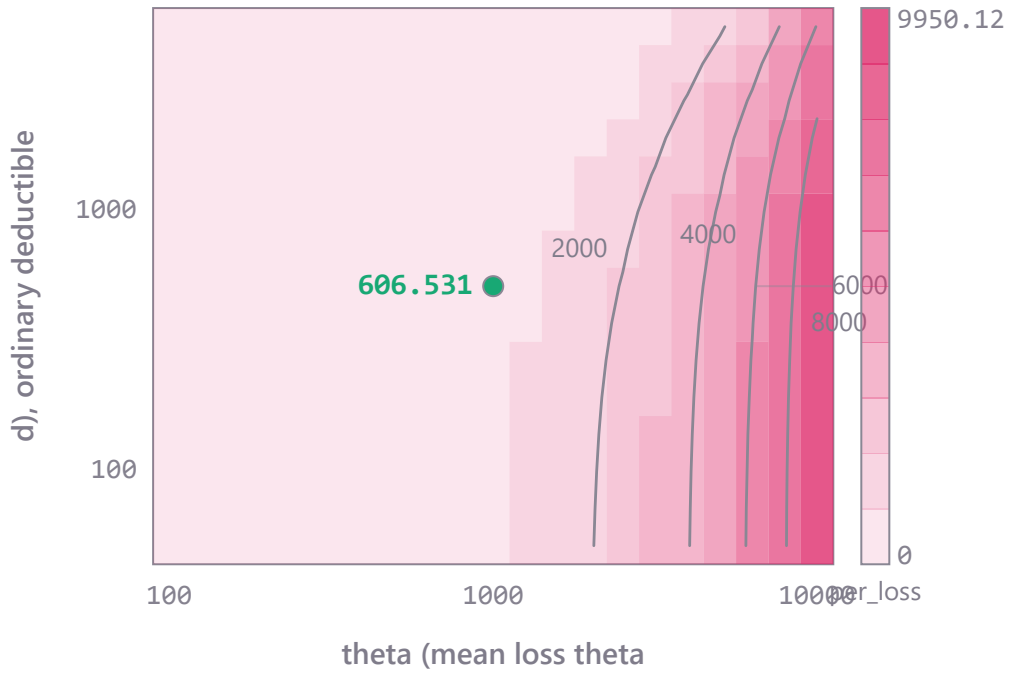
input	meaning	condition
`theta`	mean loss theta	
`d`	ordinary deductible	

output	meaning
`mean`	mean loss E[X]
`lev_d`	limited expected value E[X ∧ d]
`per_loss`	expected payment per loss E[(X - d)+]
`per_payment`	expected payment per payment (mean excess loss e(d))
`LER`	loss elimination ratio E[X ∧ d]/E[X]

- **Method:** per loss = E[X] - E[X ∧ d]; per payment = per loss / S(d); LER = E[X ∧ d]/E[X]; E[X ∧ d] = theta (1 - e^(-d/theta))
- **Assumes:** $\theta > 0$; $d \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



per_loss (expected payment per loss E[(X - d)+]) against theta (mean loss theta) from 100 to 10000 on a logarithmic axis, logarithmic output axis. One curve for each value of d (ordinary deductible): 500, 250, 1000. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, theta = 1000, per_loss = 606.531.



per_loss (expected payment per loss E[(X - d)+]) over theta (mean loss theta) from 100 to 10000 and d (ordinary deductible) from 50 to 5000 (logarithmic axes): each of the 21 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 0 to dark 9950.12); iso-lines at per_loss = 2000, 4000, 6000, 8000. Marked: the worked example, theta = 1000, d = 500, per_loss = 606.531.

```
actuarial_science_ai999x.solve('act_exponential_deductible', theta=1000.0, d=500.0)
# -> mean = 1000, lev_d = 393.469, per_loss = 606.531, per_payment = 1000, LER = 0.393469
(SOLVED)
```

```
actuarial_science_ai999x.solve('act_exponential_deductible', theta=2500.0, d=250.0)
# -> mean = 2500, lev_d = 237.906, per_loss = 2262.09, per_payment = 2500, LER = 0.0951626
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential losses with a deductible: theta = 1000, d = 500	SOLVED	mean = 1000, lev_d = 393.469, per_loss = 606.531, per_payment = 1000, LER = 0.393469
Exponential losses with a deductible: theta = 2500, d = 250	SOLVED	mean = 2500, lev_d = 237.906, per_loss = 2262.09, per_payment = 2500, LER = 0.0951626
Exponential losses with a deductible: d = 500	NEEDS_INPUT	act_exponential_deductible needs theta (mean loss theta). Example: solve('act_exponential_deductible', theta=1000, d=500
Exponential losses with a deductible: theta = 1000, d = 500, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Ordinary deductible with Pareto losses, `act_pareto_deductible`

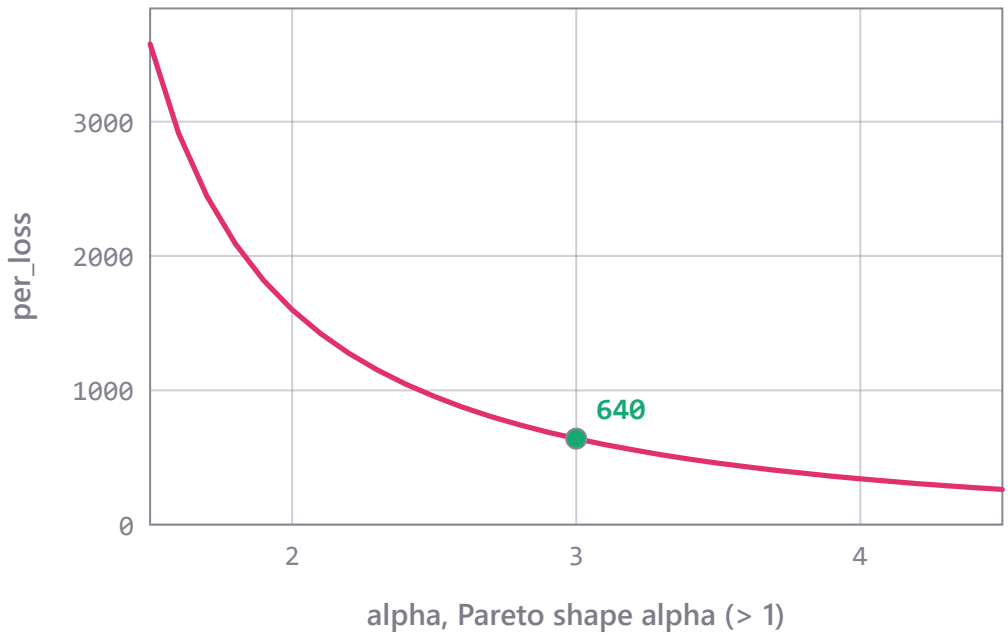
The expected payment per loss and per payment, the limited expected value and the loss elimination ratio of an ordinary deductible d when losses follow a two-parameter Pareto distribution (alpha, theta).

$$E[X \wedge d] = \frac{\theta}{\alpha - 1}\left[1 - \left(\frac{\theta}{\theta + d}\right)^{\alpha - 1}\right], \quad e(d) = \frac{\theta + d}{\alpha - 1}$$

input	meaning	condition
`alpha`	Pareto shape alpha (> 1)	
`theta`	Pareto scale theta	
`d`	ordinary deductible	

output	meaning
`mean`	mean loss E[X]
`lev_d`	limited expected value E[X ∧ d]
`per_loss`	expected payment per loss E[(X - d)+]
`per_payment`	expected payment per payment (mean excess loss e(d))
`LER`	loss elimination ratio E[X ∧ d]/E[X]

- **Method:** per loss = E[X] - E[X ∧ d]; per payment = per loss / S(d); LER = E[X ∧ d]/E[X]; E[X ∧ d] = $\theta/(\alpha - 1)$ [1 - (theta/(theta+d))^$(\alpha - 1)$]
- **Assumes:** $\alpha > 1$; $\theta > 0$; $d \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



per_loss (expected payment per loss E[(X - d)+]) against alpha (Pareto shape alpha (> 1)), with d = 500, theta = 2000. 31 points computed by actuarial-science-ai999x; marked: alpha = 3, per_loss = 640.



per_loss (expected payment per loss E[(X - d)+]) against theta (Pareto scale theta) from 200 to 20000 on a logarithmic axis. One curve for each value of alpha (Pareto shape alpha (> 1)): 3, 1.5, 6. Other inputs as in the worked example: d = 500.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, theta = 2000, per_loss = 640.

```
actuarial_science_ai999x.solve('act_pareto_deductible', alpha=3.0, theta=2000.0, d=500.0)
# -> mean = 1000, lev_d = 360, per_loss = 640, per_payment = 1250, LER = 0.36   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_pareto_deductible', alpha=2.5, theta=1000.0, d=1000.0)
# -> mean = 666.667, lev_d = 430.964, per_loss = 235.702, per_payment = 1333.33, LER = 0.646447
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pareto losses with a deductible: alpha = 3, theta = 2000, d = 500	SOLVED	mean = 1000, lev_d = 360, per_loss = 640, per_payment = 1250, LER = 0.36
Pareto losses with a deductible: alpha = 2.5, theta = 1000, d = 1000	SOLVED	mean = 666.667, lev_d = 430.964, per_loss = 235.702, per_payment = 1333.33, LER = 0.646447
Pareto losses with a deductible: theta = 2000, d = 500	NEEDS_INPUT	act_pareto_deductible needs alpha (Pareto shape alpha (> 1)). Example: solve('act_pareto_deductible', alpha=3, theta=200
Pareto losses with a deductible: alpha = 3, theta = 2000, d = 500, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Ordinary deductible with lognormal losses, `act_lognormal_deductible`

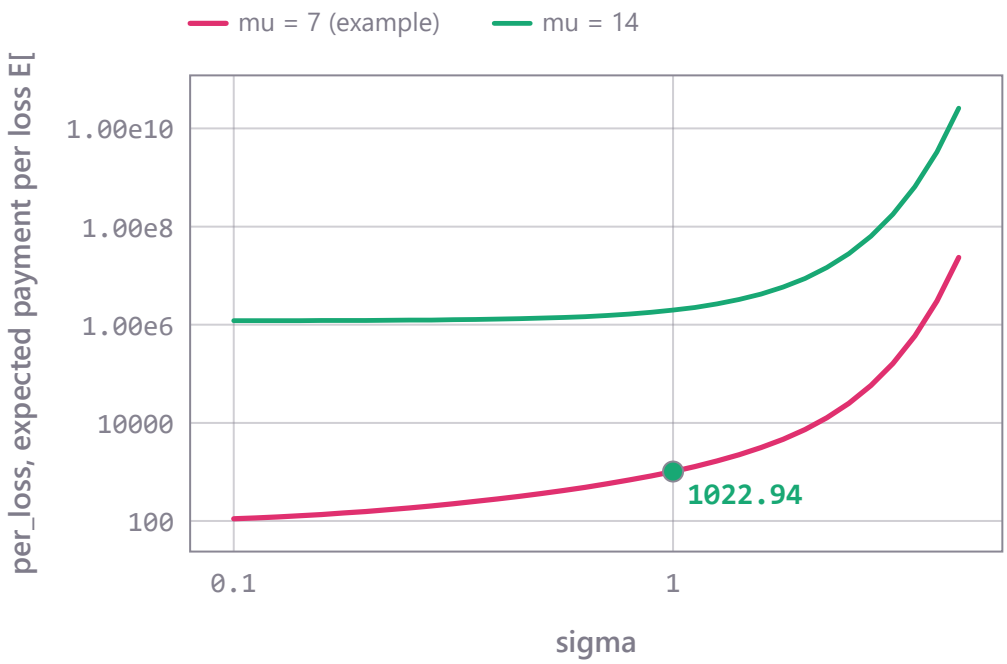
The expected payment per loss and per payment, the limited expected value and the loss elimination ratio of an ordinary deductible d when losses are lognormal (mu, sigma of the log of the loss).

$$E[X \wedge d] = e^{\mu+\sigma^2/2}\,\Phi\left(\frac{\ln d-\mu-\sigma^2}{\sigma}\right)+d\left[1-\Phi\left(\frac{\ln d-\mu}{\sigma}\right)\right]$$

input	meaning	condition
`mu`	mean of the log of the loss	
`sigma`	standard deviation of the log of the loss	
`d`	ordinary deductible	

output	meaning
`mean`	mean loss E[X]
`lev_d`	limited expected value E[X ∧ d]
`per_loss`	expected payment per loss E[(X - d)+]
`per_payment`	expected payment per payment (mean excess loss e(d))
`LER`	loss elimination ratio E[X ∧ d]/E[X]

- **Method:** per loss = E[X] - E[X ∧ d]; per payment = per loss / S(d); LER = E[X ∧ d]/E[X]; E[X ∧ d] = $e^{mu+\sigma^2/2}$ Phi((ln d - mu - sigma^2)/sigma) + d [$1 - Phi$((ln d - mu)/sigma)]
- **Assumes:** $\sigma > 0$; $d \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



per_loss (expected payment per loss E[(X - d)+]) against sigma (standard deviation of the log of the loss) from 0.1 to 10 on a logarithmic axis, logarithmic output axis. One curve for each value of mu (mean of the log of the loss): 7, 14. Other inputs as in the worked example: d = 1000.0. 34 points per curve computed by actuarial-science-ai999x; marked: the worked example, sigma = 1, per_loss = 1022.94.

```
actuarial_science_ai999x.solve('act_lognormal_deductible', mu=7.0, sigma=1.0, d=1000.0)
# -> mean = 1808.04, lev_d = 785.106, per_loss = 1022.94, per_payment = 1905.8, LER = 0.43423
(SOLVED)
```

```
actuarial_science_ai999x.solve('act_lognormal_deductible', mu=5.0, sigma=0.5, d=100.0)
# -> mean = 168.174, lev_d = 95.093, per_loss = 73.0811, per_payment = 93.0808, LER = 0.565444
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lognormal losses with a deductible: mu = 7, sigma = 1, d = 1000	SOLVED	mean = 1808.04, lev_d = 785.106, per_loss = 1022.94, per_payment = 1905.8, LER = 0.43423
Lognormal losses with a deductible: mu = 5, sigma = 0.5, d = 100	SOLVED	mean = 168.174, lev_d = 95.093, per_loss = 73.0811, per_payment = 93.0808, LER = 0.565444
Lognormal losses with a deductible: sigma = 1, d = 1000	NEEDS_INPUT	act_lognormal_deductible needs mu (mean of the log of the loss). Example: solve('act_lognormal_deductible', mu=7, sigma=
Lognormal losses with a deductible: mu = 7, sigma = 1, d = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Deductible, policy limit, coinsurance and inflation with exponential losses,

`act_exponential_coverage_modifications`

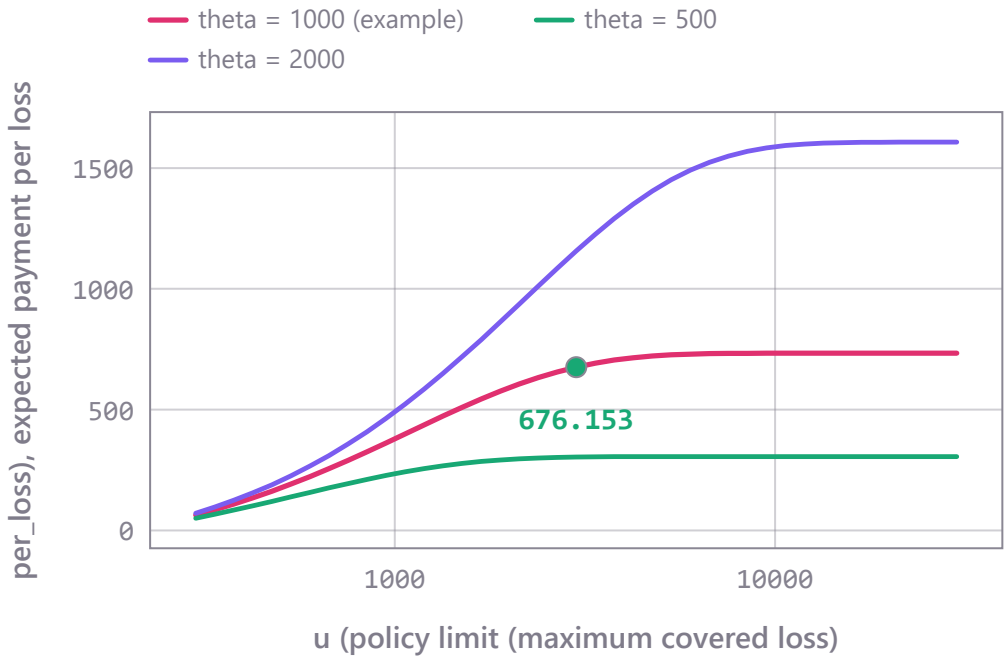
The expected insurer payment per loss and per payment for exponential losses (mean theta) under an ordinary deductible, a policy limit (maximum covered loss), coinsurance and uniform inflation.

$$E[Y^L] = c(1+r)\left[E\left[X \wedge \tfrac{u}{1+r}\right] - E\left[X \wedge \tfrac{d}{1+r}\right]\right], \quad E[Y^P] = \frac{E[Y^L]}{S_X\left(\tfrac{d}{1+r}\right)}$$

input	meaning	condition
`theta`	mean loss theta before inflation	
`d`	ordinary deductible	
`u`	policy limit (maximum covered loss)	
`c`	coinsurance fraction paid by the insurer (80% or 0.8)	
`r`	uniform loss inflation (0 for none)	

output	meaning
`per_loss`	expected payment per loss
`per_payment`	expected payment per payment
`max_payment`	largest possible payment c (u - d)

- **Method:** per loss = c $(1+r)$ [E[X ∧ $u/(1+r)$] - E[X ∧ $d/(1+r)$]]; per payment = per loss / S($d/(1+r)$)
- **Assumes:** $\theta > 0$; $0 \le d < u$; $0 < c \le 1$; $0 \le r < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



per_loss (expected payment per loss) against u (policy limit (maximum covered loss)) from 300 to 30000 on a logarithmic axis. One curve for each value of theta (mean loss theta before inflation): 1000, 500, 2000. Other inputs as in the worked example: d = 200.0, c = 0.8, r = 0.1. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, u = 3000, per_loss = 676.153.

```
actuarial_science_ai999x.solve('act_exponential_coverage_modifications', theta=1000.0, d=200.0,
u=3000.0, c=0.8, r=0.1)
# -> per_loss = 676.153, per_payment = 810.975, max_payment = 2240   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_exponential_coverage_modifications', theta=5000.0, d=500.0,
u=10000.0, c=1.0, r=0.0)
# -> per_loss = 3847.51, per_payment = 4252.16, max_payment = 9500   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential losses with deductible, limit and coinsurance: theta = 1000, d = 200, u = 3000, c = 0.8, r = 0.1	SOLVED	per_loss = 676.153, per_payment = 810.975, max_payment = 2240
Exponential losses with deductible, limit and coinsurance: theta = 5000, d = 500, u = 10000, c = 1, r = 0	SOLVED	per_loss = 3847.51, per_payment = 4252.16, max_payment = 9500
Exponential losses with deductible, limit and coinsurance: d = 200, u = 3000, c = 0.8, r = 0.1	NEEDS_INPUT	act_exponential_coverage_modifications needs theta (mean loss theta before inflation). Example: solve('act_exponential_c
Exponential losses with deductible, limit and coinsurance: theta = 1000, d = 200, u = 3000, c = 0.8, r = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Deductible, policy limit, coinsurance and inflation with Pareto losses,

`act_pareto_coverage_modifications`

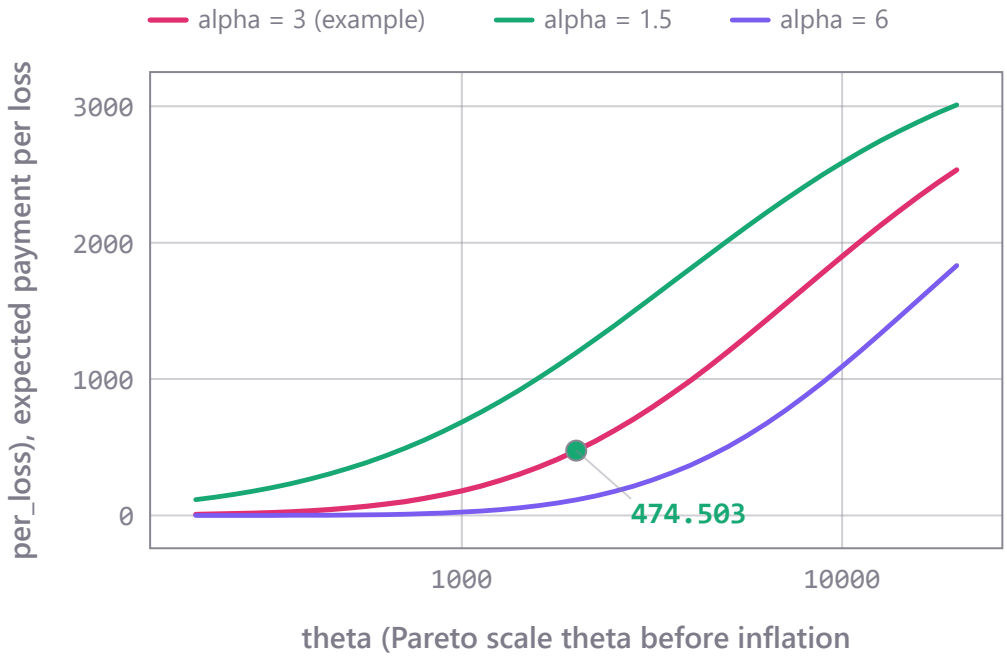
The expected insurer payment per loss and per payment for two-parameter Pareto losses under an ordinary deductible, a policy limit (maximum covered loss), coinsurance and uniform inflation.

$$E[Y^L] = c\,(1+r)\left[E\left[X \wedge \tfrac{u}{1+r}\right] - E\left[X \wedge \tfrac{d}{1+r}\right]\right], \quad E[Y^P] = \frac{E[Y^L]}{S_X\left(\tfrac{d}{1+r}\right)}$$

input	meaning	condition
`alpha`	Pareto shape alpha (> 1)	
`theta`	Pareto scale theta before inflation	
`d`	ordinary deductible	
`u`	policy limit (maximum covered loss)	
`c`	coinsurance fraction paid by the insurer (80% or 0.8)	
`r`	uniform loss inflation (0 for none)	

output	meaning
`per_loss`	expected payment per loss
`per_payment`	expected payment per payment
`max_payment`	largest possible payment c (u - d)

- **Method:** per loss = c $(1+r)$ [E[X ∧ $u/(1+r)$] - E[X ∧ $d/(1+r)$]]; per payment = per loss / S($d/(1+r)$)
- **Assumes:** $\alpha > 1$; $\theta > 0$; $0 \le d < u$; $0 < c \le 1$; $0 \le r < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



per_loss (expected payment per loss) against theta (Pareto scale theta before inflation) from 200 to 20000 on a logarithmic axis. One curve for each value of alpha (Pareto shape alpha (> 1)): 3, 1.5, 6. Other inputs as in the worked example: d = 500.0, u = 5000.0, c = 0.8, r = 0.05. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, theta = 2000, per_loss = 474.503.

```
actuarial_science_ai999x.solve('act_pareto_coverage_modifications', alpha=3.0, theta=2000.0,
d=500.0, u=5000.0, c=0.8, r=0.05)
# -> per_loss = 474.503, per_payment = 900.536, max_payment = 3600   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_pareto_coverage_modifications', alpha=2.0, theta=1000.0,
d=100.0, u=2000.0, c=0.9, r=0.0)
# -> per_loss = 518.182, per_payment = 627, max_payment = 1710   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pareto losses with deductible, limit and coinsurance: alpha = 3, theta = 2000, d = 500, u = 5000, c = 0.8, r = 0.05	SOLVED	per_loss = 474.503, per_payment = 900.536, max_payment = 3600
Pareto losses with deductible, limit and coinsurance: alpha = 2, theta = 1000, d = 100, u = 2000, c = 0.9, r = 0	SOLVED	per_loss = 518.182, per_payment = 627, max_payment = 1710
Pareto losses with deductible, limit and coinsurance: theta = 2000, d = 500, u = 5000, c = 0.8, r = 0.05	NEEDS_INPUT	act_pareto_coverage_modifications needs alpha (Pareto shape alpha (> 1)). Example: solve('act_pareto_coverage_modificati
Pareto losses with deductible, limit and coinsurance: alpha = 3, theta = 2000, d = 500, u = 5000, c = 0.8, r = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Insurer payment for one loss (deductible, limit, coinsurance), `act_single_loss_payment`

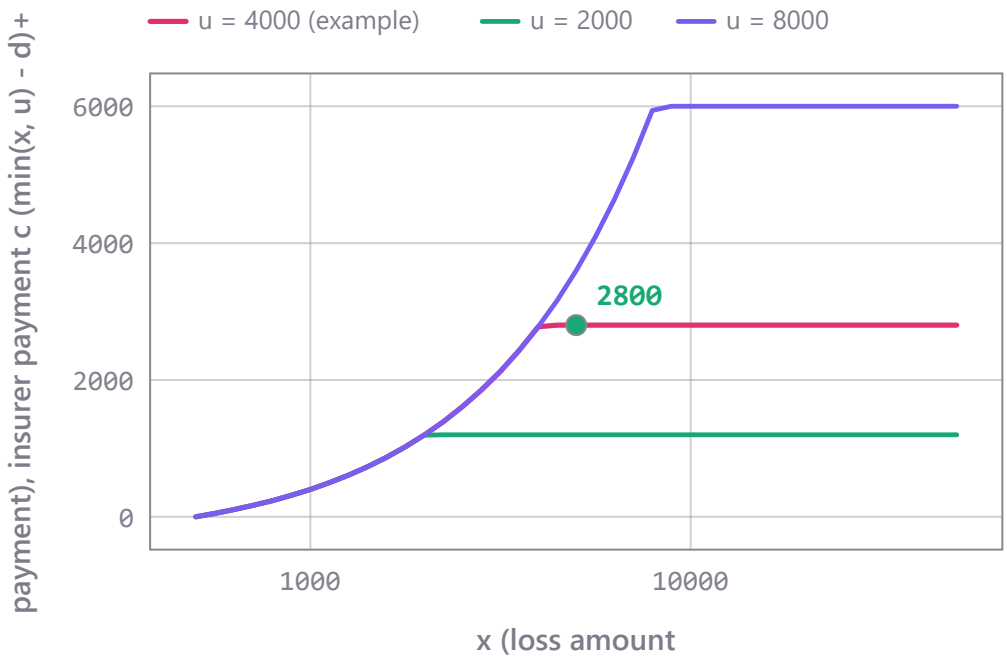
The amount the insurer pays and the amount the insured retains for a single loss x under an ordinary deductible d, a policy limit u (maximum covered loss) and coinsurance c.

$$Y = c(\min(X, u) - d)_+$$

input	meaning	condition
`x`	loss amount	
`d`	ordinary deductible	
`u`	policy limit (maximum covered loss)	
`c`	coinsurance fraction paid by the insurer (80% or 0.8)	

output	meaning
`payment`	insurer payment c (min(x, u) - d)+
`retained`	amount retained by the insured

- **Method:** payment = c x (min(x, u) - d)+
- **Assumes:** x, $d \geq 0$; $u > d$; $0 < c \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



payment (insurer payment c (min(x, u) - d)+) against x (loss amount) from 500 to 50000 on a logarithmic axis. One curve for each value of u (policy limit (maximum covered loss)): 4000, 2000, 8000. Other inputs as in the worked example: d = 500.0, c = 0.8. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, x = 5000, payment = 2800.

```
actuarial_science_ai999x.solve('act_single_loss_payment', x=5000.0, d=500.0, u=4000.0, c=0.8)
# -> payment = 2800, retained = 2200   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_single_loss_payment', x=1200.0, d=250.0, u=10000.0, c=0.9)
# -> payment = 855, retained = 345   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Insurer payment for a loss: x = 5000, d = 500, u = 4000, c = 0.8	SOLVED	payment = 2800, retained = 2200
Insurer payment for a loss: x = 1200, d = 250, u = 10000, c = 0.9	SOLVED	payment = 855, retained = 345
Insurer payment for a loss: d = 500, u = 4000, c = 0.8	NEEDS_INPUT	act_single_loss_payment needs x (loss amount). Example: solve('act_single_loss_payment', x=5000, d=500, u=4000, c=0.8).
Insurer payment for a loss: x = 5000, d = 500, u = 4000, c = 0.8, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

VaR and TVaR of a normal loss, `act_var_tvar_normal`

The value at risk and tail value at risk at security level p of a normally distributed loss.

$$\mathrm{VaR}_p = \mu + \sigma z_p, \quad \mathrm{TVaR}_p = \mu + \sigma \frac{\phi(z_p)}{1-p}$$

input	meaning	condition
`mu`	mean loss	
`sigma`	standard deviation of the loss	
`p`	security level (95% or 0.95)	

output	meaning
`VaR`	value at risk at level p (the p-quantile)
`TVaR`	tail value at risk E[X | X > VaR]

- **Method:** VaR = mu + sigma z_p; TVaR = mu + sigma phi(z_p)/(1 - p)
- **Assumes:** $\sigma > 0$; $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



TVaR (tail value at risk E[X | X > VaR]) against mu (mean loss) from 500 to 1500. One curve for each value of sigma (standard deviation of the loss): 200, 100, 400. Other inputs as in the worked example: p = 0.95. 31 points per curve computed by actuarial-science-ai999x; marked: the worked example, mu = 1000, TVaR = 1412.54.

```
actuarial_science_ai999x.solve('act_var_tvar_normal', mu=1000.0, sigma=200.0, p=0.95)
# -> VaR = 1328.97, TVaR = 1412.54   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_var_tvar_normal', mu=0.0, sigma=1.0, p=0.99)
# -> VaR = 2.32635, TVaR = 2.66521   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
VaR and TVaR of a normal loss: mu = 1000, sigma = 200, p = 0.95	SOLVED	VaR = 1328.97, TVaR = 1412.54
VaR and TVaR of a normal loss: mu = 0, sigma = 1, p = 0.99	SOLVED	VaR = 2.32635, TVaR = 2.66521
VaR and TVaR of a normal loss: sigma = 200, p = 0.95	NEEDS_INPUT	act_var_tvar_normal needs mu (mean loss). Example: solve('act_var_tvar_normal', mu=1000, sigma=200, p=0.95).
VaR and TVaR of a normal loss: mu = 1000, sigma = 200, p = 0.95, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

VaR and TVaR of an exponential loss, `act_var_tvar_exponential`

The value at risk and tail value at risk at security level p of an exponentially distributed loss with mean theta.

$$\mathrm{VaR}_p = -\theta \ln(1-p), \quad \mathrm{TVaR}_p = \mathrm{VaR}_p + \theta$$

input	meaning	condition
`theta`	mean loss theta	

input	meaning	condition
`p`	security level (95% or 0.95)	

output	meaning
`VaR`	value at risk at level p (the p-quantile)
`TVaR`	tail value at risk E[X | X > VaR]

- **Method:** VaR = -theta ln(1 - p); TVaR = VaR + theta
- **Assumes:** $\theta > 0$; $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



TVaR (tail value at risk E[X | X > VaR]) against p (security level (95% or 0.95)) from 0.475 to 1.425. One curve for each value of theta (mean loss theta): 1000, 500, 2000. 17 points per curve computed by actuarial-science-ai999x; marked: the worked example, p = 0.95, TVaR = 3995.73.

```
actuarial_science_ai999x.solve('act_var_tvar_exponential', theta=1000.0, p=0.95)
# -> VaR = 2995.73, TVaR = 3995.73   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_var_tvar_exponential', theta=500.0, p=0.99)
# -> VaR = 2302.59, TVaR = 2802.59   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
VaR and TVaR of an exponential loss: theta = 1000, p = 0.95	SOLVED	VaR = 2995.73, TVaR = 3995.73
VaR and TVaR of an exponential loss: theta = 500, p = 0.99	SOLVED	VaR = 2302.59, TVaR = 2802.59

question	outcome	result
VaR and TVaR of an exponential loss: p = 0.95	NEEDS_INPUT	act_var_tvar_exponential needs theta (mean loss theta). Example: solve('act_var_tvar_exponential', theta=1000, p=0.95).
VaR and TVaR of an exponential loss: theta = 1000, p = 0.95, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

VaR and TVaR of a Pareto loss, `act_var_tvar_pareto`

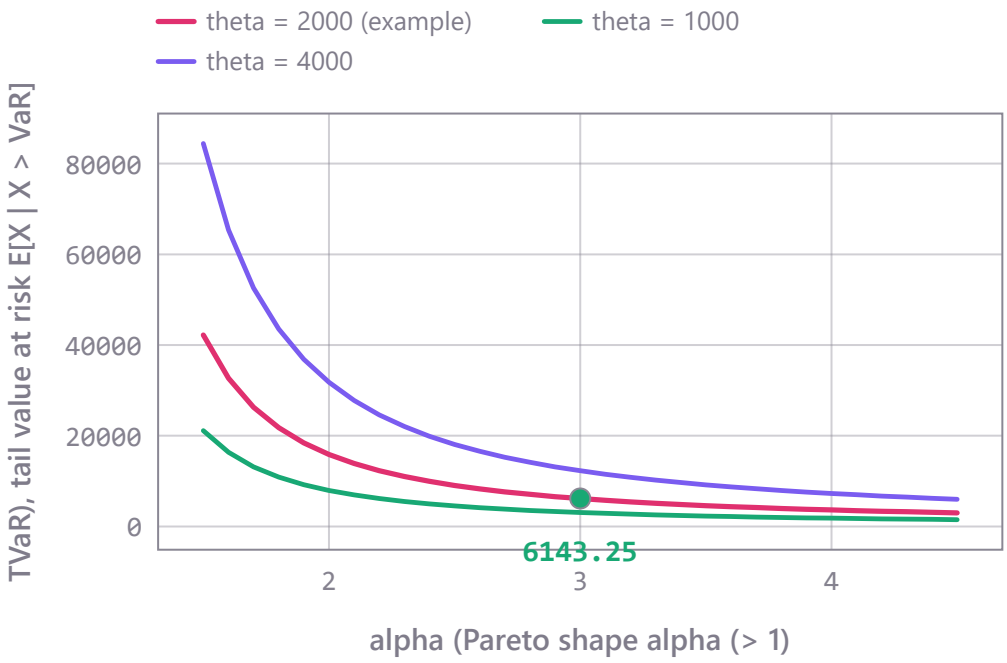
The value at risk and tail value at risk at security level p of a two-parameter Pareto loss (alpha, theta).

$$\mathrm{VaR}_p = \theta\left[(1-p)^{-1/\alpha} - 1\right], \quad \mathrm{TVaR}_p = \mathrm{VaR}_p + \frac{\theta + \mathrm{VaR}_p}{\alpha - 1}$$

input	meaning	condition
`alpha`	Pareto shape alpha (> 1)	
`theta`	Pareto scale theta	
`p`	security level (95% or 0.95)	

output	meaning
`VaR`	value at risk at level p (the p-quantile)
`TVaR`	tail value at risk E[X | X > VaR]

- **Method:** VaR = theta $[(1-p)^{-1/\alpha} - 1]$; TVaR = VaR + (theta + VaR)/(alpha - 1)
- **Assumes:** $\alpha > 1$; $\theta > 0$; $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



TVaR (tail value at risk E[X | X > VaR]) against alpha (Pareto shape alpha (> 1)) from 1.5 to 4.5. One curve for each value of theta (Pareto scale theta): 2000, 1000, 4000. Other inputs as in the worked example: p = 0.95. 31 points per curve computed by actuarial-science-ai999x; marked: the worked example, alpha = 3, TVaR = 6143.25.

```
actuarial_science_ai999x.solve('act_var_tvar_pareto', alpha=3.0, theta=2000.0, p=0.95)
# -> VaR = 3428.84, TVaR = 6143.25   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_var_tvar_pareto', alpha=2.5, theta=1000.0, p=0.99)
# -> VaR = 5309.57, TVaR = 9515.96   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
VaR and TVaR of a Pareto loss: alpha = 3, theta = 2000, p = 0.95	SOLVED	VaR = 3428.84, TVaR = 6143.25
VaR and TVaR of a Pareto loss: alpha = 2.5, theta = 1000, p = 0.99	SOLVED	VaR = 5309.57, TVaR = 9515.96
VaR and TVaR of a Pareto loss: theta = 2000, p = 0.95	NEEDS_INPUT	act_var_tvar_pareto needs alpha (Pareto shape alpha (> 1)). Example: solve('act_var_tvar_pareto', alpha=3, theta=2000, p
VaR and TVaR of a Pareto loss: alpha = 3, theta = 2000, p = 0.95, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

VaR and TVaR of a lognormal loss, `act_var_tvar_lognormal`

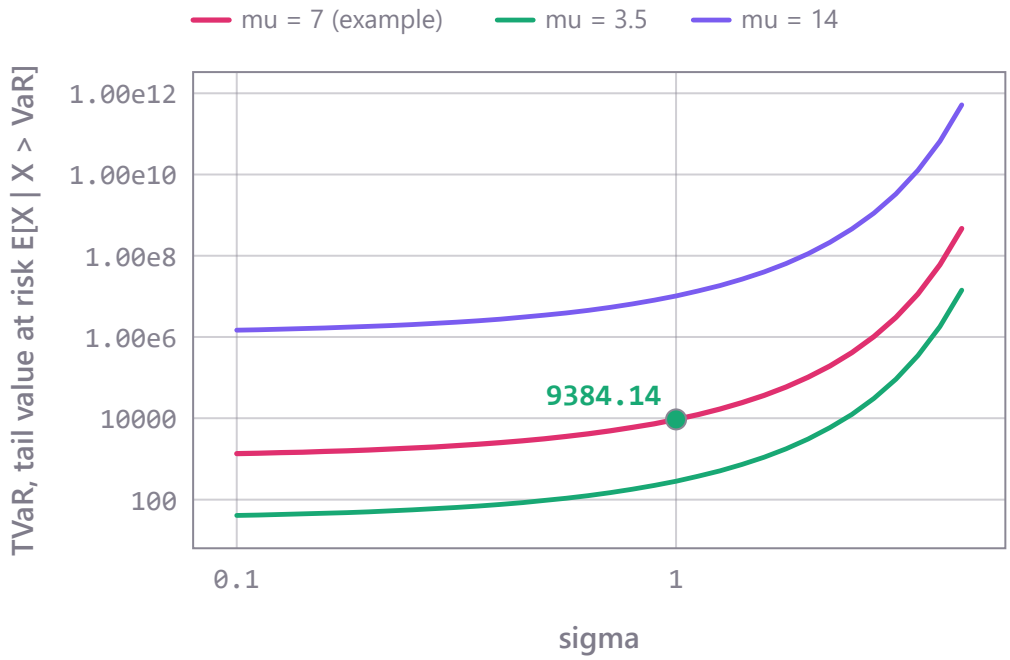
The value at risk and tail value at risk at security level p of a lognormal loss (mu, sigma of the log of the loss).

$$\mathrm{VaR}_p = e^{\mu+\sigma z_p}, \quad \mathrm{TVaR}_p = e^{\mu+\sigma^2/2}\,\frac{\Phi(\sigma - z_p)}{1-p}$$

input	meaning	condition
`mu`	mean of the log of the loss	
`sigma`	standard deviation of the log of the loss	
`p`	security level (95% or 0.95)	

output	meaning
`VaR`	value at risk at level p (the p-quantile)
`TVaR`	tail value at risk E[X | X > VaR]

- **Method:** VaR = exp(mu + sigma z_p); TVaR = e^(mu + sigma^2/2) Phi(sigma - z_p)/(1 - p)
- **Assumes:** $\sigma > 0$; $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



TVaR (tail value at risk E[X | X > VaR]) against sigma (standard deviation of the log of the loss) from 0.1 to 10 on a logarithmic axis, logarithmic output axis. One curve for each value of mu (mean of the log of the loss): 7, 3.5, 14. Other inputs as in the worked example: p = 0.95. 34 points per curve computed by actuarial-science-ai999x; marked: the worked example, sigma = 1, TVaR = 9384.14.

```
actuarial_science_ai999x.solve('act_var_tvar_lognormal', mu=7.0, sigma=1.0, p=0.95)
# -> VaR = 5680.84, TVaR = 9384.14   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_var_tvar_lognormal', mu=5.0, sigma=0.5, p=0.99)
# -> VaR = 474.933, TVaR = 570.092   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
VaR and TVaR of a lognormal loss: mu = 7, sigma = 1, p = 0.95	SOLVED	VaR = 5680.84, TVaR = 9384.14
VaR and TVaR of a lognormal loss: mu = 5, sigma = 0.5, p = 0.99	SOLVED	VaR = 474.933, TVaR = 570.092
VaR and TVaR of a lognormal loss: sigma = 1, p = 0.95	NEEDS_INPUT	act_var_tvar_lognormal needs mu (mean of the log of the loss). Example: solve('act_var_tvar_lognormal', mu=7, sigma=1, p
VaR and TVaR of a lognormal loss: mu = 7, sigma = 1, p = 0.95, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Premium principles: expected value, variance, standard deviation, `act_premium_principles`

The premium for a risk with a given mean and variance of loss by the expected value principle $(1+\theta)$ E[X], the variance principle E[X] + theta Var X or the standard deviation principle E[X] + theta sd(X).

$$\pi = (1+\theta)E[X], \quad \pi = E[X] + \theta\,\mathrm{Var}(X), \quad \pi = E[X] + \theta\sqrt{\mathrm{Var}(X)}$$

input	meaning	condition
`mean`	expected loss	
`var`	variance of the loss	
`theta`	loading factor	
`principle`	premium principle	

output	meaning
`premium`	premium
`loading`	risk loading added to the expected loss

- **Method:** expected value: $(1+\theta)$ E[X]; variance: E[X] + theta Var X; standard deviation: E[X] + theta sd(X)
- **Assumes:** mean, variance, loading >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



premium (premium) against mean (expected loss) from 500 to 1500. One curve for each value of theta (loading factor): 0.2, 0.26, 0.74. Other inputs as in the worked example: var = 250000.0, principle = expected_value. 31 points per curve computed by actuarial-science-ai999x; marked: the worked example, mean = 1000, premium = 1200.

```
actuarial_science_ai999x.solve('act_premium_principles', mean=1000.0, var=250000.0, theta=0.2,
principle='expected_value')
# -> premium = 1200, loading = 200   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_premium_principles', mean=1000.0, var=250000.0, theta=0.1,
principle='standard_deviation')
# -> premium = 1050, loading = 50   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Premium principle: mean = 1000, var = 250000, theta = 0.2, principle = expected_value	SOLVED	premium = 1200, loading = 200
Premium principle: mean = 1000, var = 250000, theta = 0.1, principle = standard_deviation	SOLVED	premium = 1050, loading = 50
Premium principle: var = 250000, theta = 0.2, principle = expected_value	NEEDS_INPUT	act_premium_principles needs mean (expected loss). Example: solve('act_premium_principles', mean=1000, var=250000, theta
Premium principle: mean = 1000, var = 250000, theta = 0.2, principle = expected_value, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Limited fluctuation full credibility standard for claim counts,

`act_limited_fluctuation_full_credibility`

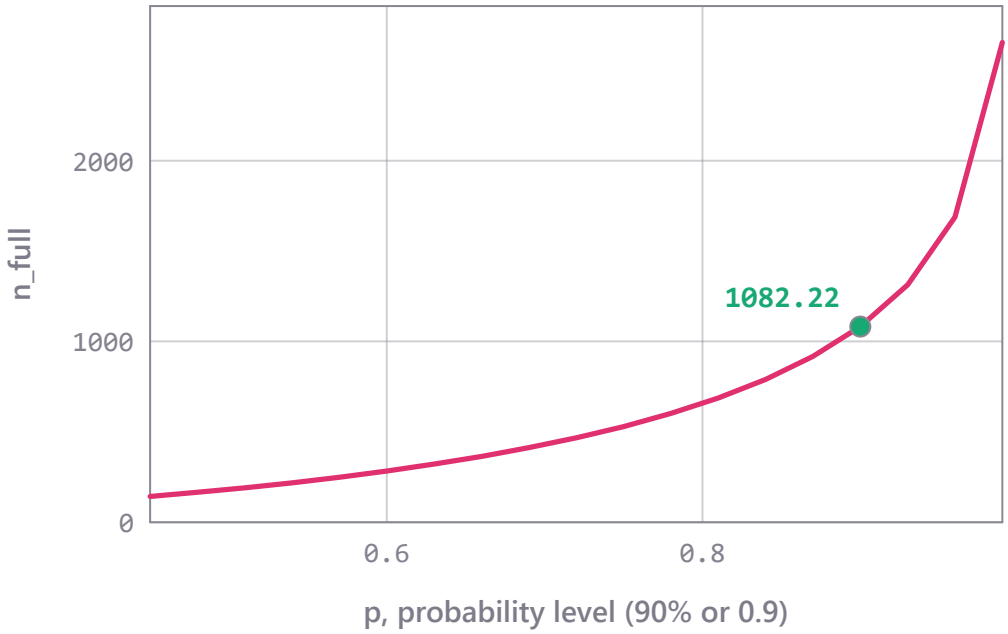
The expected number of claims needed for full credibility of the claim frequency so that the observed number is within a fraction k of its mean with probability p (Poisson claim counts).

$$n_F = \left(\frac{z_{(1+p)/2}}{k} \right)^2$$

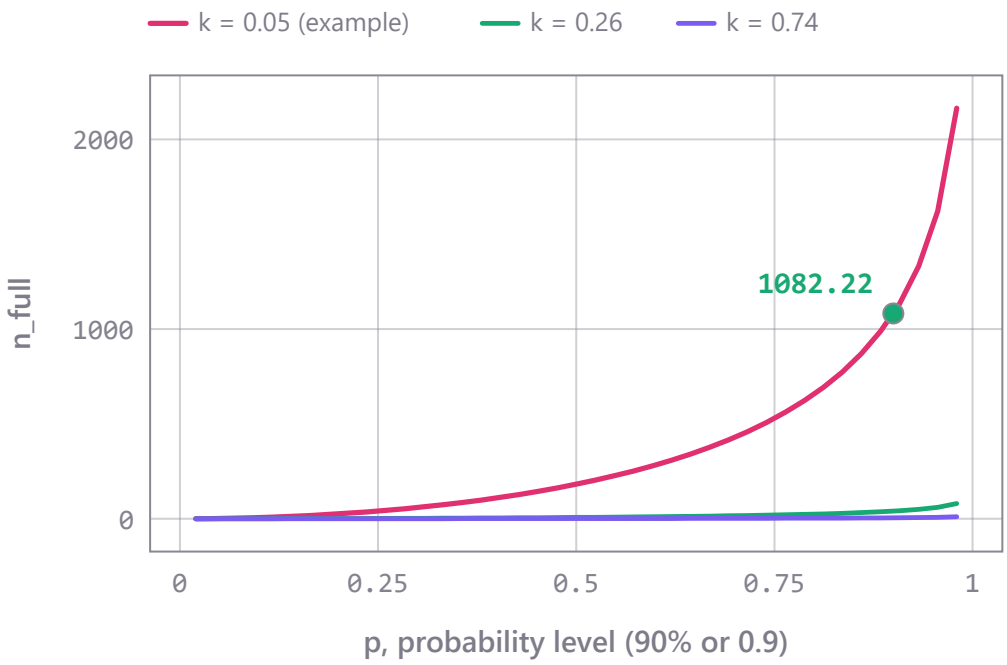
input	meaning	condition
`p`	probability level (90% or 0.9)	
`k`	tolerance as a fraction of the mean (5% or 0.05)	

output	meaning
`n_full`	expected claims needed for full credibility $(z/k)^2$
`z`	standard normal quantile z_((1+p)/2)

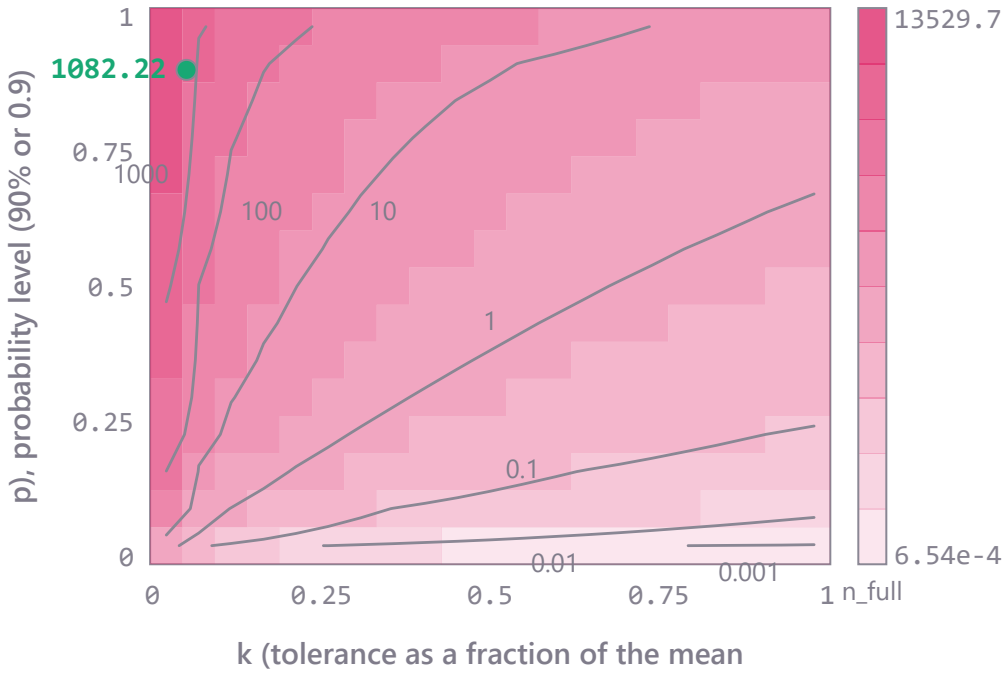
- **Method:** n_F = (z_((1+p)/2) / k)^2
- **Assumes:** $0 < p < 1$; $0 < k < 1$; Poisson claim counts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n_full (expected claims needed for full credibility (z/k)^2) against p (probability level (90% or 0.9)), with k = 0.05. 19 points computed by actuarial-science-ai999x; marked: p = 0.9, n_full = 1082.22.



n_full (expected claims needed for full credibility (z/k)^2) against p (probability level (90% or 0.9)) from 0.02 to 0.98. One curve for each value of k (tolerance as a fraction of the mean (5% or 0.05)): 0.05, 0.26, 0.74. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, p = 0.9, n_full = 1082.22.



n_full (expected claims needed for full credibility (z/k)^2) over k (tolerance as a fraction of the mean (5% or 0.05)) from 0.02 to 0.98 and p (probability level (90% or 0.9)) from 0.02 to 0.98: each of the 21 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 0.000654363 to dark 13529.7, logarithmic shading); iso-lines at n_full = 0.001, 0.01, 0.1, 1, 10, 100, 1000. Marked: the worked example, k = 0.05, p = 0.9, n_full = 1082.22.

```
actuarial_science_ai999x.solve('act_limited_fluctuation_full_credibility', p=0.9, k=0.05)
# -> n_full = 1082.22, z = 1.64485   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_limited_fluctuation_full_credibility', p=0.95, k=0.05)
# -> n_full = 1536.58, z = 1.95996   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Full credibility standard: p = 0.9, k = 0.05	SOLVED	n_full = 1082.22, z = 1.64485
Full credibility standard: p = 0.95, k = 0.05	SOLVED	n_full = 1536.58, z = 1.95996
Full credibility standard: k = 0.05	NEEDS_INPUT	act_limited_fluctuation_full_credibility needs p (probability level (90% or 0.9)). Example: solve('act_limited_fluctuati
Full credibility standard: p = 0.9, k = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Limited fluctuation full credibility for severity and aggregate losses,

`act_limited_fluctuation_severity_aggregate`

The expected number of claims needed for full credibility of the claim severity, of aggregate losses (pure premium) and of the claim frequency, for Poisson claim counts and a severity with the given mean and standard deviation.

$$n_S = n_0\left(1+\frac{\sigma_X^2}{\mu_X^2}\right), \quad n_0 = \left(\frac{z_{(1+p)/2}}{k}\right)^2$$

input	meaning	condition
`p`	probability level (90% or 0.9)	
`k`	tolerance as a fraction of the mean (5% or 0.05)	
`mean_x`	mean claim severity	
`sd_x`	standard deviation of the claim severity	

output	meaning
`claims_aggregate`	claims for full credibility of aggregate losses n0 $(1+CV^2)$
`claims_severity`	claims for full credibility of severity n0 CV^2
`claims_frequency`	claims for full credibility of frequency n0

- **Method:** $n_0 = (z/k)^2$; aggregate: n0 (1 + (sd/mean)^2); severity: n0 (sd/mean)^2
- **Assumes:** $0 < p < 1$; $0 < k < 1$; Poisson claim counts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



claims_aggregate (claims for full credibility of aggregate losses n0 (1 + CV^2)) against mean_x (mean claim severity) from 100 to 10000 on a logarithmic axis. One curve for each value of sd_x (standard deviation of the claim severity): 2000, 1000, 4000. Other inputs as in the worked example: p = 0.9, k = 0.05. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, mean_x = 1000, claims_aggregate = 5411.09.

```
actuarial_science_ai999x.solve('act_limited_fluctuation_severity_aggregate', p=0.9, k=0.05,
mean_x=1000.0, sd_x=2000.0)
# -> claims_aggregate = 5411.09, claims_severity = 4328.87, claims_frequency = 1082.22   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_limited_fluctuation_severity_aggregate', p=0.95, k=0.1,
mean_x=500.0, sd_x=500.0)
# -> claims_aggregate = 768.292, claims_severity = 384.146, claims_frequency = 384.146   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Full credibility for aggregate losses: p = 0.9, k = 0.05, mean_x = 1000, sd_x = 2000	SOLVED	claims_aggregate = 5411.09, claims_severity = 4328.87, claims_frequency = 1082.22
Full credibility for aggregate losses: p = 0.95, k = 0.1, mean_x = 500, sd_x = 500	SOLVED	claims_aggregate = 768.292, claims_severity = 384.146, claims_frequency = 384.146
Full credibility for aggregate losses: k = 0.05, mean_x = 1000, sd_x = 2000	NEEDS_INPUT	act_limited_fluctuation_severity_aggregate needs p (probability level (90% or 0.9)). Example: solve('act_limited_fluctua
Full credibility for aggregate losses: p = 0.9, k = 0.05, mean_x = 1000, sd_x = 2000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Partial credibility (square root rule), `act_partial_credibility`

The credibility factor Z = min(1, sqrt(n/n_F)) of an experience with n claims against a full credibility standard n_F, and the credibility-weighted estimate Z xbar + (1 - Z) M.

$$Z = \min\left(1, \sqrt{n/n_F}\right), \quad P_C = Z\bar{X} + (1 - Z)M$$

input	meaning	condition
`n`	observed (or expected) number of claims	
`n_full`	full credibility standard in claims	
`xbar`	observed experience (mean or rate)	
`M`	complement of credibility (manual rate)	

output	meaning
`Z`	credibility factor
`premium`	credibility-weighted estimate

- **Method:** Z = min(1, sqrt(n / n_F)); estimate = Z xbar + (1 - Z) M
- **Assumes:** $n \geq 0$; $n_F > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Z (credibility factor) against n (observed (or expected) number of claims) from 30 to 3000 on a logarithmic axis. One curve for each value of n_full (full credibility standard in claims): 1082, 541, 2164. Other inputs as in the worked example: xbar = 800.0, M = 1000.0. 41 points per curve computed by actuarial-science-ai999x; marked: the worked example, n = 300, Z = 0.526559.

```
actuarial_science_ai999x.solve('act_partial_credibility', n=300.0, n_full=1082.0, xbar=800.0,
M=1000.0)
# -> Z = 0.526559, premium = 894.688   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_partial_credibility', n=2000.0, n_full=1082.0, xbar=800.0,
M=1000.0)
# -> Z = 1, premium = 800   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Partial credibility: n = 300, n_full = 1082, xbar = 800, M = 1000	SOLVED	Z = 0.526559, premium = 894.688
Partial credibility: n = 2000, n_full = 1082, xbar = 800, M = 1000	SOLVED	Z = 1, premium = 800
Partial credibility: n_full = 1082, xbar = 800, M = 1000	NEEDS_INPUT	act_partial_credibility needs n (observed (or expected) number of claims). Example: solve('act_partial_credibility', n=3
Partial credibility: n = 300, n_full = 1082, xbar = 800, M = 1000, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Buhlmann credibility premium (known structural parameters), `act_buhlmann_credibility`

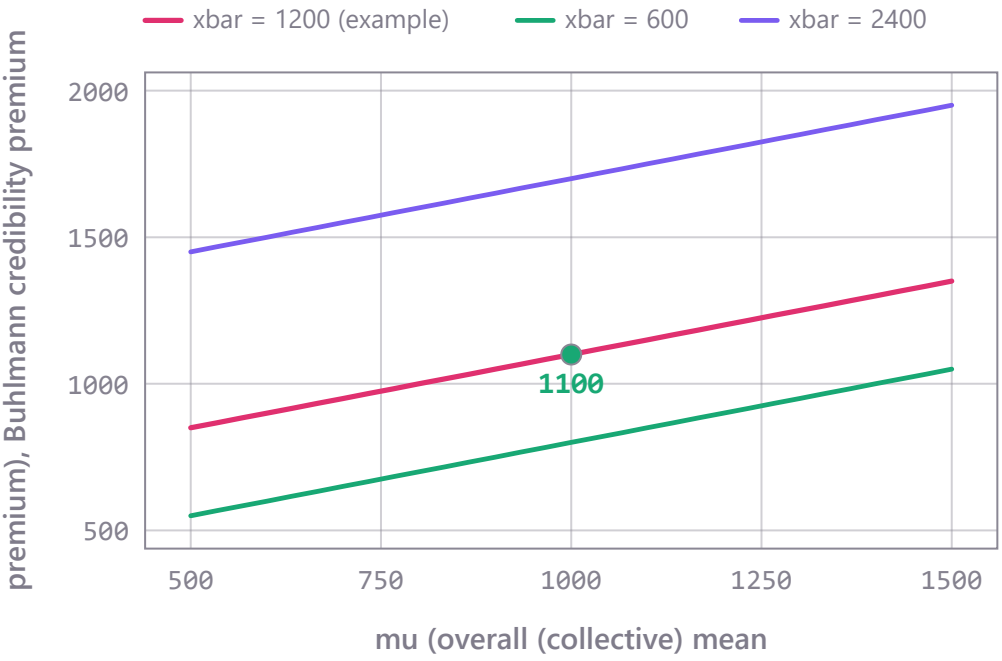
The Buhlmann credibility factor $Z = n/(n+k)$, k = EPV/VHM, and the credibility premium Z xbar + (1 - Z) mu from the overall mean, the expected process variance, the variance of the hypothetical means and n observations with mean xbar.

$$k = \frac{\text{EPV}}{\text{VHM}}, \quad Z = \frac{n}{n+k}, \quad P_C = Z\bar{X} + (1-Z)\mu$$

input	meaning	condition
`mu`	overall (collective) mean	
`EPV`	expected process variance	
`VHM`	variance of the hypothetical means	
`n`	number of observations	
`xbar`	mean of the observations	

output	meaning
`k`	Buhlmann k = EPV/VHM
`Z`	credibility factor n/(n + k)
`premium`	Buhlmann credibility premium

- **Method:** k = EPV/VHM; $Z = n/(n+k)$; $P = Z$ xbar + (1 - Z) mu
- **Assumes:** EPV, VHM > 0; $n > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



premium (Buhlmann credibility premium) against mu (overall (collective) mean) from 500 to 1500. One curve for each value of xbar (mean of the observations): 1200, 600, 2400. Other inputs as in the worked example: EPV = 50000.0, VHM = 10000.0, n = 5.0. 31 points per curve computed by actuarial-science-ai999x; marked: the worked example, mu = 1000, premium = 1100.

```
actuarial_science_ai999x.solve('act_buhlmann_credibility', mu=1000.0, EPV=50000.0, VHM=10000.0,
n=5.0, xbar=1200.0)
# -> k = 5, Z = 0.5, premium = 1100   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_buhlmann_credibility', mu=0.2, EPV=0.2, VHM=0.04, n=3.0,
xbar=0.5)
# -> k = 5, Z = 0.375, premium = 0.3125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Buhlmann credibility premium: mu = 1000, EPV = 50000, VHM = 10000, n = 5, xbar = 1200	SOLVED	k = 5, Z = 0.5, premium = 1100
Buhlmann credibility premium: mu = 0.2, EPV = 0.2, VHM = 0.04, n = 3, xbar = 0.5	SOLVED	k = 5, Z = 0.375, premium = 0.3125
Buhlmann credibility premium: EPV = 50000, VHM = 10000, n = 5, xbar = 1200	NEEDS_INPUT	act_buhlmann_credibility needs mu (overall (collective) mean). Example: solve('act_buhlmann_credibility', mu=1000, EPV=5
Buhlmann credibility premium: mu = 1000, EPV = 50000, VHM = 10000, n = 5, xbar = 1200, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Empirical Buhlmann credibility from a data table, `act_buhlmann_empirical`

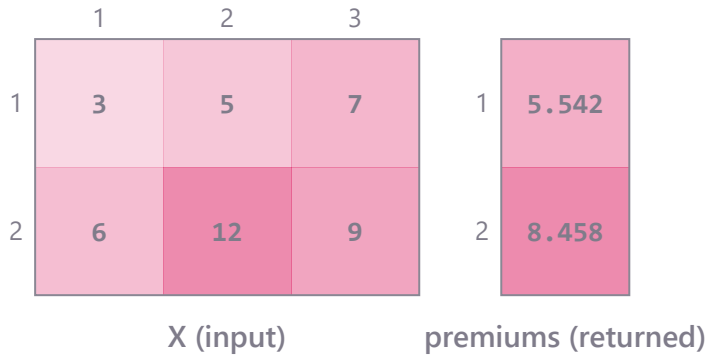
The nonparametric estimates of the expected process variance and of the variance of the hypothetical means from r risks observed for n periods each, the Buhlmann k and Z, and each risk's credibility premium.

$$\hat{v} = \frac{1}{r(n-1)} \sum_{i,j} \left(X_{ij} - \bar{X}_i \right)^2, \quad \hat{a} = \frac{1}{r-1} \sum_{i} \left(\bar{X}_i - \bar{X} \right)^2 - \frac{\hat{v}}{n}$$

input	meaning	condition
`X`	observations: one row per risk, one column per period (rows of equal length)	

output	meaning
`EPV`	estimated expected process variance
`VHM`	estimated variance of the hypothetical means
`k`	Buhlmann k = EPV/VHM
`Z`	credibility factor n/(n + k)
`premiums`	credibility premium of each risk (a list)
`mean`	overall mean

- **Method:** EPV = mean of the within-risk sample variances; VHM = sample variance of the risk means - EPV/n; Z = n/(n + EPV/VHM)
- **Assumes:** at least 2 risks and 2 periods; estimated VHM > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): X (input); premiums (returned). Returned with them by actuarial-science-ai999x: EPV = 6.5, VHM = 5.83333, k = 1.11429, Z = 0.729167, mean = 7.

```
actuarial_science_ai999x.solve('act_buhlmann_empirical', X=[[3.0, 5.0, 7.0], [6.0, 12.0, 9.0]])
# -> EPV = 6.5, VHM = 5.83333, k = 1.11429, Z = 0.729167, premiums = [5.54166666667,
8.45833333333], mean = 7   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_buhlmann_empirical', X=[[100.0, 120.0, 110.0, 130.0], [200.0,
180.0, 210.0, 190.0], [150.0, 160.0, 140.0, 170.0]])
# -> EPV = 166.667, VHM = 1558.33, k = 0.106952, Z = 0.973958, premiums = [116.041666667,
193.958333333, 155.0], mean = 155   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Empirical Buhlmann credibility: X = [[3, 5, 7], [6, 12, 9]]	SOLVED	EPV = 6.5, VHM = 5.83333, k = 1.11429, Z = 0.729167, premiums = [5.54166666667, 8.45833333333], mean = 7
Empirical Buhlmann credibility: X = [[100, 120, 110, 130], [200, 180, 210, 190], [150, 160, 140, 170]]	SOLVED	EPV = 166.667, VHM = 1558.33, k = 0.106952, Z = 0.973958, premiums = [116.041666667, 193.958333333, 155.0], mean = 155
Empirical Buhlmann credibility:	NEEDS_INPUT	act_buhlmann_empirical needs X (observations: one row per risk, one column per period (rows of equal length)). Example:
Empirical Buhlmann credibility: X = [[3, 5, 7], [6, 12, 9]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Empirical Buhlmann credibility: X = [[1, 2], [1, 2]]	INVALID_INPUT	Empirical Buhlmann credibility from a data table: the estimated variance of the hypothetical means is not positive: no c

Buhlmann-Straub credibility with exposures, `act_buhlmann_straub`

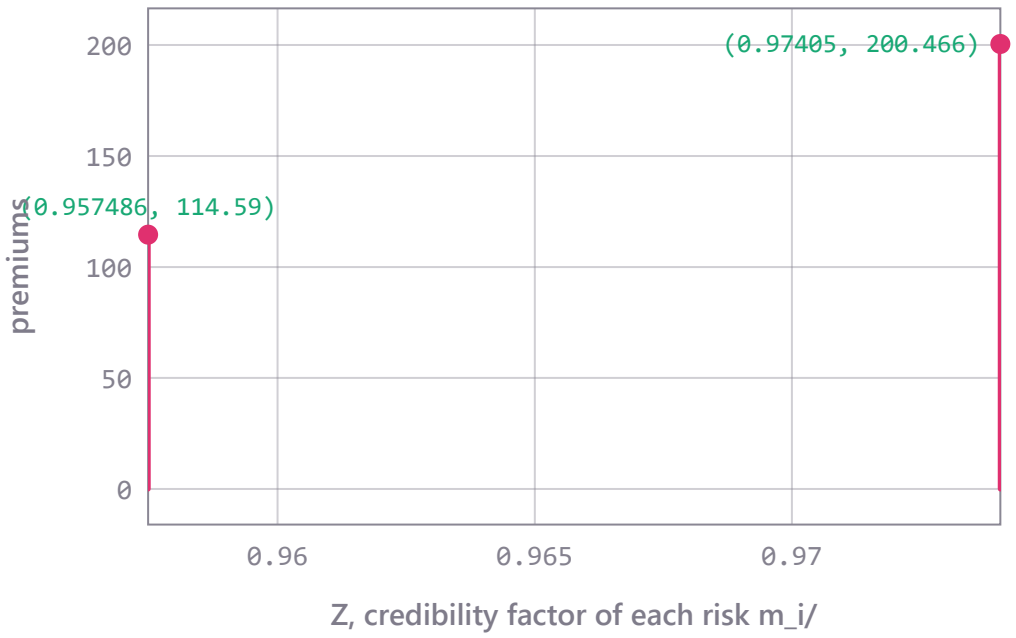
The nonparametric Buhlmann-Straub estimates of the expected process variance and of the variance of the hypothetical means from losses per unit of exposure and the exposures of r risks over n periods, each risk's credibility factor and its credibility premium (complement: the exposure-weighted overall mean).

$$\hat{v} = \frac{\sum_{i,j} m_{ij}(X_{ij} - \bar{X}_i)^2}{r(n-1)}, \quad \hat{a} = \frac{\sum_i m_i(\bar{X}_i - \bar{X})^2 - \hat{v}(r-1)}{m - \sum_i m_i^2/m}, \quad Z_i = \frac{m_i}{m_i + \hat{v}/\hat{a}}$$

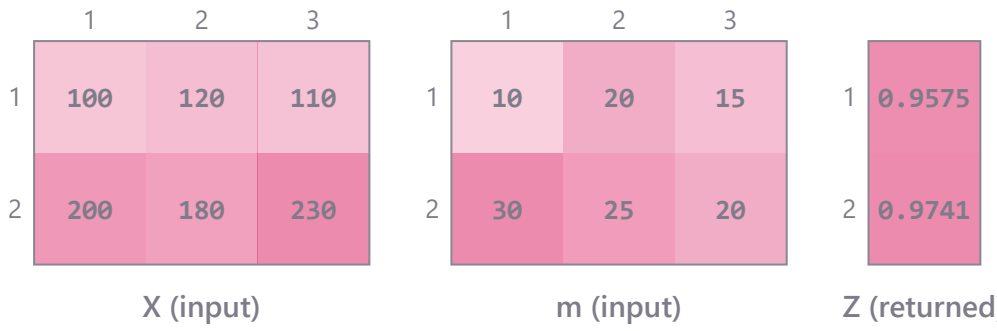
input	meaning	condition
`X`	losses per unit of exposure: one row per risk, one column per period	
`m`	exposures, the same shape as X	

output	meaning
`EPV`	estimated expected process variance
`VHM`	estimated variance of the hypothetical means
`k`	k = EPV/VHM
`Z`	credibility factor of each risk m_i/(m_i + k) (a list)
`premiums`	credibility premium per unit of exposure of each risk (a list)
`mean`	exposure-weighted overall mean

- **Method:** v = sum m_ij (X_ij - Xbar_i)^2 / (r(n-1)); a = [sum m_i (Xbar_i - Xbar)^2 - v $(r-1)$] / (m - sum m_i^2/m); $Z_i = m_i/(m_i + v/a)$
- **Assumes:** at least 2 risks and 2 periods; positive exposures; estimated VHM > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



credibility premium per unit of exposure of each risk (a list) (premiums) at each credibility factor of each risk m_i/(m_i + k) (a list) (Z), both returned by actuarial-science-ai999x for the worked example (X = [[100.0, 120.0, 110.0], [200.0, 180.0, 230.0]], m = [[10.0, 20.0, 15.0], [30.0, 25.0, 20.0]]): 0.957486 → 114.59; 0.97405 → 200.466.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): X (input); m (input); Z (returned). Returned with them by actuarial-science-ai999x: EPV = 7661.11, VHM = 3834.2, k = 1.9981, mean = 167.917.

```
actuarial_science_ai999x.solve('act_buhlmann_straub', X=[[100.0, 120.0, 110.0], [200.0, 180.0, 230.0]], m=[[10.0, 20.0, 15.0], [30.0, 25.0, 20.0]])
# -> EPV = 7661.11, VHM = 3834.2, k = 1.9981, Z = [0.957485509927, 0.974050005645], premiums = [114.590043128, 200.466171022], mean = 167.917   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_buhlmann_straub', X=[[3.0, 5.0], [8.0, 6.0], [4.0, 4.5]], m=[[50.0, 60.0], [40.0, 45.0], [80.0, 70.0]])
# -> EPV = 67.71, VHM = 1.59408, k = 42.476, Z = [0.721424823164, 0.666791972665, 0.779317780069], premiums = [4.30378603763, 6.24606986966, 4.37054010486], mean = 4.85507   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Buhlmann-Straub credibility: X = [[100, 120, 110], [200, 180, 230]], m = [[10, 20, 15], [30, 25, 20]]	SOLVED	EPV = 7661.11, VHM = 3834.2, k = 1.9981, Z = [0.957485509927, 0.974050005645], premiums = [114.590043128, 200.466171022], mean = 167.917
Buhlmann-Straub credibility: X = [[3, 5], [8, 6], [4, 4.5]], m = [[50, 60], [40, 45], [80, 70]]	SOLVED	EPV = 67.71, VHM = 1.59408, k = 42.476, Z = [0.721424823164, 0.666791972665, 0.779317780069], premiums = [4.30378603763, 6.24606986966, 4.37054010486], mean = 4.85507
Buhlmann-Straub credibility: m = [[10, 20, 15], [30, 25, 20]]	NEEDS_INPUT	act_buhlmann_straub needs X (losses per unit of exposure: one row per risk, one column per period). Example: solve('act_
Buhlmann-Straub credibility: X = [[100, 120, 110], [200, 180, 230]], m = [[10, 20, 15], [30, 25, 20]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Poisson-gamma Bayesian credibility, `act_poisson_gamma_credibility`

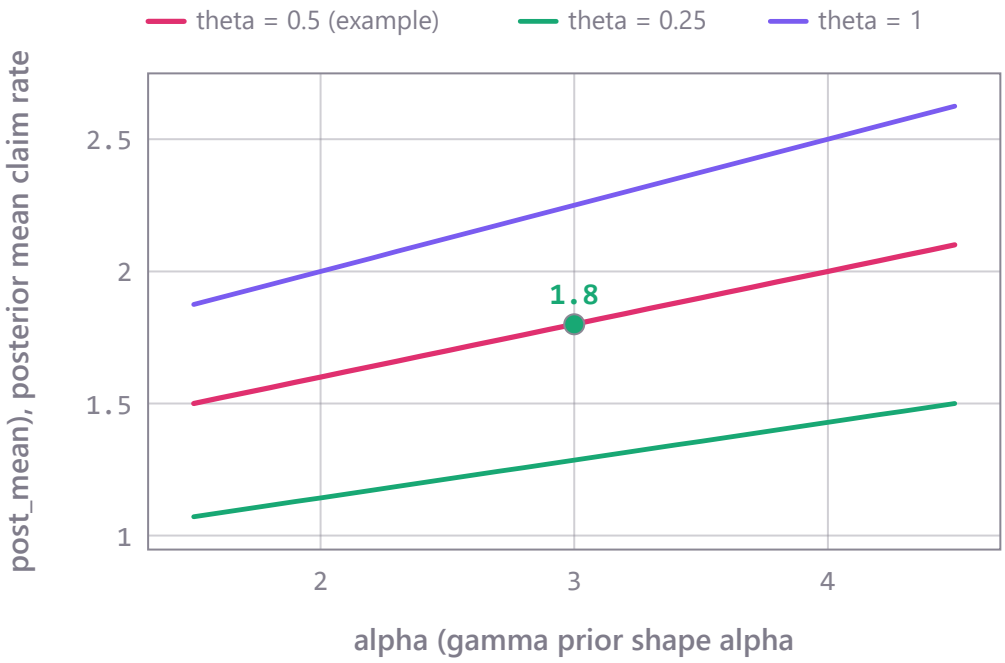
The posterior (Bayesian) mean claim frequency of a risk whose Poisson claim rate has a gamma prior (alpha, theta: mean alpha theta), after observing claim counts in n periods, with the equivalent credibility factor and the posterior gamma parameters.

$$E[\Lambda \mid x] = \frac{(\alpha + \sum x_j)\,\theta}{1 + n\theta} = Z\bar{x} + (1 - Z)\alpha\theta, \quad Z = \frac{n\theta}{1 + n\theta}$$

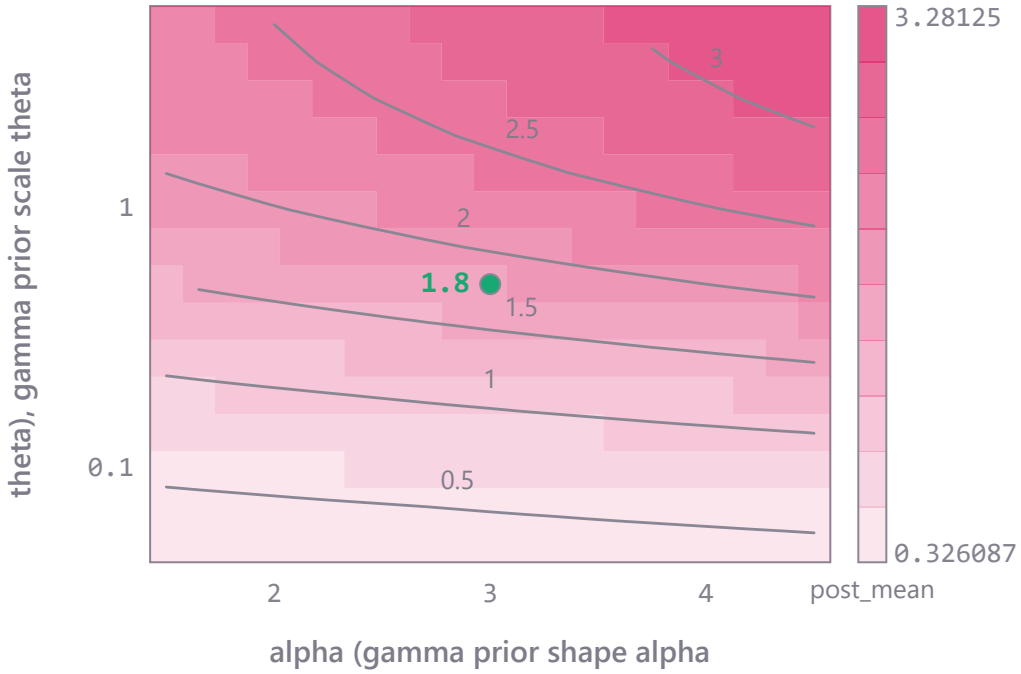
input	meaning	condition
`alpha`	gamma prior shape alpha	
`theta`	gamma prior scale theta	
`counts`	observed claim counts, one per period (a list)	

output	meaning
`post_mean`	posterior mean claim rate (alpha + sum x) theta/(1 + n theta)
`Z`	credibility factor n theta/(1 + n theta)
`prior_mean`	prior mean alpha theta
`post_alpha`	posterior shape alpha + sum x
`post_theta`	posterior scale theta/(1 + n theta)

- **Method:** posterior Gamma(alpha + sum x, theta/(1 + n theta)); its mean = Z xbar + (1 - Z) alpha theta with $Z = n$ theta/(1 + n theta)
- **Assumes:** alpha, $\theta > 0$; whole-number counts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



post_mean (posterior mean claim rate (alpha + sum x) theta/(1 + n theta)) against alpha (gamma prior shape alpha) from 1.5 to 4.5. One curve for each value of theta (gamma prior scale theta): 0.5, 0.25, 1. Other inputs as in the worked example: counts = [2.0, 1.0, 3.0]. 31 points per curve computed by actuarial-science-ai999x; marked: the worked example, alpha = 3, post_mean = 1.8.



post_mean (posterior mean claim rate (alpha + sum x) theta/(1 + n theta)) over alpha (gamma prior shape alpha) from 1.5 to 4.5 and theta (gamma prior scale theta) from 0.05 to 5 (logarithmic vertical axis): each of the 21 × 15 cells is one run of actuarial-science-ai999x, shaded by its value (light 0.326087 to dark 3.28125); iso-lines at post_mean = 0.5, 1, 1.5, 2, 2.5, 3. Other inputs as in the worked example: counts = [2.0, 1.0, 3.0]. Marked: the worked example, alpha = 3, theta = 0.5, post_mean = 1.8.

```
actuarial_science_ai999x.solve('act_poisson_gamma_credibility', alpha=3.0, theta=0.5, counts=[2.0, 1.0, 3.0])
# -> post_mean = 1.8, Z = 0.6, prior_mean = 1.5, post_alpha = 9, post_theta = 0.2   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_poisson_gamma_credibility', alpha=1.0, theta=0.2, counts=[0.0, 0.0, 1.0, 0.0])
# -> post_mean = 0.222222, Z = 0.444444, prior_mean = 0.2, post_alpha = 2, post_theta = 0.111111   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Poisson-gamma credibility: alpha = 3, theta = 0.5, counts = [2, 1, 3]	SOLVED	post_mean = 1.8, Z = 0.6, prior_mean = 1.5, post_alpha = 9, post_theta = 0.2
Poisson-gamma credibility: alpha = 1, theta = 0.2, counts = [0, 0, 1, 0]	SOLVED	post_mean = 0.222222, Z = 0.444444, prior_mean = 0.2, post_alpha = 2, post_theta = 0.111111
Poisson-gamma credibility: theta = 0.5, counts = [2, 1, 3]	NEEDS_INPUT	act_poisson_gamma_credibility needs alpha (gamma prior shape alpha). Example: solve('act_poisson_gamma_credibility', alp
Poisson-gamma credibility: alpha = 3, theta = 0.5, counts = [2, 1, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Chain-ladder age-to-age and cumulative development factors, `act_chain_ladder_factors`

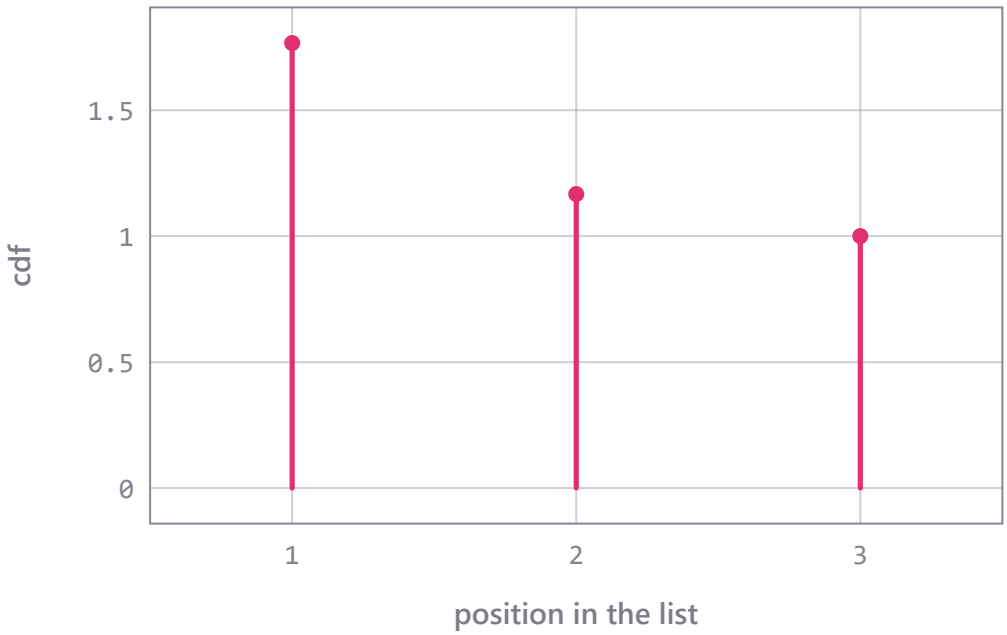
The volume-weighted age-to-age development factors of a cumulative loss triangle and the cumulative factors to ultimate from each development age (no tail factor).

$$\hat{f}_j = \frac{\sum_i C_{i,j+1}}{\sum_i C_{i,j}}, \qquad F_j = \prod_{k \geq j} \hat{f}_k$$

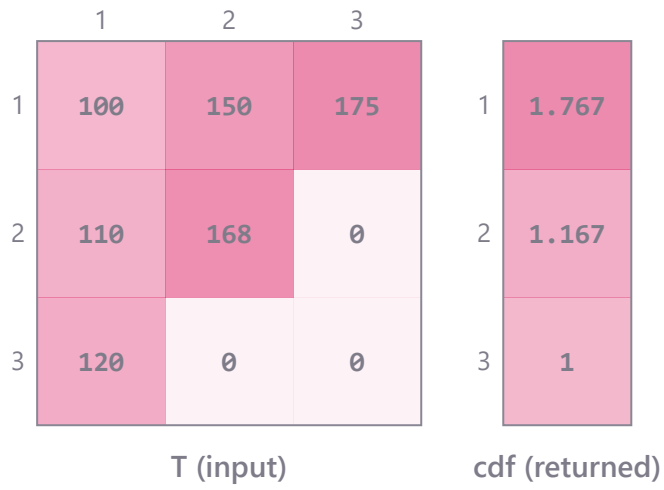
input	meaning	condition
`T`	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	

output	meaning
`ldf`	age-to-age factors f_1 ... f_(N-1) (a list)
`cdf`	cumulative development factors to ultimate from each age, last = 1 (a list)

- **Method:** f_j = sum_i C_(i,j+1) / sum_i C_(i,j) over origins with both ages known; CDF_j = product of f from age j on
- **Assumes:** square triangle with $N \geq 2$; known cells positive, unknown cells 0; no tail factor
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cumulative development factors to ultimate from each age, last = 1 (a list) (cdf) returned by actuarial-science-ai999x for the worked example, one value for each position in the list (3 values, first 1.76667, last 1, largest 1.76667). Inputs: T = [[100.0, 150.0, 175.0], [110.0, 168.0, 0.0]....



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): T (input); cdf (returned).

```
actuarial_science_ai999x.solve('act_chain_ladder_factors', T=[[100.0, 150.0, 175.0], [110.0, 168.0, 0.0], [120.0, 0.0, 0.0]])
# -> ldf = [1.51428571429, 1.16666666667], cdf = [1.76666666667, 1.16666666667, 1.0]   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_chain_ladder_factors', T=[[1000.0, 1800.0, 2100.0, 2200.0], [1100.0, 2000.0, 2300.0, 0.0], [1200.0, 2100.0, 0.0, 0.0], [1300.0, 0.0, 0.0, 0.0]])
# -> ldf = [1.78787878788, 1.15789473684, 1.04761904762], cdf = [2.16875522139, 1.21303258145, 1.04761904762, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chain-ladder development factors: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]]	SOLVED	ldf = [1.51428571429, 1.16666666667], cdf = [1.76666666667, 1.16666666667, 1.0]
Chain-ladder development factors: T = [[1000, 1800, 2100, 2200], [1100, 2000, 2300, 0], [1200, 2100, 0, 0], [1300, 0, 0, 0]]	SOLVED	ldf = [1.78787878788, 1.15789473684, 1.04761904762], cdf = [2.16875522139, 1.21303258145, 1.04761904762, 1.0]
Chain-ladder development factors:	NEEDS_INPUT	act_chain_ladder_factors needs T (cumulative loss triangle: origin years in rows, development ages in columns, unknown c
Chain-ladder development factors: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Age-to-age factors: T = [[100, 150, 175], [110, 168, 5], [120, 0, 0]]	INVALID_INPUT	Chain-ladder age-to-age and cumulative development factors: the cells below the latest diagonal must be written as 0.

Chain-ladder ultimate losses and reserves, `act_chain_ladder_reserve`

The ultimate losses, the reserve (ultimate less reported to date) of each origin year and the total reserve of a cumulative loss triangle by the volume-weighted chain-ladder method (no tail factor).

$$\hat{C}_{i,N} = C_{i,N-i} \prod_{k \geq N-i} \hat{f}_k, \qquad R_i = \hat{C}_{i,N} - C_{i,N-i}$$

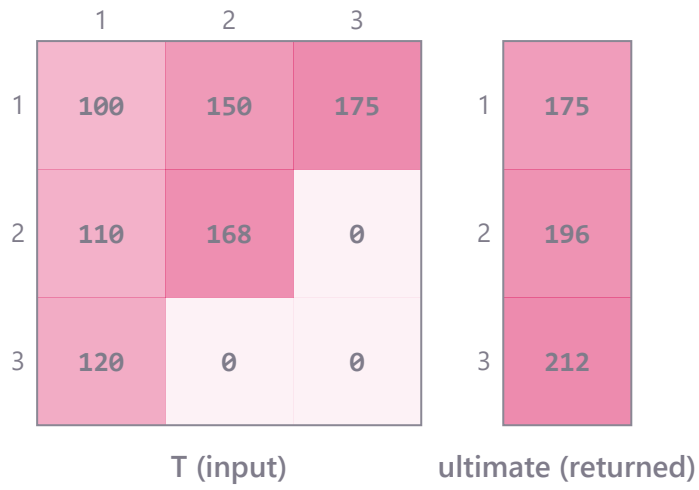
input	meaning	condition
`T`	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	

output	meaning
`ultimate`	ultimate losses of each origin year (a list)
`reserve`	reserve of each origin year (a list)
`total_reserve`	total reserve

- **Method:** ultimate_i = latest_i x CDF at its age; reserve_i = ultimate_i - latest_i
- **Assumes:** square triangle with $N \geq 2$; known cells positive, unknown cells 0; no tail factor
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ultimate losses of each origin year (a list) (ultimate) returned by actuarial-science-ai999x for the worked example, one value for each position in the list (3 values, first 175, last 212, largest 212); with reserve (reserve of each origin year (a list)). Inputs: T = [[100.0, 150.0, 175.0], [110.0, 168.0, 0.0]....



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): T (input); ultimate (returned). Returned with them by actuarial-science-ai999x: total_reserve = 120.

```
actuarial_science_ai999x.solve('act_chain_ladder_reserve', T=[[100.0, 150.0, 175.0], [110.0, 168.0,
0.0], [120.0, 0.0, 0.0]])
# -> ultimate = [175.0, 196.0, 212.0], reserve = [0.0, 28.0, 92.0], total_reserve = 120   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_chain_ladder_reserve', T=[[1000.0, 1800.0, 2100.0, 2200.0],
[1100.0, 2000.0, 2300.0, 0.0], [1200.0, 2100.0, 0.0, 0.0], [1300.0, 0.0, 0.0, 0.0]])
# -> ultimate = [2200.0, 2409.52380952, 2547.36842105, 2819.3817878], reserve = [0.0,
109.523809524, 447.368421053, 1519.3817878], total_reserve = 2076.27   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chain-ladder reserve: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]]	SOLVED	ultimate = [175.0, 196.0, 212.0], reserve = [0.0, 28.0, 92.0], total_reserve = 120
Chain-ladder reserve: T = [[1000, 1800, 2100, 2200], [1100, 2000, 2300, 0], [1200, 2100, 0, 0], [1300, 0, 0, 0]]	SOLVED	ultimate = [2200.0, 2409.52380952, 2547.36842105, 2819.3817878], reserve = [0.0, 109.523809524, 447.368421053, 1519.3817878], total_reserve = 2076.27
Chain-ladder reserve:	NEEDS_INPUT	act_chain_ladder_reserve needs T (cumulative loss triangle: origin years in rows, development ages in columns, unknown c
Chain-ladder reserve: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Chain-ladder with simple-average development factors, `act_chain_ladder_simple_average`

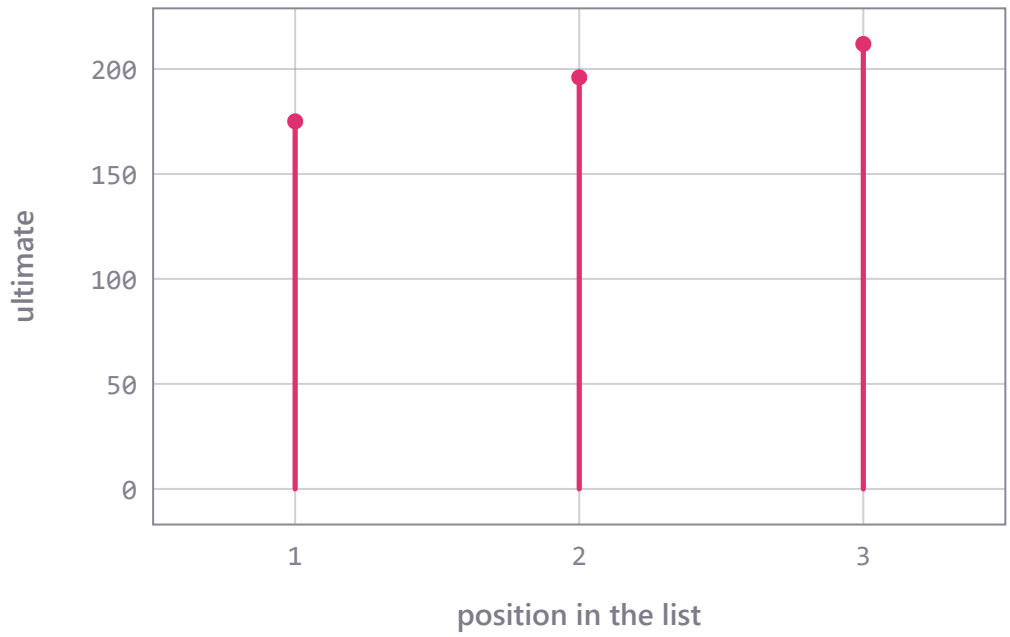
The simple-average age-to-age factors (mean of the individual link ratios), the ultimate losses of each origin year and the total reserve of a cumulative loss triangle (no tail factor).

$$\hat{f}_j = \frac{1}{n_j} \sum_i \frac{C_{i,j+1}}{C_{i,j}}$$

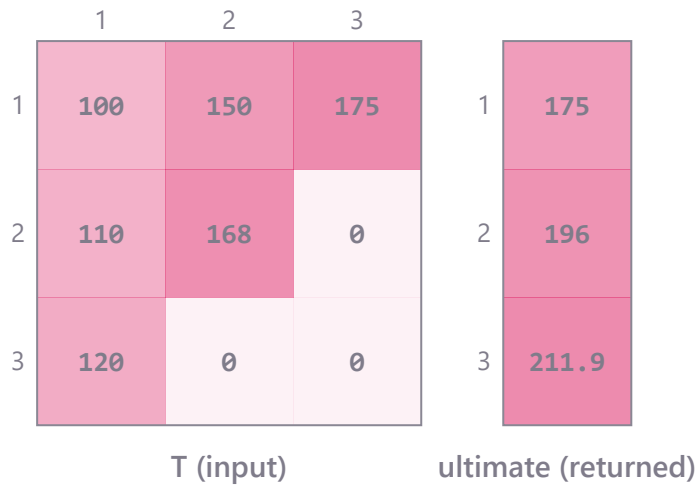
input	meaning	condition
`T`	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	

output	meaning
`ldf`	simple-average age-to-age factors (a list)
`ultimate`	ultimate losses of each origin year (a list)
`total_reserve`	total reserve

- **Method:** f_j = mean over origins of C_(i,j+1)/C_(i,j); ultimate_i = latest_i x product of f
- **Assumes:** square triangle with $N \geq 2$; known cells positive, unknown cells 0; no tail factor
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ultimate losses of each origin year (a list) (ultimate) returned by actuarial-science-ai999x for the worked example, one value for each position in the list (3 values, first 175, last 211.909, largest 211.909). Inputs: T = [[100.0, 150.0, 175.0], [110.0, 168.0, 0.0]....



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): T (input); ultimate (returned). Returned with them by actuarial-science-ai999x: total_reserve = 119.909.

```
actuarial_science_ai999x.solve('act_chain_ladder_simple_average', T=[[100.0, 150.0, 175.0], [110.0,
168.0, 0.0], [120.0, 0.0, 0.0]])
# -> ldf = [1.51363636364, 1.16666666667], ultimate = [175.0, 196.0, 211.909090909], total_reserve
= 119.909   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_chain_ladder_simple_average', T=[[1000.0, 1800.0, 2100.0,
2200.0], [1100.0, 2000.0, 2300.0, 0.0], [1200.0, 2100.0, 0.0, 0.0], [1300.0, 0.0, 0.0, 0.0]])
# -> ldf = [1.78939393939, 1.15833333333, 1.04761904762], ultimate = [2200.0, 2409.52380952,
2548.33333333, 2822.83994709], total_reserve = 2080.7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Simple average chain-ladder: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]]	SOLVED	ldf = [1.51363636364, 1.16666666667], ultimate = [175.0, 196.0, 211.909090909], total_reserve = 119.909
Simple average chain-ladder: T = [[1000, 1800, 2100, 2200], [1100, 2000, 2300, 0], [1200, 2100, 0, 0], [1300, 0, 0, 0]]	SOLVED	ldf = [1.78939393939, 1.15833333333, 1.04761904762], ultimate = [2200.0, 2409.52380952, 2548.33333333, 2822.83994709], total_reserve = 2080.7
Simple average chain-ladder:	NEEDS_INPUT	act_chain_ladder_simple_average needs T (cumulative loss triangle: origin years in rows, development ages in columns, un
Simple average chain-ladder: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Bornhuetter-Ferguson reserve, `act_bornhuetter_ferguson`

The Bornhuetter-Ferguson reserve and ultimate losses of each origin year from a cumulative loss triangle, the earned premiums and an expected loss ratio: the unreported part (1 - 1/CDF) of the expected losses, with chain-ladder volume-weighted factors.

$$R_i^{BF} = P_i \cdot \mathrm{ELR}\left(1 - \frac{1}{F_{N-i}}\right), \qquad \hat{U}_i = C_{i,N-i} + R_i^{BF}$$

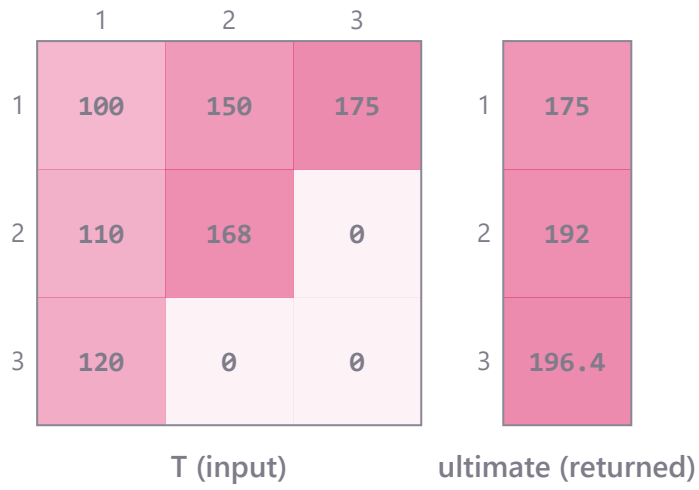
input	meaning	condition
`T`	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	
`P`	earned premium of each origin year (a list)	
`elr`	expected (a priori) loss ratio (70% or 0.7)	

output	meaning
`ultimate`	ultimate losses of each origin year (a list)
`reserve`	reserve of each origin year (a list)
`total_reserve`	total reserve

- **Method:** reserve_i = P_i ELR (1 - 1/CDF_i); ultimate_i = latest_i + reserve_i
- **Assumes:** square triangle; one positive premium per origin year; 0 < ELR <= 500%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ultimate losses of each origin year (a list) (ultimate) returned by actuarial-science-ai999x for the worked example, one value for each P (earned premium of each origin year (a list)) (3 values, first 175, last 196.377, largest 196.377); with reserve (reserve of each origin year (a list)). Inputs: T = [[100.0, 150.0, 175.0], [110.0, 168.0, 0.0]..., P = [200.0, 210.0, 220.0], elr = 0.8.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): T (input); ultimate (returned). Returned with them by actuarial-science-ai999x: total_reserve = 100.377. Other inputs: P = [200.0, 210.0, 220.0], elr = 0.8.

```
actuarial_science_ai999x.solve('act_bornhuetter_ferguson', T=[[100.0, 150.0, 175.0], [110.0, 168.0,
0.0], [120.0, 0.0, 0.0]], P=[200.0, 210.0, 220.0], elr=0.8)
# -> ultimate = [175.0, 192.0, 196.377358491], reserve = [0.0, 24.0, 76.3773584906], total_reserve
= 100.377   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_bornhuetter_ferguson', T=[[1000.0, 1800.0, 2100.0, 2200.0],
[1100.0, 2000.0, 2300.0, 0.0], [1200.0, 2100.0, 0.0, 0.0], [1300.0, 0.0, 0.0, 0.0]], P=[3000.0,
3200.0, 3400.0, 3600.0], elr=0.7)
# -> ultimate = [2200.0, 2401.81818182, 2517.97520661, 2658.0431433], reserve = [0.0,
101.818181818, 417.975206612, 1358.0431433], total_reserve = 1877.84   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bornhuetter-Ferguson reserve: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]], P = [200, 210, 220], elr = 0.8	SOLVED	ultimate = [175.0, 192.0, 196.377358491], reserve = [0.0, 24.0, 76.3773584906], total_reserve = 100.377
Bornhuetter-Ferguson reserve: T = [[1000, 1800, 2100, 2200], [1100, 2000, 2300, 0], [1200, 2100, 0, 0], [1300, 0, 0, 0]], P = [3000, 3200, 3400, 3600], elr = 0.7	SOLVED	ultimate = [2200.0, 2401.81818182, 2517.97520661, 2658.0431433], reserve = [0.0, 101.818181818, 417.975206612, 1358.0431433], total_reserve = 1877.84
Bornhuetter-Ferguson reserve: P = [200, 210, 220], elr = 0.8	NEEDS_INPUT	act_bornhuetter_ferguson needs T (cumulative loss triangle: origin years in rows, development ages in columns, unknown c
Bornhuetter-Ferguson reserve: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]], P = [200, 210, 220], elr = 0.8, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Cape Cod (Stanard-Buhlmann) reserve, `act_cape_cod`

The Cape Cod expected loss ratio (reported losses over used-up premium), and the resulting reserves and ultimate losses of each origin year, from a cumulative loss triangle and the earned premiums.

$$\mathrm{ELR}_{CC} = \frac{\sum_i C_{i,N-i}}{\sum_i P_i / F_{N-i}}, \qquad R_i = P_i \, \mathrm{ELR}_{CC} \left(1 - \frac{1}{F_{N-i}}\right)$$

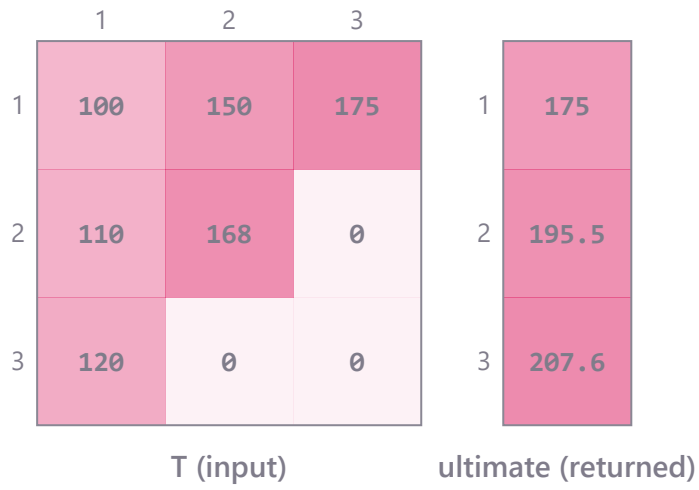
input	meaning	condition
`T`	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	
`P`	earned premium of each origin year (a list)	

output	meaning
`elr`	Cape Cod expected loss ratio
`ultimate`	ultimate losses of each origin year (a list)
`reserve`	reserve of each origin year (a list)
`total_reserve`	total reserve

- **Method:** ELR = sum latest_i / sum (P_i / CDF_i); reserve_i = P_i ELR (1 - 1/CDF_i)
- **Assumes:** square triangle; one positive premium per origin year; no trend, no decay
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ultimate losses of each origin year (a list) (ultimate) returned by actuarial-science-ai999x for the worked example, one value for each P (earned premium of each origin year (a list)) (3 values, first 175, last 207.613, largest 207.613); with reserve (reserve of each origin year (a list)). Inputs: T = [[100.0, 150.0, 175.0], [110.0, 168.0, 0.0]..., P = [200.0, 210.0, 220.0].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): T (input); ultimate (returned). Returned with them by actuarial-science-ai999x: elr = 0.917689, total_reserve = 115.144. Other inputs: P = [200.0, 210.0, 220.0].

```
actuarial_science_ai999x.solve('act_cape_cod', T=[[100.0, 150.0, 175.0], [110.0, 168.0, 0.0],
[120.0, 0.0, 0.0]], P=[200.0, 210.0, 220.0])
# -> elr = 0.917689, ultimate = [175.0, 195.530665669, 207.613313388], reserve = [0.0,
27.5306656694, 87.6133133882], total_reserve = 115.144   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_cape_cod', T=[[1000.0, 1800.0, 2100.0, 2200.0], [1100.0,
2000.0, 2300.0, 0.0], [1200.0, 2100.0, 0.0, 0.0], [1300.0, 0.0, 0.0, 0.0]], P=[3000.0, 3200.0,
3400.0, 3600.0])
# -> elr = 0.751138, ultimate = [2200.0, 2409.25642168, 2548.5100265, 2757.25381925], reserve =
[0.0, 109.25642168, 448.510026497, 1457.25381925], total_reserve = 2015.02   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cape Cod reserve: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]], P = [200, 210, 220]	SOLVED	elr = 0.917689, ultimate = [175.0, 195.530665669, 207.613313388], reserve = [0.0, 27.5306656694, 87.6133133882], total_reserve = 115.144
Cape Cod reserve: T = [[1000, 1800, 2100, 2200], [1100, 2000, 2300, 0], [1200, 2100, 0, 0], [1300, 0, 0, 0]], P = [3000, 3200, 3400, 3600]	SOLVED	elr = 0.751138, ultimate = [2200.0, 2409.25642168, 2548.5100265, 2757.25381925], reserve = [0.0, 109.25642168, 448.510026497, 1457.25381925], total_reserve = 2015.02
Cape Cod reserve: P = [200, 210, 220]	NEEDS_INPUT	act_cape_cod needs T (cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written
Cape Cod reserve: T = [[100, 150, 175], [110, 168, 0], [120, 0, 0]], P = [200, 210, 220], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Mack's standard error of chain-ladder reserves, `act_mack_standard_error`

The standard errors (root mean squared errors of prediction) of the chain-ladder reserve of each origin year and of the total reserve by Mack's distribution-free model, with the variance parameters σ^2 of each

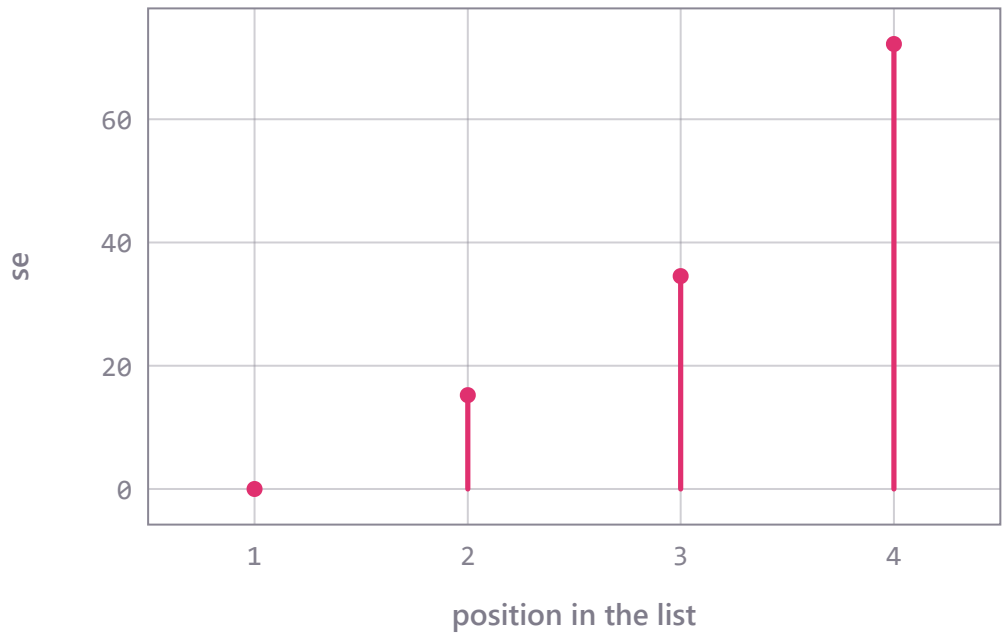
development age.

$$\widehat{\mathrm{mse}}(\hat{R}_i) = \hat{C}_{iN}^2 \sum_{k=N-i}^{N-1} \frac{\hat{\sigma}_k^2}{\hat{f}_k^2} \left(\frac{1}{\hat{C}_{ik}} + \frac{1}{\sum_j C_{jk}} \right)$$

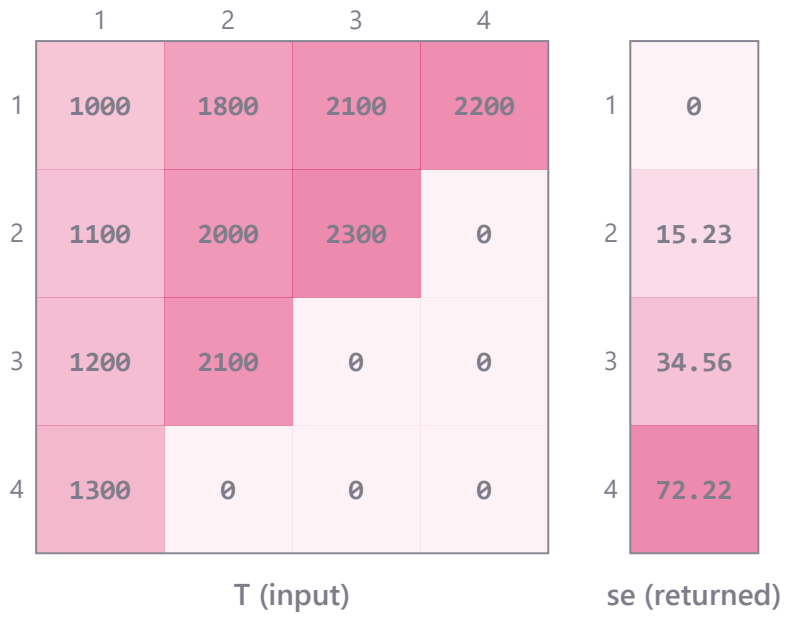
input	meaning	condition
`T`	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	

output	meaning
`se`	standard error of each origin year's reserve (a list)
`total_se`	standard error of the total reserve
`sigma2`	Mack variance parameters σ^2 of each age (a list)

- **Method:** sigma_k^2 = sum C_ik (C_(i,k+1)/C_ik - f_k)^2/(n_k - 1); mse(R_i) = C_iN^2 sum_k $sigma_k^2/f_k^2 (1/C_ik + 1/S_k)$; total adds the covariance terms
- **Assumes:** square triangle with $N \geq 4$; known cells positive, unknown cells 0; last σ^2 by Mack's extrapolation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



standard error of each origin year's reserve (a list) (se) returned by actuarial-science-ai999x for the worked example, one value for each position in the list (4 values, first 0, last 72.2173, largest 72.2173). Inputs: T = [[1000.0, 1800.0, 2100.0, 2200.0], [1100.0, 2....



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): T (input); se (returned). Returned with them by actuarial-science-ai999x: total_se = 90.6808.

```
actuarial_science_ai999x.solve('act_mack_standard_error', T=[[1000.0, 1800.0, 2100.0, 2200.0],
[1100.0, 2000.0, 2300.0, 0.0], [1200.0, 2100.0, 0.0, 0.0], [1300.0, 0.0, 0.0, 0.0]])
# -> se = [0.0, 15.2267485994, 34.5563923039, 72.2173151114], total_se = 90.6808, sigma2 =
[1.43939393939, 0.263157894737, 0.0481119696749]   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_mack_standard_error', T=[[500.0, 900.0, 1100.0, 1150.0,
1160.0], [550.0, 1000.0, 1250.0, 1300.0, 0.0], [600.0, 1050.0, 1350.0, 0.0, 0.0], [620.0, 1200.0,
0.0, 0.0, 0.0], [700.0, 0.0, 0.0, 0.0, 0.0]])
# -> se = [0.0, 0.920569091089, 6.21333668409, 43.5180914566, 89.6552167478], total_se = 105.917,
sigma2 = [3.75268601892, 0.990382028518, 0.0174081237911, 0.000305985736009]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mack standard error: T = [[1000, 1800, 2100, 2200], [1100, 2000, 2300, 0], [1200, 2100, 0, 0], [1300, 0, 0, 0]]	SOLVED	se = [0.0, 15.2267485994, 34.5563923039, 72.2173151114], total_se = 90.6808, sigma2 = [1.43939393939, 0.263157894737, 0.0481119696749]
Mack standard error: T = [[500, 900, 1100, 1150, 1160], [550, 1000, 1250, 1300, 0], [600, 1050, 1350, 0, 0], [620, 1200, 0, 0, 0], [700, 0, 0, 0, 0]]	SOLVED	se = [0.0, 0.920569091089, 6.21333668409, 43.5180914566, 89.6552167478], total_se = 105.917, sigma2 = [3.75268601892, 0.990382028518, 0.0174081237911, 0.000305985736009]
Mack standard error:	NEEDS_INPUT	act_mack_standard_error needs T (cumulative loss triangle: origin years in rows, development ages in columns, unknown ce
Mack standard error: T = [[1000, 1800, 2100, 2200], [1100, 2000, 2300, 0], [1200, 2100, 0, 0], [1300, 0, 0, 0]], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Expected claims reserving method, `act_expected_claims_method`

The ultimate losses (premium x expected loss ratio), the reserve (ultimate less paid or reported to date) of each origin year and the total reserve by the expected claims method.

$$\hat{U}_i = P_i \cdot \mathrm{ELR}, \qquad R_i = \hat{U}_i - C_i$$

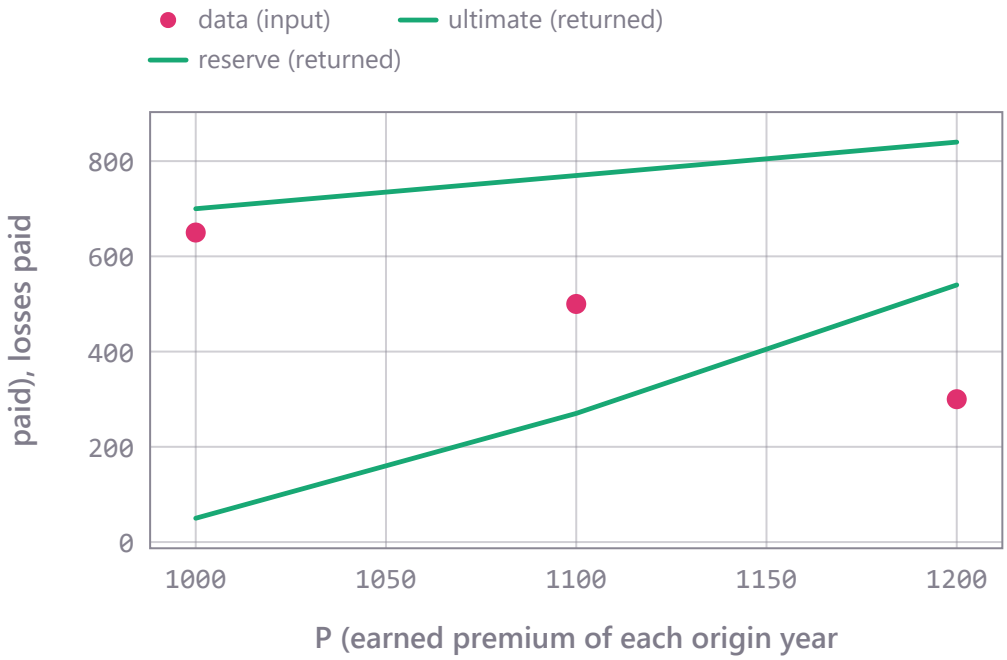
input	meaning	condition
`P`	earned premium of each origin year (a list)	
`elr`	expected loss ratio (65% or 0.65)	
`paid`	losses paid (or reported) to date of each origin year (a list)	

output	meaning
`ultimate`	expected ultimate losses (a list)
`reserve`	reserve of each origin year (a list)
`total_reserve`	total reserve

- **Method:** ultimate_i = P_i x ELR; reserve_i = ultimate_i - paid_i
- **Assumes:** one paid amount per premium; 0 < ELR <= 500%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



expected ultimate losses (a list) (ultimate) returned by actuarial-science-ai999x for the worked example, one value for each P (earned premium of each origin year (a list)) (3 values, first 700, last 840, largest 840); with reserve (reserve of each origin year (a list)). Inputs: P = [1000.0, 1100.0, 1200.0], elr = 0.7, paid = [650.0, 500.0, 300.0].



The data paid (losses paid (or reported) to date of each origin year (a list)) against P (earned premium of each origin year (a list)) as given in the worked example (3 points); drawn on it from what actuarial-science-ai999x returned: the returned ultimate; the returned reserve. Also returned: total_reserve = 860. Other inputs: elr = 0.7.

```
actuarial_science_ai999x.solve('act_expected_claims_method', P=[1000.0, 1100.0, 1200.0], elr=0.7,
paid=[650.0, 500.0, 300.0])
# -> ultimate = [700.0, 770.0, 840.0], reserve = [50.0, 270.0, 540.0], total_reserve = 860
(SOLVED)
```

```
actuarial_science_ai999x.solve('act_expected_claims_method', P=[5000.0], elr=0.65, paid=[1000.0])
# -> ultimate = [3250.0], reserve = [2250.0], total_reserve = 2250   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Expected claims method: P = [1000, 1100, 1200], elr = 0.7, paid = [650, 500, 300]	SOLVED	ultimate = [700.0, 770.0, 840.0], reserve = [50.0, 270.0, 540.0], total_reserve = 860
Expected claims method: P = [5000], elr = 0.65, paid = [1000]	SOLVED	ultimate = [3250.0], reserve = [2250.0], total_reserve = 2250
Expected claims method: elr = 0.7, paid = [650, 500, 300]	NEEDS_INPUT	act_expected_claims_method needs P (earned premium of each origin year (a list)). Example: solve('act_expected_claims_me
Expected claims method: P = [1000, 1100, 1200], elr = 0.7, paid = [650, 500, 300], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Indicated rate change by the loss ratio method, `act_loss_ratio_rate_change`

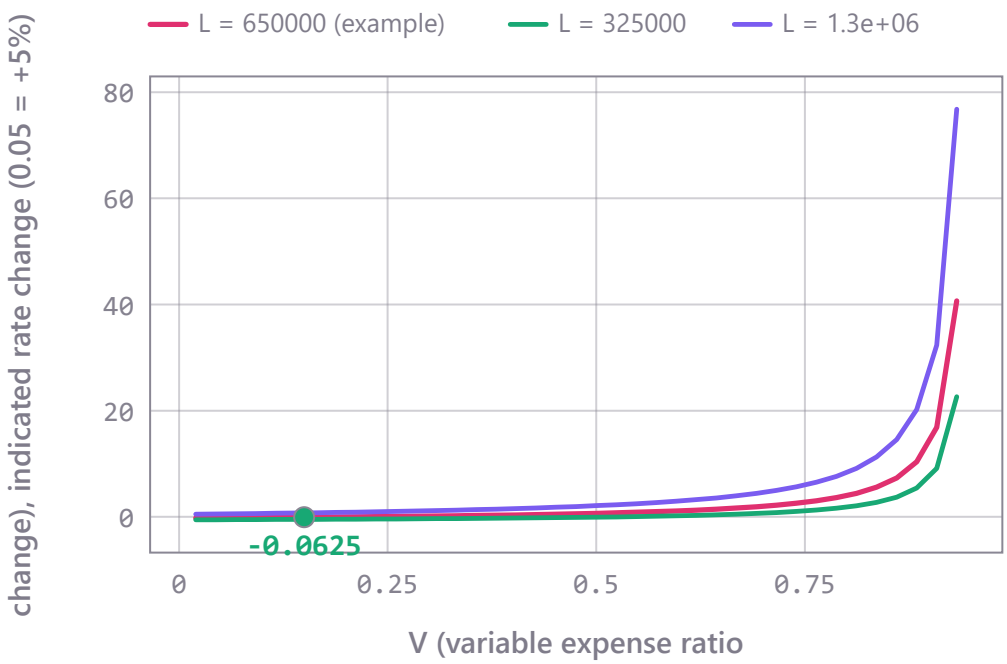
The indicated rate change from projected losses (with loss adjustment expenses), earned premium at current rates, the fixed expense ratio, the variable expense ratio and the profit provision.

$$\Delta = \frac{L/EP + F}{1 - V - Q} - 1$$

input	meaning	condition
`L`	projected ultimate losses and LAE	
`EP`	earned premium at current rates	
`F`	fixed expense ratio (to premium)	
`V`	variable expense ratio	
`Q`	profit and contingencies provision	

output	meaning
`change`	indicated rate change (0.05 = +5%)
`loss_ratio`	projected loss ratio L/EP
`permissible`	permissible loss ratio $1 - V - Q$

- **Method:** indicated change = (L/EP + F)/(1 - V - Q) - 1
- **Assumes:** $EP > 0$; $0 \leq F$, $V < 1$; $V + Q < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



change (indicated rate change (0.05 = +5%)) against V (variable expense ratio) from 0.02 to 0.98. One curve for each value of L (projected ultimate losses and LAE): 650000, 325000, 1.3e+06. Other inputs as in the worked example: EP = 1000000.0, F = 0.1, Q = 0.05. 39 points per curve computed by actuarial-science-ai999x; marked: the worked example, V = 0.15, change = -0.0625.

```
actuarial_science_ai999x.solve('act_loss_ratio_rate_change', L=650000.0, EP=1000000.0, F=0.1,
V=0.15, Q=0.05)
# -> change = -0.0625, loss_ratio = 0.65, permissible = 0.8   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_loss_ratio_rate_change', L=75000.0, EP=100000.0, F=0.05, V=0.2, Q=0.03)
# -> change = 0.038961, loss_ratio = 0.75, permissible = 0.77   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Loss ratio rate change: L = 650000, EP = 1000000, F = 0.1, V = 0.15, Q = 0.05	SOLVED	change = -0.0625, loss_ratio = 0.65, permissible = 0.8
Loss ratio rate change: L = 75000, EP = 100000, F = 0.05, V = 0.2, Q = 0.03	SOLVED	change = 0.038961, loss_ratio = 0.75, permissible = 0.77
Loss ratio rate change: EP = 1000000, F = 0.1, V = 0.15, Q = 0.05	NEEDS_INPUT	act_loss_ratio_rate_change needs L (projected ultimate losses and LAE). Example: solve('act_loss_ratio_rate_change', L=6
Loss ratio rate change: L = 650000, EP = 1000000, F = 0.1, V = 0.15, Q = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Indicated rate by the pure premium method, `act_pure_premium_rate`

The pure premium (losses per exposure) and the indicated rate per exposure from projected losses, exposures, the fixed expense per exposure, the variable expense ratio and the profit provision.

$$R = \frac{L/X + F}{1 - V - Q}$$

input	meaning	condition
`L`	projected ultimate losses and LAE	
`X`	earned exposures	
`Fe`	fixed expense per exposure	
`V`	variable expense ratio	
`Q`	profit and contingencies provision	

output	meaning
`pure_premium`	pure premium L/X
`rate`	indicated rate per exposure $(PP + F)/(1 - V - Q)$

- **Method:** rate = (L/X + F)/(1 - V - Q)
- **Assumes:** $X > 0$; $0 \leq V < 1$; $V + Q < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rate (indicated rate per exposure (PP + F)/(1 - V - Q)) against V (variable expense ratio) from 0.02 to 0.98. One curve for each value of L (projected ultimate losses and LAE): 500000, 250000, 1e+06. Other inputs as in the worked example: X = 2000.0, Fe = 30.0, Q = 0.05. 39 points per curve computed by actuarial-science-ai999x; marked: the worked example, V = 0.15, rate = 350.

```
actuarial_science_ai999x.solve('act_pure_premium_rate', L=500000.0, X=2000.0, Fe=30.0, V=0.15,
Q=0.05)
# -> pure_premium = 250, rate = 350   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_pure_premium_rate', L=90000.0, X=600.0, Fe=20.0, V=0.2, Q=0.0)
# -> pure_premium = 150, rate = 212.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pure premium rate: L = 500000, X = 2000, Fe = 30, V = 0.15, Q = 0.05	SOLVED	pure_premium = 250, rate = 350
Pure premium rate: L = 90000, X = 600, Fe = 20, V = 0.2, Q = 0	SOLVED	pure_premium = 150, rate = 212.5
Pure premium rate: X = 2000, Fe = 30, V = 0.15, Q = 0.05	NEEDS_INPUT	act_pure_premium_rate needs L (projected ultimate losses and LAE). Example: solve('act_pure_premium_rate', L=500000, X=2
Pure premium rate: L = 500000, X = 2000, Fe = 30, V = 0.15, Q = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Kaplan-Meier product-limit survival estimate, `act_kaplan_meier`

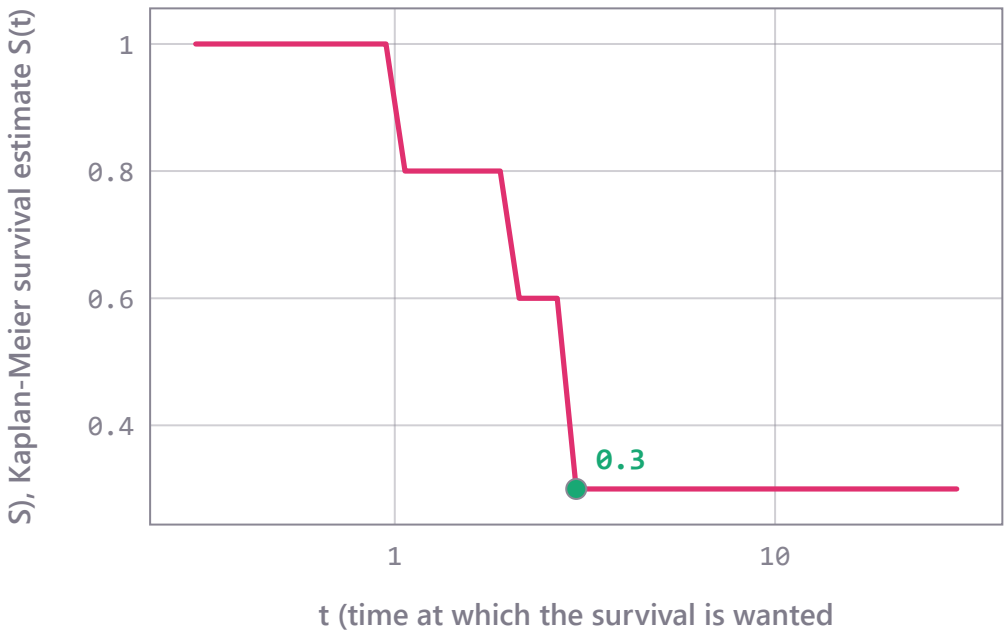
The Kaplan-Meier estimate of the survival function at time t from observed times with death (1) or censoring (0) indicators; a life censored at a death time is counted at risk then.

$$\hat{S}(t) = \prod_{t_j \le t} \left(1 - \frac{d_j}{n_j}\right)$$

input	meaning	condition
`times`	observed times (a list)	
`events`	1 for a death or failure, 0 for censoring (a list)	
`t`	time at which the survival is wanted	

output	meaning
`S`	Kaplan-Meier survival estimate S(t)
`F`	estimated probability of death by t
`deaths`	number of deaths observed up to t

- **Method:** S(t) = product over death times $t_j \le t$ of $(1 - d_j/n_j)$
- **Assumes:** one indicator per time; positive times; indicators 0 or 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



S (Kaplan-Meier survival estimate S(t)) against t (time at which the survival is wanted) from 0.3 to 30 on a logarithmic axis. Other inputs as in the worked example: times = [1.0, 2.0, 2.0, 3.0, 5.0], events = [1.0, 1.0, 0.0, 1.0, 0.0]. 41 points computed by actuarial-science-ai999x; marked: the worked example, t = 3, S = 0.3.

```
actuarial_science_ai999x.solve('act_kaplan_meier', times=[1.0, 2.0, 2.0, 3.0, 5.0], events=[1.0,
1.0, 0.0, 1.0, 0.0], t=3.0)
# -> S = 0.3, F = 0.7, deaths = 3   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_kaplan_meier', times=[2.0, 3.0, 3.0, 5.0, 6.0, 8.0, 9.0],
events=[1.0, 1.0, 1.0, 0.0, 1.0, 0.0, 1.0], t=6.0)
# -> S = 0.380952, F = 0.619048, deaths = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kaplan-Meier estimate: times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3	SOLVED	S = 0.3, F = 0.7, deaths = 3
Kaplan-Meier estimate: times = [2, 3, 3, 5, 6, 8, 9], events = [1, 1, 1, 0, 1, 0, 1], t = 6	SOLVED	S = 0.380952, F = 0.619048, deaths = 4
Kaplan-Meier estimate: events = [1, 1, 0, 1, 0], t = 3	NEEDS_INPUT	act_kaplan_meier needs times (observed times (a list)). Example: solve('act_kaplan_meier', times=[1.0, 2.0, 2.0, 3.0, 5.
Kaplan-Meier estimate: times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template
Kaplan-Meier estimate: times = [1, 2, 3], events = [1, 2, 0], t = 2	INVALID_INPUT	Kaplan-Meier product-limit survival estimate: every event indicator must be 1 (death or failure) or 0 (censored).

Nelson-Aalen cumulative hazard estimate, `act_nelson_aalen`

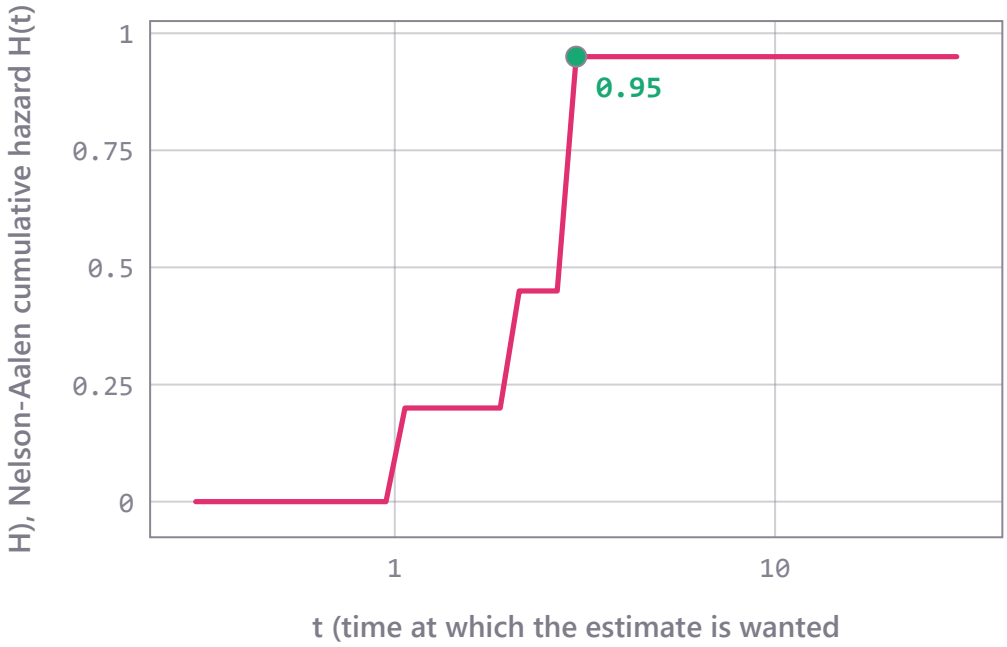
The Nelson-Aalen estimate of the cumulative hazard at time t from observed times with death (1) or censoring (0) indicators, and the survival estimate exp(-H(t)).

$$\hat{H}(t) = \sum_{t_j \le t} \frac{d_j}{n_j}, \quad \hat{S}(t) = e^{-\hat{H}(t)}$$

input	meaning	condition
`times`	observed times (a list)	
`events`	1 for a death or failure, 0 for censoring (a list)	
`t`	time at which the estimate is wanted	

output	meaning
`H`	Nelson-Aalen cumulative hazard H(t)
`S`	survival estimate exp(-H(t))

- **Method:** H(t) = sum over death times $t_j \le t$ of d_j/n_j
- **Assumes:** one indicator per time; positive times; indicators 0 or 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



H (Nelson-Aalen cumulative hazard H(t)) against t (time at which the estimate is wanted) from 0.3 to 30 on a logarithmic axis. Other inputs as in the worked example: times = [1.0, 2.0, 2.0, 3.0, 5.0], events = [1.0, 1.0, 0.0, 1.0, 0.0]. 41 points computed by actuarial-science-ai999x; marked: the worked example, t = 3, H = 0.95.

```
actuarial_science_ai999x.solve('act_nelson_aalen', times=[1.0, 2.0, 2.0, 3.0, 5.0], events=[1.0,
1.0, 0.0, 1.0, 0.0], t=3.0)
# -> H = 0.95, S = 0.386741   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_nelson_aalen', times=[2.0, 3.0, 3.0, 5.0, 6.0, 8.0, 9.0],
events=[1.0, 1.0, 1.0, 0.0, 1.0, 0.0, 1.0], t=6.0)
# -> H = 0.809524, S = 0.44507   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nelson-Aalen estimate: times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3	SOLVED	H = 0.95, S = 0.386741
Nelson-Aalen estimate: times = [2, 3, 3, 5, 6, 8, 9], events = [1, 1, 1, 0, 1, 0, 1], t = 6	SOLVED	H = 0.809524, S = 0.44507
Nelson-Aalen estimate: events = [1, 1, 0, 1, 0], t = 3	NEEDS_INPUT	act_nelson_aalen needs times (observed times (a list)). Example: solve('act_nelson_aalen', times=[1.0, 2.0, 2.0, 3.0, 5.
Nelson-Aalen estimate: times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template

Exponential maximum-likelihood fit with censoring, `act_exponential_mle`

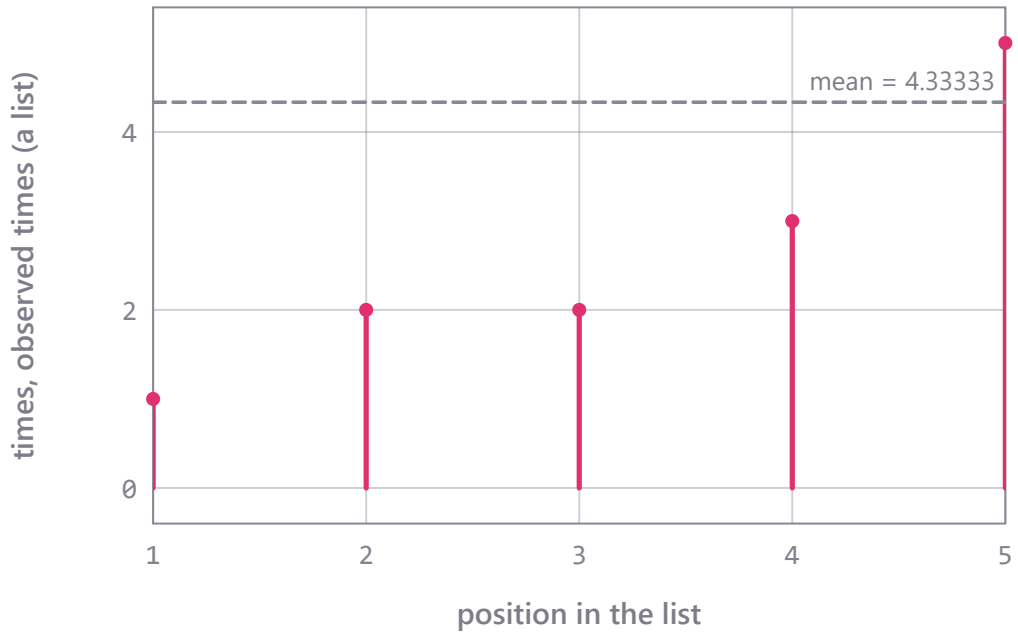
The maximum-likelihood constant hazard (deaths divided by total time observed), the mean lifetime and the fitted survival probability at time t from times with death (1) or censoring (0) indicators.

$$\hat{\lambda} = \frac{\sum_i \delta_i}{\sum_i t_i}, \quad \hat{S}(t) = e^{-\hat{\lambda} t}$$

input	meaning	condition
`times`	observed times (a list)	
`events`	1 for a death or failure, 0 for censoring (a list)	
`t`	time for the fitted survival probability	

output	meaning
`hazard`	maximum-likelihood hazard rate (deaths / total exposure)
`mean`	fitted mean lifetime
`S`	fitted survival probability exp(-lambda t)

- **Method:** lambda = sum of death indicators / sum of times; S(t) = e(− λ t)
- **Assumes:** one indicator per time; positive times; at least one death
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The input times (observed times (a list)) as given in the worked example, one value per position (5 values); marked from what actuarial-science-ai999x returned: mean = 4.33333. Also returned: hazard = 0.230769, S = 0.50042. Other inputs: events = [1.0, 1.0, 0.0, 1.0, 0.0], t = 3.0.

```
actuarial_science_ai999x.solve('act_exponential_mle', times=[1.0, 2.0, 2.0, 3.0, 5.0], events=[1.0, 1.0, 0.0, 1.0, 0.0], t=3.0)
# -> hazard = 0.230769, mean = 4.33333, S = 0.50042   (SOLVED)
```

```
actuarial_science_ai999x.solve('act_exponential_mle', times=[10.0, 4.0, 7.5, 12.0], events=[1.0, 1.0, 1.0, 1.0], t=5.0)
# -> hazard = 0.119403, mean = 8.375, S = 0.550452   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential maximum likelihood fit: times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3	SOLVED	hazard = 0.230769, mean = 4.33333, S = 0.50042
Exponential maximum likelihood fit: times = [10, 4, 7.5, 12], events = [1, 1, 1, 1], t = 5	SOLVED	hazard = 0.119403, mean = 8.375, S = 0.550452
Exponential maximum likelihood fit: events = [1, 1, 0, 1, 0], t = 3	NEEDS_INPUT	act_exponential_mle needs times (observed times (a list)). Example: solve('act_exponential_mle', times=[1.0, 2.0, 2.0, 3
Exponential maximum likelihood fit: times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: actuarial_science_ai999x.template